FPT SOFTWARE

NVIDIA GR00T Foundation model feasibility



PROJECT REPORT

UNITREE G1 × NVIDIA GR00T

Client
Shimizu Corporation

Period
24/08/2026 – 06/11/2026

Version · Issued
v1.0 · 06/11/2026

CONTENTS

Project report — Unitree G1 × NVIDIA GR00T

PART I BACKGROUND & FOUNDATION 5

I.01 Overview & results 5
- I.01.1 Research problem 5
- I.01.2 Experimental scenarios 6
- I.01.3 Research objectives 7
- I.01.4 Research scope 7
- I.01.5 Research figures 8
- I.01.6 Implementation process 9
- I.01.7 Implementation plan 10

I.02 Infrastructure & performance 11
- I.02.1 Devices & connections 11
- I.02.2 Software & versions 12
- I.02.3 Inference performance 12
- I.02.4 Data collection performance 13
- I.02.5 Training performance 13
- I.02.6 Performance analysis 14

PART II PROBLEM RESEARCH 15

II.01 Model research 15
- II.01.1 Quick glossary 15
- II.01.2 How it works 15
- II.01.3 Input data 16
- II.01.4 Output data 17
- II.01.5 Model architecture 18
- II.01.6 Published results 19
- II.01.7 Version comparison 20
- II.01.8 Experimental checkpoints 21
- II.01.9 Research conclusions 21

II.02 Pipeline research 23
- II.02.1 Overall flow 23
- II.02.2 Environment initialisation 25
- II.02.3 Connection 25
- II.02.4 Getting the observation 26
- II.02.5 Sending the observation 26
- II.02.6 Receiving the action chunk 27
- II.02.7 Processing the action chunk 27
- II.02.8 Running the robot 27
- II.02.9 Running the closed loop 28
- II.02.10 The 12 notebooks and where they appear in the report 28

II.03 Motion research 30
- II.03.1 G1 joint structure 30
- II.03.2 Forward kinematics 31
- II.03.3 Inverse kinematics 32
- II.03.4 Dynamics 32
- II.03.5 Joint rate limit 34
- II.03.6 Physics steps: how motion is computed 34
- II.03.7 Simulation: a car through each physics step 36
- II.03.8 Control ticks and physics steps 37
- II.03.9 Inside one control tick 38
- II.03.10 Simulation time and real time 39

II.04 **Data research** 40
II.04.1 Problem 1: Visual Gap 40
II.04.2 Solution 1: Domain Randomization 40
II.04.3 Problem 2: Sim Data Scarcity 42
II.04.4 Solution 2: Real2sim 42
II.04.5 Problem 3: Teleop Bottleneck 43
II.04.6 Solution 3: Demo Generation 43
II.04.7 Combination and conclusion 44

PART III DEPLOYMENT SCENARIOS 45

III.01 **Scenario Apple & Plate** 45
III.01.1 Video Showcase 45
III.01.2 Scenario setting 46
III.01.3 Scenario goals 47
III.01.4 Scenario data 47
III.01.5 Problems encountered 47
III.01.6 Proposed method 48
III.01.7 Implementation process 49
III.01.8 Results achieved 50
III.01.9 Results analysis 51
III.01.10 Scenario conclusion 53

III.02 **Scenario Multi Fruit** 54
III.02.1 Video Showcase 54
III.02.2 Scenario setting 55
III.02.3 Scenario goals 56
III.02.4 Scenario data 56
III.02.5 Problems encountered 57
III.02.6 Proposed method 57
III.02.7 Workflow 60
III.02.8 Fine-tune 61
III.02.9 Results 61
III.02.10 Result analysis 63
III.02.11 Scenario conclusion 64

III.03 **Scenario Hammer & Bin** 65
III.03.1 Video Showcase 65
III.03.2 Scenario setting 66
III.03.3 Scenario goals 67
III.03.4 Scenario data 67
III.03.5 Problems encountered 68
III.03.6 Proposed approach 68
III.03.7 Workflow 69
III.03.8 Fine-tune 70
III.03.9 Results achieved 71
III.03.10 Result analysis 72
III.03.11 Scenario conclusion 74

PART IV DELIVERY & VERIFICATION 75

IV.01 **Deliverables** 75
IV.01.1 Deliverable package structure 75
IV.01.2 Intended users by role 76
IV.01.3 Detailed content inventory 76
IV.01.4 Separately delivered source data 79
IV.01.5 Licensing and terms of use 79

IV.02 **Environment setup** 81
IV.02.1 Connection architecture 81
IV.02.2 Hardware and software requirements 82

IV.02.3 Place the deliverable package on the workstation 82
IV.02.4 Install the client on the workstation 83
IV.02.5 Install the server on the GPU server 84
IV.02.6 Copy the delivered checkpoints to the server 86
IV.02.7 Set up the connection 87

IV.03 Run the GROOT server 89
IV.03.1 Server configuration per scenario 89
IV.03.2 Start the server 90
IV.03.3 Check the connection from the workstation 91

IV.04 Run the verification 93
IV.04.1 General procedure 93
IV.04.2 Baseline checks without a server 94
IV.04.3 Verification commands per scenario 95
IV.04.4 Read and compare the results 96
IV.04.5 Run interactively in Jupyter 96

IV.05 Video evaluation with VSS 98
IV.05.1 What VSS does in the project 98
IV.05.2 Server requirements 99
IV.05.3 Installing VSS 3.2.1 100
IV.05.4 Post-install configuration and health checks 101
IV.05.5 Three ways to judge, and the results on 99 episodes 102
IV.05.6 Re-running the three methods on 2 episodes (step 9–11 videos) 104
IV.05.7 Web UI 105
IV.05.8 Findings and actions 106
IV.05.9 Day-to-day operation 107

PART V EVALUATION & APPENDIX 108

V.01 Overall evaluation 108
V.01.1 Summary of results 108
V.01.2 Overall findings 109
V.01.3 Lessons learned 110
V.01.4 Future directions 112

V.02 Appendix 114
V.02.1 Glossary 114
V.02.2 References 115

I.01 Overview & results

Controlling the G1 humanoid robot for pick-and-place with NVIDIA GR00T.

More than two months of research and experiments (24/08 – 06/11/2026): studying the GR00T foundation model, **independently reproducing** the NVIDIA Isaac Lab Arena pipeline, creating **real2sim** data from a real G1 robot, and completing the full cycle for a **new task**, from teleop data collection to fine-tuning and evaluation.

CLIENT	PERFORMED BY	PERIOD
Shimizu Corporation	**FPT Software**	**24/08/2026 – 06/11/2026**

ENVIRONMENT
Isaac Sim / Isaac Lab simulation

1 Research problem

The **Unitree G1** humanoid robot must pick up an object by itself and place it in the right spot using only a **camera image** and a **language command**, without pre-programming each motion. The project studies how to hand this job to the **NVIDIA GR00T** foundation model: the model takes the image, the command and the joint state, and generates upper-body motion. Everything is done in Isaac Sim / Isaac Lab simulation.



Four research questions that the three experimental scenarios need to answer:

1 QUESTION

Can GR00T control G1 to pick and place objects?

Yes: 92 % · 66 % · 73 %

Three scenarios →

2 QUESTION

How much data and effort does it take to teach a new task?

200 demos, ~1 hour of fine-tuning

Scenario 3 →

3 QUESTION

How do results change when the environment differs from the one used for data recording?

92 % → 56 % → 0 %

Scenario 1 →

4 QUESTION

Can real-robot data go through simulation and back to the real robot (real2sim2real)?

Real2sim: 66 % · sim2real: phase 2

Scenario 2 →

2 Experimental scenarios

Three scenarios of increasing difficulty: from **reproducing** NVIDIA's existing pipeline, to **real-robot data** transferred into simulation, and finally a **completely new task** built by the project itself.



SCENARIO 1

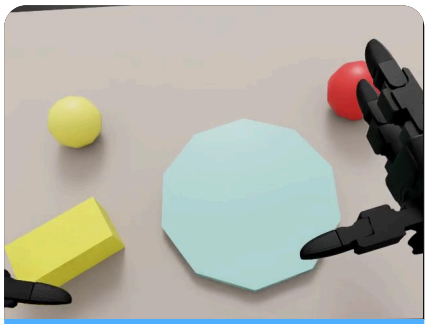
Apple → plate

`"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."`

PURPOSE Independently reproduce NVIDIA's sample pipeline

DATA 208 NVIDIA demos, ready-made model

SUCCESS Apple lies on the plate, < 8 cm from the center, at rest



SCENARIO 2

4 fruits

`"Pick up the green grapes and place it on the plate"`

PURPOSE Pick the right fruit per the command among distractor fruits

DATA 790 real2sim episodes from real G1 data

SUCCESS Fruit is lifted, lies on the plate, other fruits not displaced



SCENARIO 3

Hammer → bin

`"put the hammer in the bin"`

PURPOSE Teach a completely new task, end to end

DATA Self-collected via Meta Quest teleop: 200–400 episodes

SUCCESS Hammer in the bin, at rest, hand released

3 Research objectives

OVERALL OBJECTIVE

Assess the ability to use **GR00T** to control the **G1** robot for pick-and-place (question 1), and determine how much data and effort it takes to bring the model to a new task. The overall objective is broken down into the 6 specific objectives below.

1 FOUNDATION

Master GR00T and the control pipeline

Understand the architecture and fine-tuning workflow from GR00T N1 to N1.7; rebuild the GR00T ↔ G1 flow ourselves (client, safety layer, AGILE balance, Isaac Lab backend), without depending on NVIDIA's ready-made framework.

II.01–II.03

2 SCENARIO 1

Reproduce NVIDIA's results

Run the apple → plate checkpoint on the self-built pipeline and compare against the original environment to validate the pipeline before using it for new tasks.

III.01

3 SCENARIO 2

Leverage real-robot data

Convert real G1 teleop data (joint angles and video only) into a simulation dataset via real2sim; fine-tune the model to pick the right fruit per the command.

III.02 Question 4

4 SCENARIO 3

Teach a completely new task

The full "hammer → bin" cycle: build the environment, teleop with a VR headset, generate additional data, fine-tune, evaluate; record the number of demos and time required.

III.03 Question 2

5 VISUAL GAP

Test and analyze visual differences

Measure how the success rate changes when the images the model receives differ from those at data recording: between simulation scenes and between real and rendered images; identify the factors causing the drop and propose ways to narrow it, such as domain randomization.

Scenarios 1, 2

Questions 3, 4

6 CROSS-CUTTING

Quantitative, reproducible evaluation

Closed-loop scoring with clear success conditions, ≥ 50 episodes per configuration, Wilson 95 % confidence intervals, failure analysis by phase; each run changes only one factor.

III.01–III.03

4 Research scope

All experiments take place in simulation; the pipeline is designed so that the backend can be swapped for a

real robot in a later phase.

IN SCOPE

- ✓ Isaac Sim 6.0 / Isaac Lab 3.0 simulation, Isaac Lab Arena 0.2.1
- ✓ G1 standing in place, two arms + Dex3 hands
- ✓ GR00T N1.7-3B: NVIDIA checkpoint and self fine-tuned
- ✓ Data: NVIDIA demos, public real G1 dataset (real2sim), self-collected teleop in simulation
- ✓ Fine-tuning on 1 H100 GPU

OUT OF SCOPE · LATER PHASE

- – Deployment on the real G1 robot
- – Combined locomotion and manipulation
- – Training a model from scratch
- – Data collection on the real robot
- – Large-scale multi-GPU training

5 Research figures

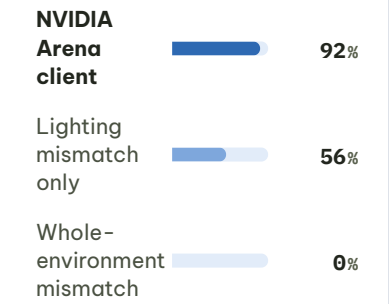
The best result of each scenario, together with the runs that led to that figure. Click each scenario to see the details.

SCENARIO 1

Apple → plate

92

original NVIDIA environme



The model is highly sensitive to visuals: same checkpoint, and changing only the environment moves the result from 92 % down to 0 %.

SCENARIO 2

4 fruits

66%

33/50 · 95 %: 52–78 %

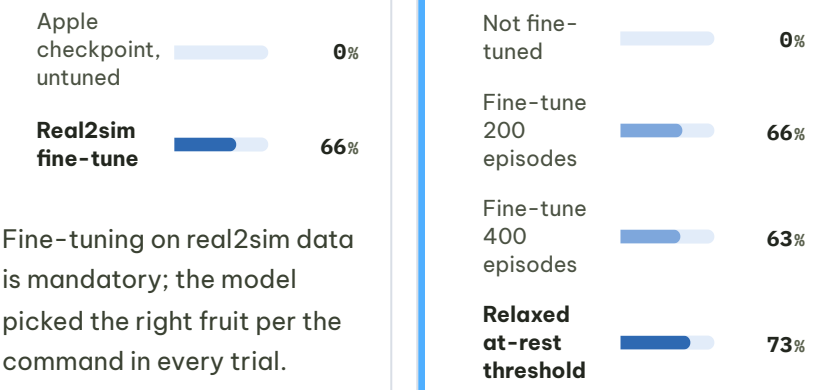
Fine-tuning on real2sim data is mandatory; the model picked the right fruit per the command in every trial.

SCENARIO 3

Hammer → bin

73%

73/100 · 95 %: 64–81 %



A new task from 0 % to 73 % with only self-collected data and 1–2 hours of training per run (61 min with mix200, 2 h 15 min with mix400).

6 Implementation process

The project follows three steps: build a **common foundation**, run **three scenarios** on that foundation, then use the evaluation results to **improve**. Each scenario goes through the same four stages (data source → core work → model → closed-loop evaluation) and differs in the core work: scenario 1 reuses NVIDIA's data and model, with the focus on **studying and rewriting the flow** of GR00T ↔ G1; scenario 2 creates its own simulation data from real-robot data, with the focus on **real2sim**; scenario 3 builds a new task from scratch, with the focus on **data augmentation** from teleop demos. The dark cells in the diagram mark that core work.

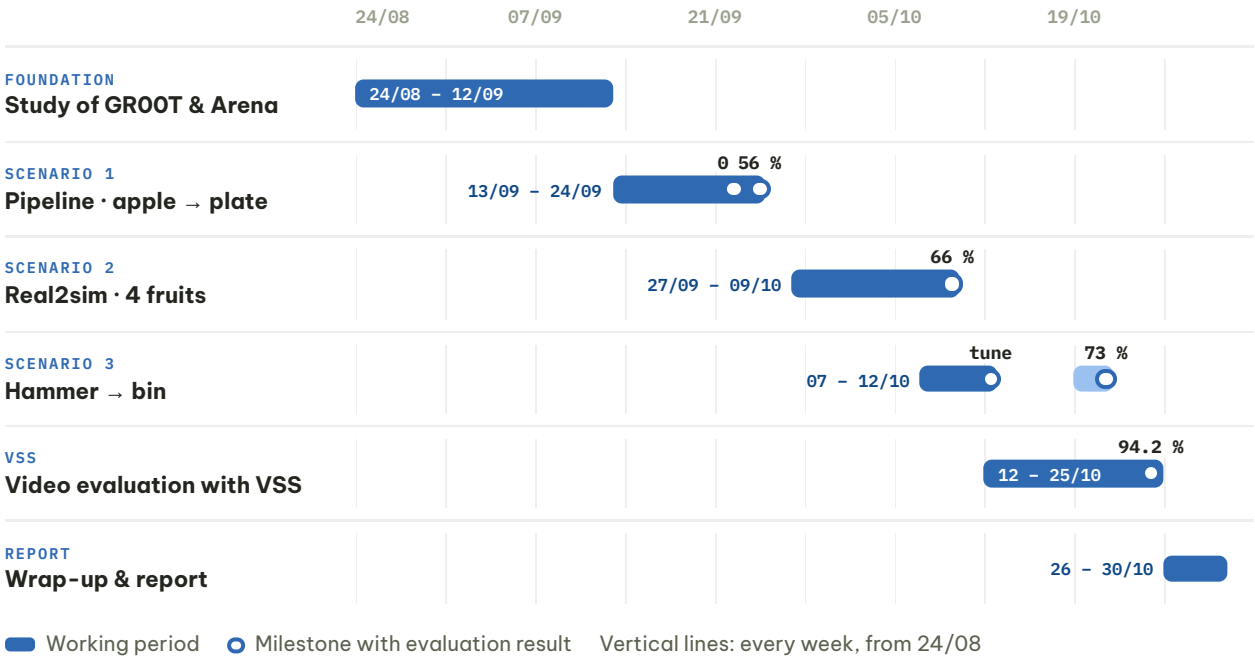
COMMON FOUNDATION — Study of GR00T N1 → N1.7 · Isaac Lab Arena · independent GR00T ↔ G1 pipeline (client, safety layer, AGILE, Isaac Lab backend)

	DATA SOURCE	CORE WORK	GR00T N1.7 MODEL	CLOSED-LOOP RESULT
SCENARIO 1 **Study & rewrite the flow · apple → plate**	NVIDIA **Static apple dataset** 208 XR teleop episodes in simulation	REWRITE THE FLOW **12 notebooks → independent pipeline** Open-loop validation, demo replay	READY-MADE **NVIDIA checkpoint** Already fine-tuned for apple → plate	ORIGINAL ARENA ENVIRONMENT **92 %** Rebuilt Galileo 56 % · self-built scene 0 %
SCENARIO 2 **Real2sim · 4 fruits**	REAL G1 ROBOT **1,095 teleop trajectories** Joint angles + video only	REAL2SIM **790 simulation episodes** FK → scene, distractors, replay, filtering	FINE-TUNE · H100 **Multi-task 4-fruit model** 20,000 steps · 2 h 34 min	50 EPISODES **66 %** Apple checkpoint 0 % · 0 wrong picks
SCENARIO 3 **Data augmentation · hammer → bin**	META QUEST TELEOP **100 teleop demos** Collected in Isaac Lab, self-built environment	DATA AUGMENTATION **mix200 · mix400** 100 teleop + 100 generated; mix400 adds 200 teleop	FINE-TUNE · H100 **Hammer → bin model** mix200 ~1 h · mix400 ~2 h 15 min	100 EPISODES **73 %** Untuned 0 % → 66 % → 73 %

↺ **IMPROVEMENT LOOP** — Failure analysis by phase → fix the environment or the scoring, add data → retrain and re-evaluate. Examples: rebuilding the Galileo scene (scenario 1), fixing the camera not rendering every step (scenario 2), mix200 → mix400 and at-rest threshold 0.01 → 0.02 (scenario 3).

7 Implementation plan

The project runs as **three consecutive scenarios** on the same foundation, each scenario taking about two weeks. The two weeks before the end (12–25/10) evaluate episode videos with VSS (section IV.05), in parallel with the end of scenario 3; the last week (26–30/10) is for wrapping up and writing the report. The chart below shows the working periods and the milestones with results; details of each scenario are in the sections of part III.



I.02 Infrastructure & performance

Two machines, one VR headset — and the numbers that limit the system.

Simulation, teleop and evaluation run on **one workstation**; training and model serving run on **one remote H100 GPU server**. This section describes the configuration used, the software to install on each machine, and the measured performance figures — especially **model latency**, the most important limit when moving to the real robot.

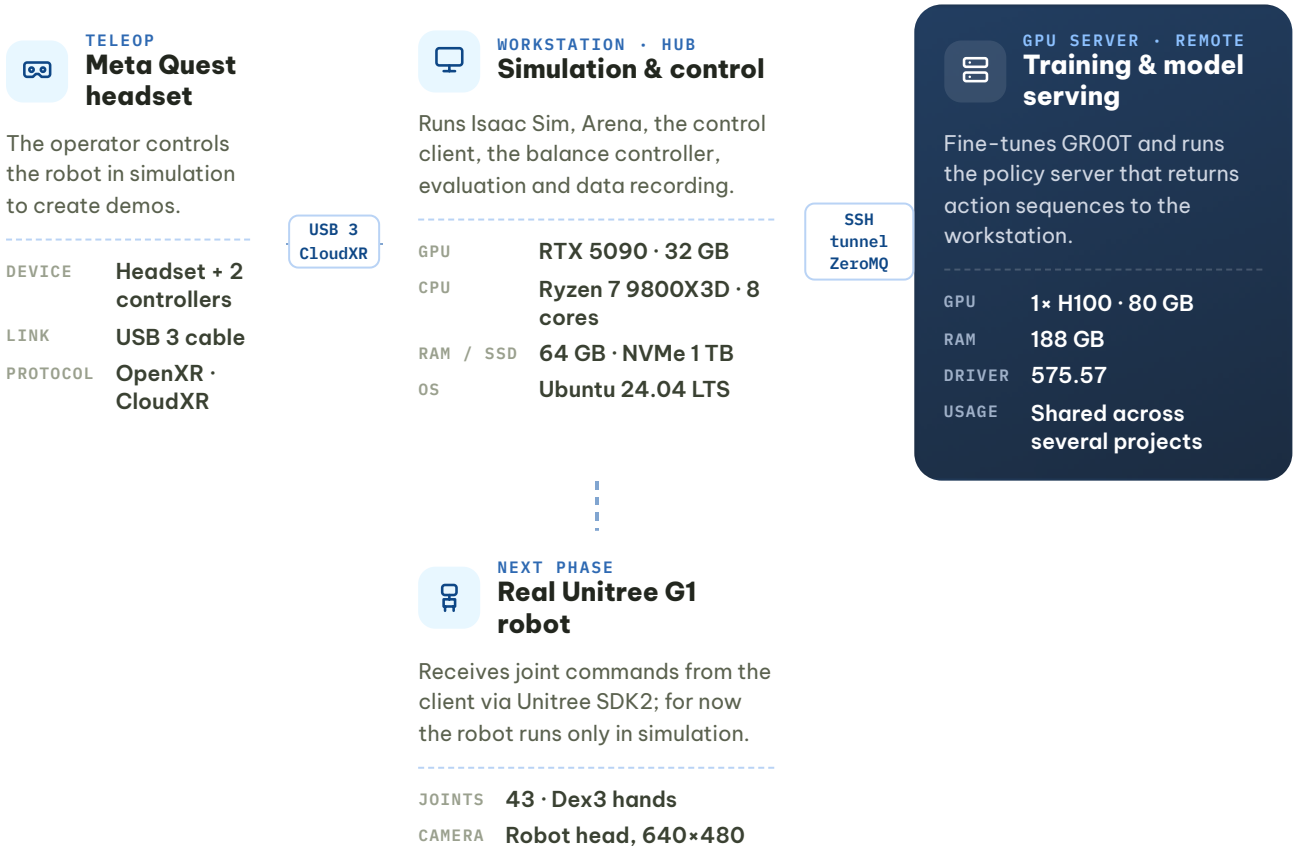
WORKSTATION
RTX 5090 · Ryzen 7 9800X3D · 64 GB

SERVER
1× H100 80 GB · 188 GB

SIMULATION
Isaac Sim 6.0.1 · Isaac Lab 3.0-beta2

1 Devices & connections

The workstation is the hub: it receives operator input from the VR headset, runs the robot simulation, and sends observations to the GPU server to get back action sequences.



2 Software & versions

Each column shows the software layers of one machine, from the platform (bottom) up to the project's applications (top, highlighted).

Workstation — SIMULATION

g1gr00t client + AGILE
numpy, pyzmq, msgpack, opencv, onnxruntime — `Python 3.10+`

Isaac Lab Arena
Runs in Docker 29; separate branch for the hammer task — `0.2.1`

Isaac Lab
Official pairing with Isaac Sim 6.0 — `3.0.0-beta2`

Isaac Sim
Python 3.12, local install — `6.0.1`

NVIDIA driver + CUDA
Version 580 is enough for Isaac Sim 6.0; Isaac Sim 6.1 will require ≥ 595 — `580 · 12.6`

Ubuntu 24.04 LTS
Operating system — `LTS`

Also: Jupyter, pytest, pandas, matplotlib for notebooks and testing.

GPU server — MODEL

GR00T N1.7-3B + checkpoint
ZeroMQ policy server; 12.6 GB per checkpoint — `3B`

Cosmos-Reason2-2B
Vision-language backbone, downloaded from Hugging Face — `2B`

Isaac-GR00T
Python environment via uv; the exact commit tested with Arena — `4b1dca9`

Linux + NVIDIA driver — `575.57`

NVIDIA's model license must be accepted on Hugging Face.

Real robot — NEXT PHASE

g1gr00t client
Reused as-is; only a real-robot backend is added — `Python 3.10+`

Unitree SDK2
unitree_sdk2py package, already declared in the configuration — `–`

On-robot computer
Or a computer placed next to the robot

Not deployed in this phase.

3 Inference performance

How many times per second the robot receives commands, how many actions each model call yields, and how long the model takes to respond.

CONTROL FREQUENCY
50 Hz
Physics 200 Hz, 4 steps per tick · real2sim 20 Hz

PER MODEL CALL
40 steps
= 0.8 s of motion · real2sim 16 steps, 8

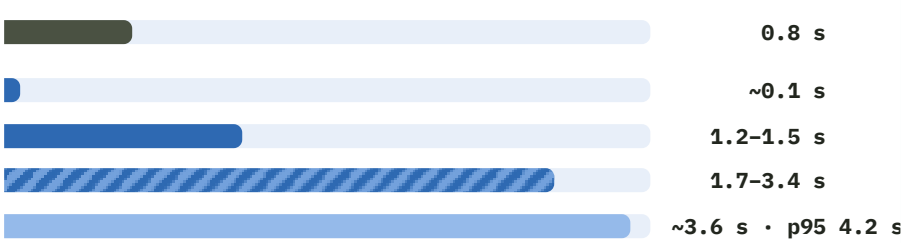
SERVER INFERENCE
~100 ms
Model computes one 40-step sequence on the

MODEL LATENCY
~1.2 s
Median per call during evaluation, via SSH

executed | H100 | tunnel to the H100

Time budget: motion vs. model latency

seconds · scale 0–4 s



The model responds **more slowly** than the time the robot needs to finish one sequence. In simulation this is fine because the simulation pauses and waits; on the real robot, the robot will stall between sequences unless the next call is made in advance and the GPU is placed close to the robot. The model itself takes only ~100 ms; most of the remaining latency lies in the SSH tunnel link and data serialization.

4 Data collection performance

Measured while recording hammer-task demos with the Meta Quest headset over CloudXR.

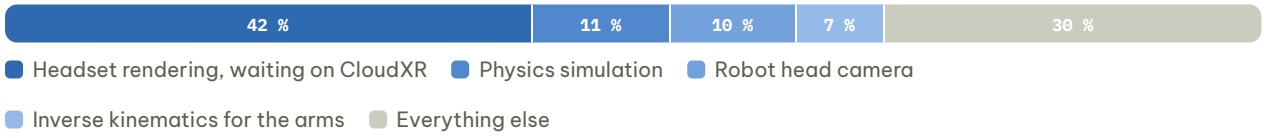
HEADSET RENDERING
42 %
The largest share of a loop, including waiting on CloudXR

PHYSICS SIMULATION
11 %
Robot head camera 10 %, inverse kinematics 7 %

RAM
~12 GB
Grows ~15 MB/s while recording, freed when the demo is saved

Where a teleop loop spends its time

measured while recording hammer-task demos



The teleop bottleneck is the video stream to the headset, not the simulation. The process uses ~12 GB of RAM and grows ~15 MB/s while recording (camera image buffer, freed when the demo is saved).

5 Training performance

Fine-tuning GR00T N1.7 on one H100 GPU and preparing the training data.

FINE-TUNE TIME
61 min
10,000 steps · 0.36 s/step · batch 32

GPU MEMORY
≈ 53 GB
LLM frozen; vision, projector, diffusion tuned

CHECKPOINT
12.6 GB
Weights only; ≈ 27 GB with optimizer

METRIC	VALUE	NOTES
Training speed	0.36 s / step	Batch 32, 1× H100 (2.9 s/step when the GPU is occupied by other jobs)
GPU memory during training	≈ 53 GB	LLM frozen; vision, projector, diffusion tuned
10,000 training steps	61 min	Loss 1.27 → 0.020
Checkpoint size	12.6 GB	Weights only; ≈ 27 GB with optimizer state
Augmented data generation	100 demos / 17 min	133 runs, 100 successful runs kept
Data size	131 MB / 13 GB	200 hammer episodes in LeRobot format / original HDF5 with images
Automated testing	84 unit tests	In-simulation tests: ~3.5 min

6 Performance analysis

Reading the three cards above together: at each stage, where time and resources are actually spent, and what that means for the next phase.

INFERENCE

The bottleneck is the link, not the model

The model computes in only ~100 ms, yet each call takes ~1.05–1.3 s, longer than one action sequence (0.8 s). The gap lies mostly in the SSH tunnel and data serialization. When 4 robots call at once, latency rises to ~3.6 s (p95 4.2 s) because the server processes requests one after another.

DATA COLLECTION

The bottleneck is the video to the headset

Rendering and waiting on CloudXR take 42 % of each loop, nearly 4 times the physics simulation (11 %). RAM grows ~15 MB/s while recording, so each demo should be saved regularly rather than recording one long continuous session.

TRAINING

One H100 is enough, if it is not shared

10,000 fine-tune steps take 61 min and use ≈ 53 GB of GPU memory, which fits one H100. When the GPU is occupied by other jobs, speed drops nearly 8 times (2.9 s/step vs. 0.36 s/step).

Improvement directions: place the GPU server near the robot or on the same local network, and request the next sequence in advance so the robot does not stall; use a wired connection for the headset during data collection; dedicate a GPU to fine-tuning runs.

II.01 Model research

GR00T N1 and the N1.7 version used in the project.

GR00T N1 is an open VLA model for humanoid robots, made of a vision–language model to **understand** and a Diffusion Transformer to **act**. The project uses its successor, **GR00T N1.7-3B**. This section covers, in order: what the model does, what it is made of, whether it is effective, and how the project applies it.

PAPER
arXiv:2503.14734 (03/2025)

PROJECT MODEL
GR00T N1.7-3B

KEYWORDS
VLA · flow matching · data pyramid

1 Quick glossary

Token
A small piece of an image or text after encoding; the model reads a sequence of tokens instead of the raw image or text.

VLA (Vision-Language-Action)
A model that looks at an image, reads an instruction and outputs actions for the robot.

System 2 / System 1
The understanding part (slow) and the acting part (fast) of GR00T.

Action / action chunk
An action is a set of target joint angles for one step; an action chunk is the upcoming segment of motion.

Flow matching
How the action chunk is generated: start from random noise and refine it step by step into motion.

Embodiment
One type of robot body (G1, GR-1…); each type has its own encoders.

Pre-train / fine-tune
Learning first on large general data / further tuning for a specific robot and task.

Checkpoint
The trained model weights, saved as a file for reuse.

Teleop
A person controls the robot remotely (here with a VR headset) to record demos.

2 How it works

The video below illustrates how a **Vision-Language-Action** model such as GR00T turns a camera image and an instruction into motion. Click a step on the right to jump the video to the matching part; each step

notes how the project applies it.



7 STEPS · CLICK TO SEEK THE VIDEO

From image and instruction to motion

0:00 IMAGE TOKENS

Camera image becomes image tokens

The image is split into small patches, passed through a vision encoder (trained on image–caption pairs) and then projected into the language model's space.

> **Project:** G1 head camera image 640×480; GR00T N1.7 uses the SigLIP-2 encoder.

0:12 TEXT TOKENS

Instruction becomes text tokens

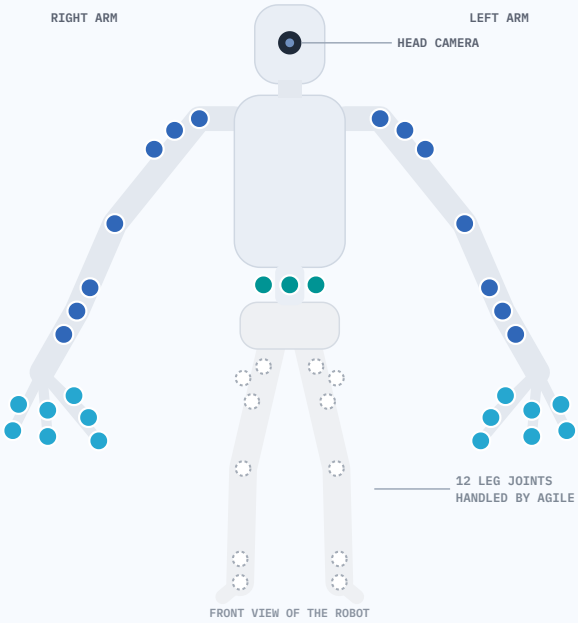
An instruction such as "pick up the dessert item" is split into word pieces.

> **Project:** e.g. "put the hammer in the bin"; the instruction must match the wording used when the data was recorded exactly.

3 Input data

On each call, the model receives three kinds of data from **the current moment only**: one camera image, the upper-body joint angles and the instruction. The model receives no history of previous frames, and does not see the 12 leg joints (the legs are handled by AGILE).

1	**head camera image** 640×480, current frame	
31	**upper-body joint angles** arms, fingers, waist	
1	**instruction** unchanged for the whole episode	



STATE · G1 ROBOT

Blue dots: 31 joints sent to the model · white dots: 12 leg joints controlled by AGILE



ROBOT'S VIEW · HEAD CAMERA

First frame of scenario 1

INSTRUCTION SENT

```
"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
```

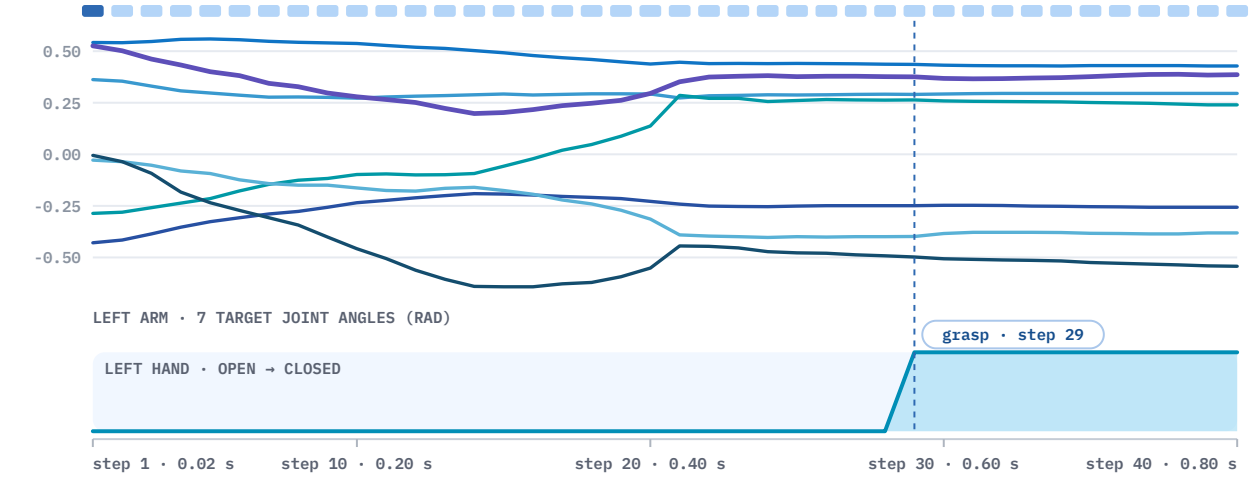
> **In the project:** scenarios 1 and 3 use the `ego_view` camera as above; scenario 2 (real2sim) uses the `rs_view` camera with the same 31 joints.

4 Output data

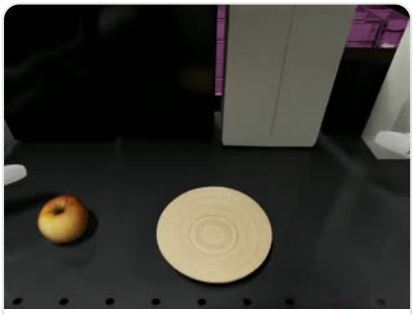
On each call, the model returns **a chunk of 40 steps** for the near future, each step with 35 values. The robot executes one step per 20 ms tick, so one chunk covers 0.8 s of motion before a new chunk is needed.

40 **steps per chunk** 20 ms per step = 0.8 s of motion	**35** **values per step** 31 joint angles + 4 leg commands	**1** **chunk per call** called again when the chunk is nearly used up

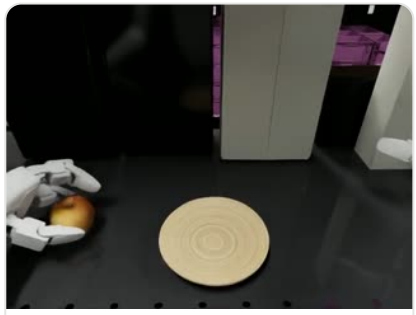
EXAMPLE · ONE 40-STEP CHUNK FROM THE APPLE → PLATE DEMO (EPISODE 0, FROM SECOND 0.4)



Shoulder pitch · Shoulder roll · Shoulder yaw · Elbow · Wrist roll · Wrist pitch · Wrist yaw · Left-hand grip


Step 1 · hand moves towards the apple


Step 21 · hand reaches the apple


Step 40 · apple grasped

This is an action chunk recorded in the demo data, in the same form as the chunk the model returns. In open-loop checks, the deviation of the model's predicted chunk for the same moment from this chunk is 10–70 times smaller than the motion amplitude.

ONE STEP · 35 VALUES

Left arm	Right arm	Left hand	Right hand	Waist	Locomot…	Height
7	7	7	7	3	3	1

31 target joint angles → sent straight to the joints · 4 commands → passed to AGILE

ABSOLUTE ANGLES

The 31 joint values are target angles, not changes from the previous step. Before use, the client still clamps them to the joint limits and the per-tick velocity limits (II.02, card 7).

LEG COMMANDS

The model does not control the 12 leg joints directly. It only outputs locomotion commands and the hip height (command 0.72 m); AGILE uses them to keep balance. All three project scenarios stand still in place.

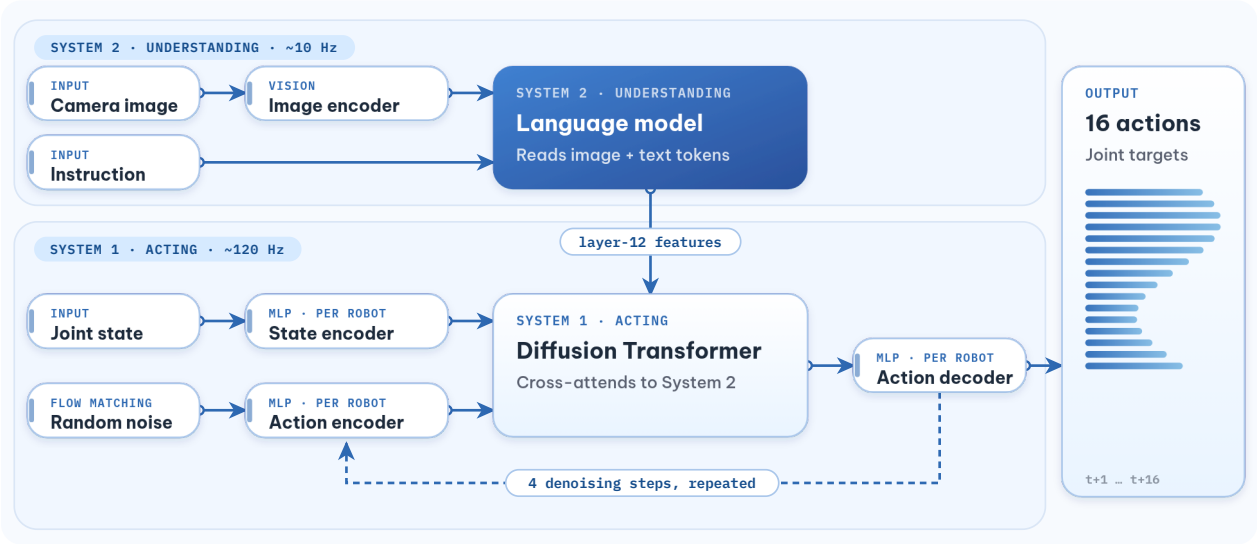
In the project: scenarios 1 and 3 use exactly the 40-step chunk at 50 Hz as above. Scenario 2 fixes the hip, so there are no leg commands: a 16-step chunk at 20 Hz (also 0.8 s), 31 values per step, 8 steps executed per call.

5 Model architecture

The understanding part (called **System 2** in the paper) is a vision–language model that runs slowly, about 10 times per second. The acting part (**System 1**, called the "action expert" in the video) takes the output of the understanding part together with the robot's current joint angles, and runs much faster. Click each block in the diagram to see what it does.

DATA FLOW IN GR00T N1

DIAGRAM · FIGURES FROM THE PAPER



SYSTEM 2 · UNDERSTANDING

Language model

Reads the image tokens and the instruction together to understand the scene and the request. Features from layer 12 are sent to the acting part. Runs about 10 times per second.

In the project: N1.7 uses Cosmos-Reason2-2B; this part is frozen during fine-tuning.

In the project: GR00T N1.7 keeps this two-part structure; the understanding part is replaced by a newer model (card 7). The inputs are the G1 head camera image and 31 upper-body joint angles. The three MLP blocks (state encoder, action encoder, action decoder) are specific to each robot type: this is the part that has to be learned when bringing G1 into the model.

TOTAL PARAMETERS

UNDERSTANDING PART

SPEED

2.2 B
GR00T N1, the version in the paper

1.34 B
Most parameters sit in the vision-language model

10 · 120 Hz
Understanding part ~10 Hz, control loop ~120 Hz

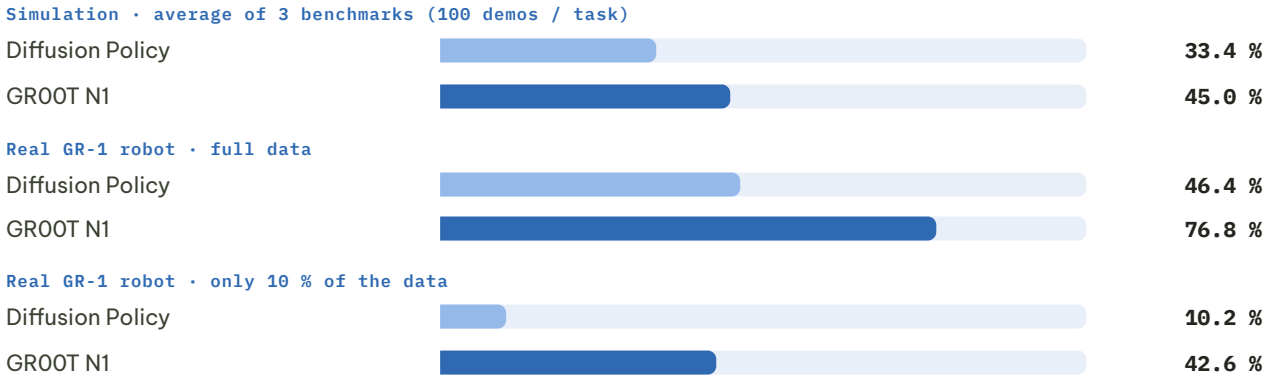
Technical details · paper –

- Understanding part: Eagle-2, consisting of the SigLIP-2 image encoder and the SmolLM2 language model. A 224×224 image is compressed to 64 tokens (pixel shuffle).
- Features are taken from layer 12 of the language model instead of the last layer: the paper reports this is both faster and more successful.
- Acting part: Diffusion Transformer (DiT), interleaving cross-attention (attending to the understanding part's tokens) and self-attention (between state tokens and action tokens).
- States and actions pass through MLP encoders/decoders specific to each robot type (embodiment).

6 Published results

The paper compares GR00T N1 with Diffusion Policy, a model that learns only from the task's own data, without pre-training. GR00T N1 is better in every configuration, with the largest gap when data is scarce.

Success rate: Diffusion Policy vs. GR00T N1 — paper figures



In the project: this is why a few hundred demos are enough to fine-tune a new task for G1, and why real2sim data and generated data are good enough.

Detailed table per simulation benchmark · table 4 of the paper –

BENCHMARK	DIFFUSION POLICY 30 / 100 / 300 DEMOS	GR00T-N1-2B 30 / 100 / 300 DEMOS
RoboCasa (24 tasks)	14.7 / 25.6 / 43.2	**17.4 / 32.1 / 49.6**
DexMimicGen (9 tasks)	23.7 / 46.9 / 68.4	**29.6 / 58.5 / 74.2**
GR-1 Tabletop (24 tasks)	21.3 / 32.7 / 40.4	**43.2 / 50.0 / 49.3**

Average success rate (%). GR00T N1 leads in every cell; only GR-1 Tabletop barely improves from 100 to 300 demos.

7 Version comparison

The figures in card 6 are for N1 on the Fourier GR-1 robot. All project experiments use N1.7 on the Unitree G1.

ASPECT	GR00T N1 (PAPER)	GR00T N1.7 (PROJECT)
Scale	~2.2B parameters	~3B parameters
Understanding part (System 2)	NVIDIA Eagle-2	Cosmos-Reason2-2B (Qwen3-VL architecture), images at native aspect ratio
Acting part (System 1)	DiT + flow matching	Unchanged
Multi-robot use	Separate MLP per robot	Unchanged; the state is padded to a fixed length

8 Experimental checkpoints

The three checkpoints used in the three scenarios all start from GR00T N1.7-3B; they differ in who fine-tuned them and with what data.

CHECKPOINT	TASK	SOURCE	FINE-TUNE DATA	SUCCESS
Scenario 1 `GN1x-Tuned-Arena-G1-Static-PickNPlace`	Apple → plate	NVIDIA, reused	208 teleop demos in simulation	92 % original Arena 56 % reconstructed Galileo 0 % self-built scene
Scenario 2 `g1_fruits_n17`	Pick the correct 1 of 4 fruit types	Fine-tuned by the project	790 real2sim episodes from real G1 data	66 % (33/50)
Scenario 3 `hammer_mix200_n17`	Hammer → bin	Fine-tuned by the project	200 episodes: teleop + generated	66 % (66/100)
Scenario 3 `hammer_mix400_n17`	Hammer → bin	Fine-tuned by the project	400 episodes: mix200 + 200 new teleop demos	63 % (63/100); 73 % with a relaxed at-rest threshold

> **License:** N1.7 is under the NVIDIA Open Model Agreement; the Arena checkpoint states "non-commercial use".

9 Research conclusions

What the project learned after using GR00T N1.7 across all three scenarios.

STRENGTHS

Teaches new tasks fast, with little data

- A few hundred demos are enough: the hammer task went from 0 % to 66–73 % with 200–400 episodes (section III.03).
- Fine-tuning takes ~1 hour on one H100, and runs on a single GPU.
- Understands natural instructions: the 4-fruit model picks the right fruit as instructed, never grabbing the wrong one (section III.02).
- One model controls the arms, hands and waist at the same time.

LIMITATIONS

Sensitive to visuals, must be tuned per task

- Scene-dependent: the apple task drops from 92 % to 56 % when the lighting differs, and to 0 % when the whole scene differs (section III.01).
- Not reusable for other tasks: the apple checkpoint scores 0 % on the 4-fruit task.
- Each call takes ~1.05–1.3 s over the network (the model computes for ~100 ms), longer than a 0.8 s motion chunk (section I.02).
- NVIDIA's Arena checkpoint states "non-commercial use".

Conclusion: GR00T is a suitable foundation for G1 pick-and-place tasks. The deciding factor for results is not the model but the data: collect it in the actual working environment, or make it visually diverse enough.

Reproducing each step of the GR00T ↔ G1 flow with research notebooks.

To avoid depending on NVIDIA's ready-made framework, the project **rebuilt the entire flow itself**, from camera image to joint command. Each step was studied in its own **notebook** — written by hand, inspecting the data, checking against a reference — before being packaged into tested source code. The `nbXX` tag in each card's PROCESSING box names the notebook (in `codes/standalone_g1_gr00t/notebooks/`) where that step was written and verified. The package contains 12 notebooks: the 7 that build the pipeline are covered in this section, and the other 5 use that pipeline to verify and evaluate the scenarios in Part III. Card 10 lists all 12 notebooks and where each one appears in the report.

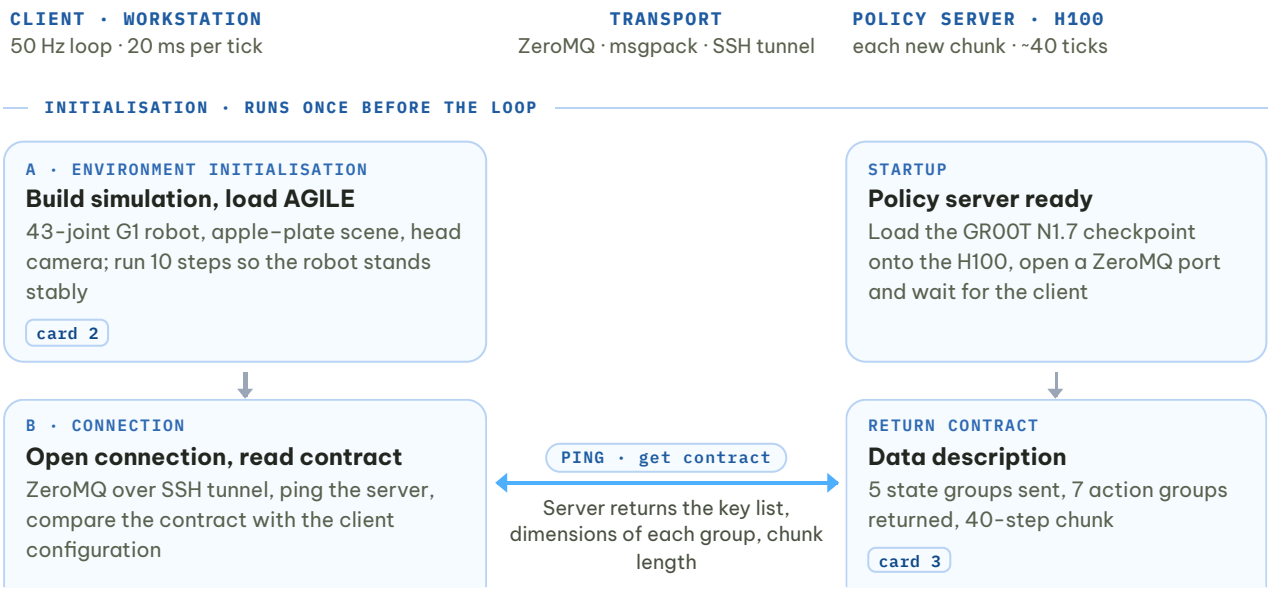
NOTEBOOKS
12 notebooks · 7 here, 5 in Part III

SOURCE CODE
Package g1gr00t · 84 unit tests

KEYWORDS
notebook → package · layer-by-layer verification

1 Overall flow

Before running, the client builds the simulation environment and connects to the server once. After that, every control loop passes through two machines. The **client** on the workstation runs the simulation and every data-processing step: reading sensors, packaging, decoding, safety checks, sending commands to the joints. The **policy server** on an H100 GPU only runs the model. The client runs every 20 ms but calls the server only when it needs a new chunk, i.e. about once every 40 ticks. The card label shows which card presents that step.



card 3

CONTROL LOOP · REPEATS UNTIL THE EPISODE ENDS

1 · READ SENSORS
G1 robot in simulation
640×480 head camera image, 43 joint angles, torso IMU
card 4

2 · PACKAGE
Build observation
43 → 31 upper-body joints by joint name, add the instruction, check dimensions
card 5

SEND · get_action
0.92 MB image + 31 joint angles + instruction

RECEIVE
Contract check
Match the name and dimensions of each group against the checkpoint configuration
card 3

3 · WAIT
Wait for a new chunk
The simulation pauses and waits; on a real robot the call must be made in advance to avoid stalling
card 7

INFERENCE
GR00T N1.7
The understanding part reads image + instruction; DiT denoises in 4 steps into a motion chunk
~100 ms

4 · DECODE & SAFETY
Check the chunk
Split joint groups, block NaN/Inf, clamp to joint, locomotion and height limits
card 6 card 7

RECEIVE · action chunk
40 steps × 35 values, 6.8 KB; full call round trip ~1.05–1.3 s

RETURN
40-step chunk
31 upper-body joints + 3 locomotion + 1 height for each step

5 · SCHEDULING
Take one step per tick
40 steps = 0.8 s of motion; prefetch the next chunk when the current one is about to run out
card 7

6 · RATE LIMITING
Smooth the command
Each tick the arm changes by at most 0.2 rad, especially at the boundary between two chunks
card 7

7 · ACTUATION
Send commands to the joints
31 upper-body joints from GR00T, 12 leg joints from AGILE, then 4 physics steps
card 8

repeats every 20 ms tick

WHILE THE ROBOT RUNS 40 STEPS

The server is idle. The client only calls again when the chunk is about to run out; with prefetching (card 7), the observation is sent early so the new chunk arrives in time and the robot does not have to stop.

For each call, most of the time is spent in transport: the model computes for ~100 ms while the full call round trip takes ~1.05–1.3 s.

BEFOREHAND · OFFLINE

Demos → LeRobot dataset → fine-tune on H100 → checkpoint loaded into the policy server.

AFTER EACH EPISODE

Reset, then run the next episode; results of N episodes are aggregated into a success rate (card 9).

2 Environment initialisation

INPUT

- G1 robot (USD, 43 joints)
- Scene, apple, plate
- AGILE policy (ONNX)

PROCESSING nb05 nb04

- Build the simulation: ground, lighting → robot → head camera → objects → finger friction
- Record the initial state, run 10 steps to settle
- Load AGILE, reset the LSTM state

OUTPUT

- **Robot standing in the default pose**
- **5 ms physics step, 20 ms control tick**

RESULTS ACHIEVED

ROBOT STABLE

0.786 m

pelvis height after 10 steps, then held steady

JOINT ORDER

43 / 43

simulation joints match the order table we built

SCENE BUILD

~4 s

after pre-downloading 1.3 GB of assets (first time > 15 minutes)

3 Connection

INPUT

- Policy server address (via SSH tunnel)
- Checkpoint loaded on the H100

PROCESSING nb01

- Open a ZeroMQ connection, ping the server
- Read the data contract from the server
- Compare with the client configuration before running

OUTPUT

- **Connection ready**
- **Key list, dimensions of each group, chunk length**

RESULTS ACHIEVED

CONTRACT READ FROM SERVER

5 → 7 groups

5 state groups sent, 7 action groups returned

CHUNK LENGTH

40 steps

= 0.8 s of motion at 50 Hz

UNRESPONSIVE PORT

0.5 s

returns a clean error, re-creates the connection

4 Getting the observation

INPUT

- Simulation state after the tick just run

PROCESSING nb05 nb02

- Render the head camera
- Read 43 joint angles and velocities, torso IMU
- Reorder joints by name across 4 ordering systems

OUTPUT

- 640×480 image
- 43 joint angles + IMU (for AGILE)
- 31 upper-body joint angles (for GR00T)

RESULTS ACHIEVED

JOINT ORDERING SYSTEMS
4 systems
simulation, WBC, model, dataset

ROUND-TRIP CONVERSION
0 error
simulation → WBC → simulation loses no values

FRESH IMAGE
1 per tick
rendered at the last physics step; without this step the model sees a frozen image

5 Sending the observation

INPUT

- 640×480 image
- 31 upper-body joint angles
- Instruction

PROCESSING nb02 nb01

- Package according to the contract, check dimensions
- Send get_action over ZeroMQ (msgpack)
- Server runs GR00T N1.7

OUTPUT

- 0.92 MB request to the server
- Server generates a 40-step chunk

RESULTS ACHIEVED

LATENCY PER CALL
~1.05 s
1 robot · p95 1.39 s

MODEL COMPUTE
~100 ms
compute time on the server; most of the rest is in transport

4 ROBOTS AT ONCE
~3.6 s
sent together as one batch · p95 4.2 s

The instruction sent affects the returned chunk

left-arm error, rad



The Arena default instruction gives a larger error than even the nonsense instruction, so the client sends the dataset instruction verbatim.

6 Receiving the action chunk

INPUT

- Server response: 7 keys, 6.8 KB

PROCESSING nb01 nb02

- Decode msgpack
- Split 31 joint angles (40×31), locomotion (40×3), height (40×1)
- Reorder into the client joint order by name

OUTPUT

- 40-step × 35-value chunk in the client

RESULTS ACHIEVED

PER RECEIPT
40 × 35 values
40 steps, 35 values per step

SIZE
6.8 KB
~135 times smaller than the outgoing request

RETURNED GROUPS
7 keys
2 arms, 2 hands, waist, locomotion, height

7 Processing the action chunk

INPUT

- The 40 × 35 chunk just received

PROCESSING nb08 nb02

- Block NaN/Inf, clamp to joint, locomotion and height limits
- Scheduling: take one step per tick, call ahead before running out
- Limit the change per tick

OUTPUT

- Safe command for the current tick: 31 joint angles + 4 leg commands

RESULTS ACHIEVED

OUT OF LIMITS
0 / 1,240
model-returned values exceeding joint limits, in one test chunk

DELIBERATE TEST
81 values clamped
1 joint angle, 40 locomotion commands, 40 height commands

WAIT TIME
0.28 s
with prefetching, versus 0.84 s with sequential calls

8 Running the robot

INPUT

- 31 target joint angles
- Locomotion + height commands
- Current observation (for AGILE)

PROCESSING nb04 nb05

- AGILE computes 12 leg joint angles from an 80-value vector
- Write 43 target angles to the simulation
- Run 4 physics steps of 5 ms

OUTPUT

- Robot moves for one tick (20 ms)
- New state for the next tick

RESULTS ACHIEVED

AGILE VS REFERENCE

50 ticks

bit-identical, same LSTM state

STANDING STABLY

10 s

500 ticks, maximum tilt 9.8°

DEMO REPLAY

121 ticks

without the model: success, apple 2.9 cm from the plate centre

9 Running the closed loop

INPUT

- Checkpoint, scene, instruction
- The steps in cards 4–8

PROCESSING nb06 nb07 nb08

- Each tick: get observation → process → run robot
- Once every ~40 ticks: send observation, receive a new chunk
- Ends on success, fall or timeout; reset, then run the next episode

OUTPUT

- Per-episode results
- Logs, videos, statistics

RESULTS ACHIEVED

Success rate: same model, only the scene changes — 50 episodes

Self-built scene	0 / 50
Galileo scene (as during data recording)	28 / 50

A successful episode takes about 120 ticks, which corresponds to exactly 3 server calls. In the self-built scene the robot still lifted the apple 11/50 times but did not place it on the plate; the minimum apple–plate distance (median) dropped from 20.6 cm to 3.8 cm when the Galileo scene was rebuilt.

10 The 12 notebooks and where they appear in the report

nb01, nb02 and nb04–nb08 build and verify each block of the pipeline in cards 2–9; nb03 checks the model against real data without simulation; nb09–nb12 apply the pipeline to scenarios 2 and 3. Each of these scenarios has one notebook that builds the scene and runs a single episode, plus one notebook that runs N episodes to compute the success rate.

NOTEBOOK	CONTENT	COVERED IN	ENVIRONMENT
Pipeline building blocks · section II.02			
`nb01_server_wire`	ZeroMQ + msgpack connection, reading the data contract, latency measurement, connection-loss handling	Cards 3, 5, 6	`.venv` · needs server
`nb02_joints_obs_action`	4 joint orderings, observation packing, action-chunk decoding, chunk scheduling	Cards 4–7	`.venv` · needs server

NOTEBOOK	CONTENT	COVERED IN	ENVIRONMENT
`nb04_agile_onnx`	AGILE leg policy: 80-dim observation vector, bit-exact check against the reference	Cards 2, 8	`.venv`
`nb05_backend_anatomy`	Hand-built Isaac Sim backend: robot, head camera, apple/plate, friction, joint read/write	Cards 2, 4, 8	Isaac Sim
`nb08_runner_policy_safety`	Safety layer (NaN blocking, limit clamping, rate limiting), runner, logging	Cards 7, 9	Isaac Sim
Closed loop · section II.02 card 9 and scenario 1			
`nb06_full_pipeline`	All blocks joined into a closed loop in a self-built scene: standing, demo replay, running the model	Card 9 · III.01	Isaac Sim · needs server
`nb07_full_pipeline_loop`	Rebuilt Galileo scene, 50 episodes: 0/50 → 28/50	Card 9 · III.01 · IV.04	Isaac Sim · needs server
Open-loop verification · scenario 1			
`nb03_open_loop_dataset`	Sends real images and states from the dataset to the server and compares the returned chunk with the recorded actions	III.01 step 3	`.venv` · needs server
Applied to scenarios 2 and 3 · Part III			
`nb09_multi_object_task_pipeline`	Real2sim scene with 4 fruit types, instruction selection, one episode	III.02	Isaac Sim · needs server
`nb10_eval_multi_object`	nb09 scene run for N episodes: 66 %	III.02 · IV.04	Isaac Sim · needs server
`nb11_hammer_to_bin`	Arena hammer → bin scene rebuilt in Isaac Sim, one episode	III.03	Isaac Sim · needs server
`nb12_eval_hammer_to_bin`	nb11 scene run for N episodes with failure classification: 66 % (mix200), 73 % (mix400)	III.03 · IV.04	Isaac Sim · needs server

Most of the notebook code has been packaged into `g1gr00t`; the building-block notebooks (nb01, nb04, nb05, nb08) include cells that compare the hand-written code with the package. How to re-run the notebooks: section IV.04.

II.03 Motion research

From joint angles to motion and force.

GR00T only outputs **target joint angles**. Turning those numbers into real motion requires understanding **kinematics** (joint angles ↔ hand and camera positions) and **dynamics** (torque, inertia, contact, balance). This section summarises that knowledge and where it appears in the project; each concept has an illustration to try.

ROBOT
Unitree G1 · 43 joints

APPLIED IN
Real2sim · teleop · joint control · balance

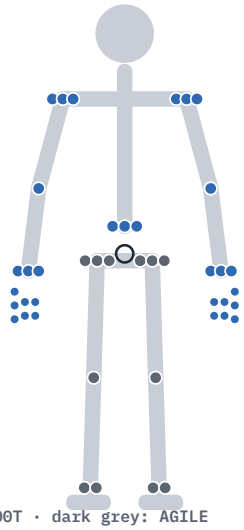
KEYWORDS
FK · IK · PD · contact

1 G1 joint structure

The G1 is a tree of links connected by revolute joints, rooted at the pelvis. Each joint group is controlled by a different component. Click a group to see it.

THE 43 JOINTS OF THE G1 · FRONT VIEW

SCHEMATIC



blue: GR00T · dark grey: AGILE
circle: root (pelvis)

12	**Legs** Hip (3), knee, ankle (2) per side	AGILE
3	**Waist** Yaw, roll, pitch	GR00T
14	**Arms** Shoulder (3), elbow, wrist (3) per side	GR00T
14	**Dex3 hands** Index (2), middle (2), thumb (3) per side	GR00T

GR00T controls the 31 upper-body joints (the waist is almost fixed in the tasks); AGILE handles the 12 leg joints.

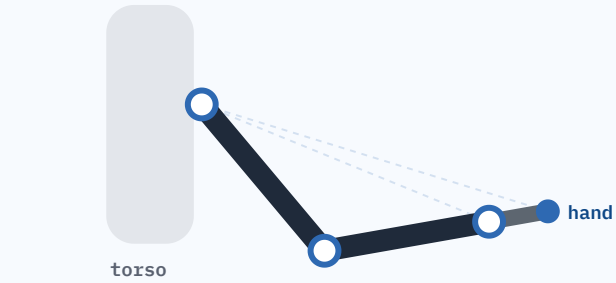
Four different joint orders coexist: the order in simulation (USD tree traversal), the order of the whole-body controller, GR00T's group order and AGILE's input order. A single shifted column is enough to make the robot fall or the model go "blind", so the project maps everything by **joint name** and tests it automatically.

2 Forward kinematics

Forward kinematics (FK) computes the position of each link (hand, fingertip, camera) from the joint angles by multiplying, in turn, the transforms along the joint tree described in the URDF file. Drag the sliders to see it: knowing the joint angles means knowing exactly where the hand is.

ARM, SIDE VIEW · 3 JOINTS

ILLUSTRATION · APPROXIMATE LINK LENGTHS



table

FK RESULT

$T_{hand} = T_{shoulder}(0.70\ rad) \cdot T_{elbow}(1.05\ rad) \cdot T_{wrist}(0.00\ rad)$

Hand in front of shoulder	52.2 cm
Hand below shoulder	16.1 cm

Each joint adds one rotation and one translation; multiplying them in turn from the shoulder outward gives the hand position. The real robot does this in 3D, for every link.

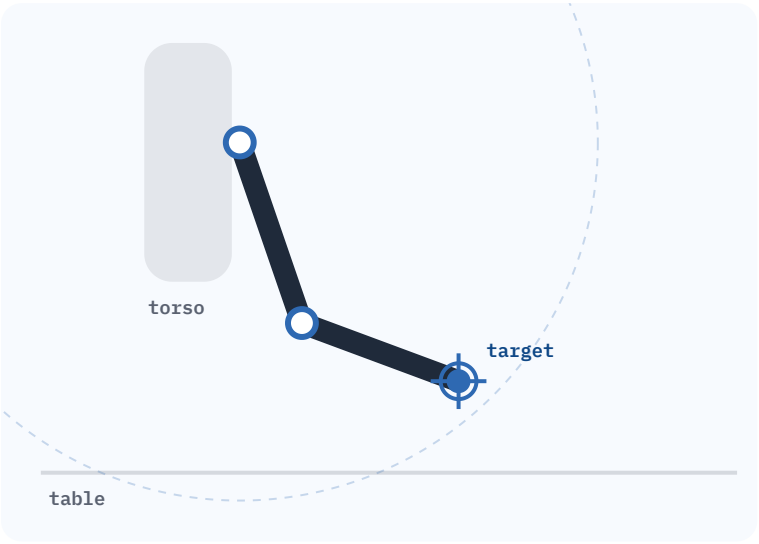
Use in real2sim: the real G1 dataset has no object positions. A custom FK (numpy only) computes the palm trajectory from the joint angles; where the hand closes is where the fruit is, and where the hand opens is where the plate is. The body pose is inferred from the assumption that both feet rest flat on the floor. See the full illustration in section II.04.

3 Inverse kinematics

Inverse kinematics (IK) works the other way: given a desired wrist position, find the joint angles that reach it. In teleop with the Meta Quest headset, the controller pose becomes the robot's wrist target, and the **Pink** IK solver computes the joint angles every frame. Click or drag in the frame to set the target; try placing it out of reach.

CLICK / DRAG TO SET THE WRIST TARGET

ILLUSTRATION · 2-JOINT IK



IK SOLUTION

Shoulder angle	0.33 rad
Elbow angle	0.88 rad
Distance to target	48.8 cm

✓ Target reached: wrist coincides with the target

Use in the project: IK is only used to **create data** for the hammer task, never while the model runs.

- **Teleoperation with the Meta Quest headset** (sections II.04, III.03): the controller pose becomes a wrist target in the pelvis frame and Pink solves the arm joint angles at every control step (embodiment `g1_wbc_agile_pink`); IK takes ~7 % of each loop while recording (section I.02). With a target out of reach, IK clamps at the joint limits and the arm stands still without any error, exactly as in the illustration above.
- **Generating extra demos** (section II.04): a script shifts the start, grasp and release points of wrist trajectories taken from teleoperated demos and sends the same kind of command as the headset; Pink turns them into joint angles. All 100 generated demos of mix200 went through this step.
- **During evaluation**, GR00T outputs joint angles directly (embodiment `g1_wbc_agile_joint`), so no IK is needed: the dataset stores the joint angles IK computed while recording, and the model learns those angles directly. The real2sim scenario (section III.02) does not use IK either, since the joint angles come with the real-robot data; FK is used there instead (card 2).

4 Dynamics

Kinematics only tells **where the hand is** given the joint angles. **Dynamics** answers the next question: how to make the joint **actually rotate to** that angle. GR00T sends only **target angles**, not forces; the motor at each joint must produce enough torque to drive the link (which has mass and inertia) to the target

without oscillating.



P · STIFFNESS (K_p)

Like a spring

The further from the target, the stronger the pull back. The stiffer the spring, the faster the joint tracks the target, but it tends to overshoot and oscillate back and forth.

D · DAMPING (K_d)

Like a shock absorber

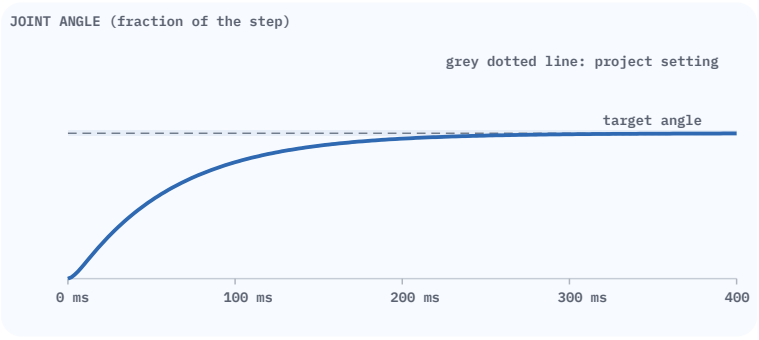
The faster the joint rotates, the stronger the braking force. With enough damping the joint reaches the target smoothly without overshoot; too much and it gets there slowly.

```
torque = Kp × (target angle − current angle)  −  Kd × joint velocity
         (spring pulls toward target)            (shock absorber brakes)
```

In the project: K_p and K_d are not hand-tuned but derived from the motor inertia using two intuitive parameters: **natural frequency 10 Hz** (speed) and **damping ratio ζ = 2** (smoothness). Result: the joint reaches 90 % of the target after ~140 ms with no overshoot. Drag the sliders below to compare.

JOINT RECEIVES A STEP COMMAND · ACTUAL ANGLE OVER TIME

ILLUSTRATION · IDEAL SECOND-ORDER SYSTEM



RESPONSE

Reach 90 % of target	**141 ms**
Overshoot	**0 %**
Settle within ±2 %	**237 ms**

ζ ≥ 1 · overdamped: reaches the target smoothly, no overshoot. A higher natural frequency tracks faster but requires more motor torque; the project chooses ζ = 2 so the arm does not oscillate.

Who computes the PD torque in simulation?

OPTION 1 · IMPLICIT PD

The physics solver computes it

PhysX computes the PD force inside the solver step. Very stable even with stiff joints; the default in Isaac Lab.

OPTION 2 · EXPLICIT PD (IDEAL PD)

Code computes it, then applies the torque

Applies exactly the formula above at every step, closer to the real robot's driver. Arena uses this option for the G1.

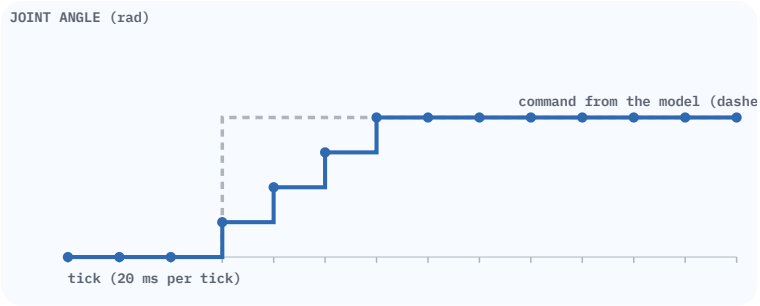
Why it matters: for the same sequence of target angles, the two options give slightly different arm motion. A model trained on demos recorded with one option should be evaluated with that same option.

5 Joint rate limit

A safety layer blocks joint-angle jumps: per tick (20 ms) the arm angle changes by at most 0.2 rad, the waist 0.1 rad, the hands 1.5 rad. The thresholds come from real data (arm: 0.15 rad/tick at the 99.9th percentile). Try a large step command:

STEP COMMAND AND ANGLE AFTER LIMITING · ARM JOINT

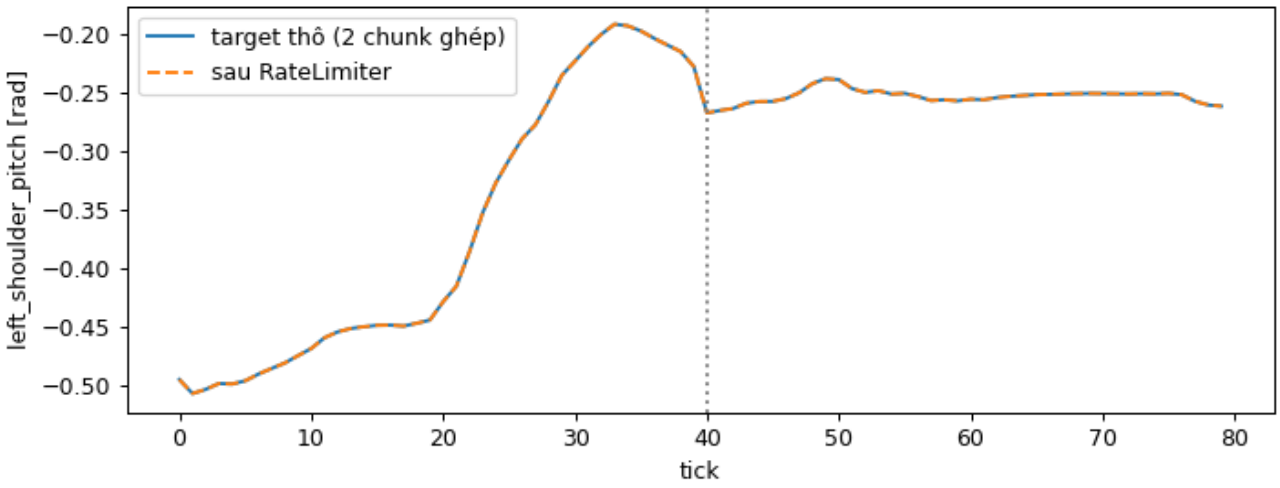
ILLUSTRATION



RESULT

Command step size	0.80 rad
Maximum per tick	0.2 rad
Ticks slowed down	3

Command split into 4 small steps, taking 80 ms instead of 20 ms



Real data: left shoulder joint command over 80 ticks (2 sequences of 40 steps joined). With normal motion, the line after the limiter (dashed) coincides with the original command: the safety layer only intervenes on abnormal jumps.

6 Physics steps: how motion is computed

A computer cannot simulate continuous motion. It splits time into very short **physics steps** (5 ms) and at each step recomputes force → acceleration → velocity → position. The easiest example is a car on a road; a robot arm joint is computed exactly the same way.

EXAMPLE · A CAR

Recomputed every 5 ms

At each step: look at the forces acting now (engine, friction) → acceleration a = F / m → new velocity v + a·Δt → new position x + v·Δt. The driver does not press the pedal 200 times a second: they decide every 20 ms and hold the pedal for those 20 ms, so **one control tick = 4 physics steps**.

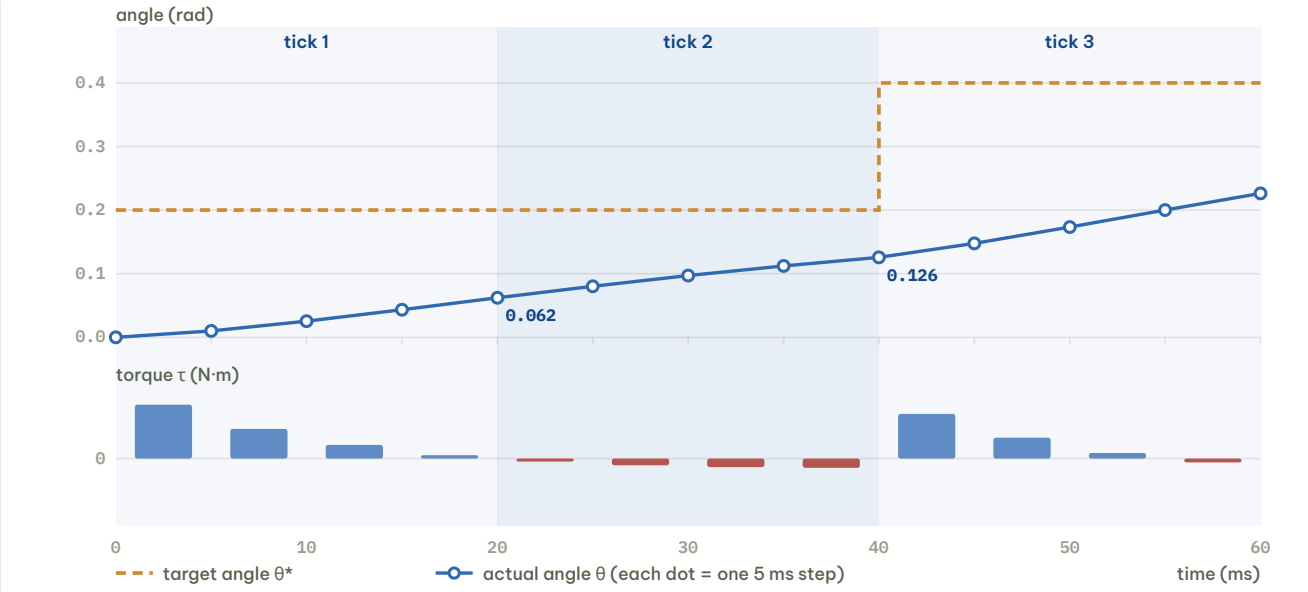
APPLIED · AN ARM JOINT

A spring and damper pull the joint to the target angle

The model does not set the joint angle directly; it only gives a **target angle** θ*. The PD controller (card 4) produces a torque like a spring pulling the joint towards θ* and a damper braking when it turns fast. At every physics step the torque is recomputed from the current angle and angular velocity.

CAR	ARM JOINT	MEANING
Position x (m)	Joint angle θ (rad)	State being computed
Velocity v (m/s)	Angular velocity ω (rad/s)	State being computed
Force F (N)	Torque τ (N·m)	Recomputed every physics step
Mass m	Inertia I	The larger, the harder to speed up
Driver picks the pedal	Pipeline picks the target angle θ*	Changes only every control tick (20 ms)

```
every physics step (Δt = 5 ms):
  τ = Kp·(θ* − θ) − Kd·ω     # spring pulls to θ*, damper brakes fast turning
  α = τ / I                  # angular acceleration
  ω = ω + α·Δt               # new angular velocity
  θ = θ + ω·Δt               # new angle
```



Illustrative example (I = 0.05; K_p = 100; K_d = 4; not the G1's real parameters). Target 0.2 rad in ticks 1–2, then 0.4 rad in tick 3. The target changes in steps at the start of a tick; the actual angle rises gradually over the 5 ms steps; the torque is large while the joint is far from the target and turns negative to brake once it moves fast.

TICK	STEP	TIME	θ*	θ BEFORE	T	Ω AFTER	θ AFTER
1	1	5 ms	0.2	0.000	20.0	2.00	0.010
	2	10 ms	0.2	0.010	11.0	3.10	0.026
	3	15 ms	0.2	0.026	5.1	3.61	0.044

TICK	STEP	TIME	θ*	θ BEFORE	T	Ω AFTER	θ AFTER
	4	20 ms	0.2	0.044	1.2	3.73	0.062
2	5	25 ms	0.2	0.062	-1.1	3.62	0.080
	6	30 ms	0.2	0.080	-2.5	3.37	0.097
	7	35 ms	0.2	0.097	-3.2	3.05	0.112
	8	40 ms	0.2	0.112	-3.4	2.71	0.126
3	**9**	**45 ms**	**0.4**	**0.126**	**16.6**	**4.37**	**0.148**

- **The target only changes per control tick**: 0.2 rad for the first 8 steps, 0.4 rad from step 9 (tick 3).
- **The torque is recomputed every physics step**: 20 N·m at step 1 while far from the target, then smaller, and negative from step 5 to brake so the joint does not overshoot.
- **The angle rises gradually, it does not jump**: the command is 0.2 rad but after 20 ms the joint is at 0.062 rad, after 40 ms at 0.126 rad, like a car that does not reach speed the moment the pedal is pressed.
- **Why the step must be short**: computed only every 20 ms, the 20 N·m of the first step would act for 20 ms with nothing braking it; the joint would overshoot and oscillate, and finger–object contacts would also go wrong.

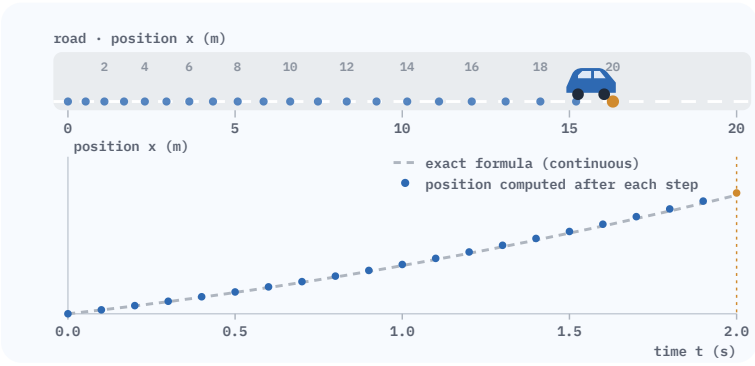
> **In Isaac Sim:** the PD controller is solved inside the physics solver (*implicit*, card 4), which is more stable than the line-by-line calculation above, but the idea is the same: the target is held during the tick, torque and angle are updated every 5 ms step, for all 43 joints at once, together with the contacts and friction of the hammer, bin and fingers.

7 Simulation: a car through each physics step

The car starts at velocity v_0 and accelerates evenly at a for 2 seconds. Press **Run**: the car does not glide but **jumps to its new position each time a physics step ends**; each dot is a position the computer computed, and between two dots the computer does not know where the car is. Change the step length to see that long steps give sparse dots and positions that drift from the exact formula.

A CAR ON A ROAD · POSITION AFTER EACH PHYSICS STEP

ILLUSTRATION · CONSTANT ACCELERATION



CURRENT STEP

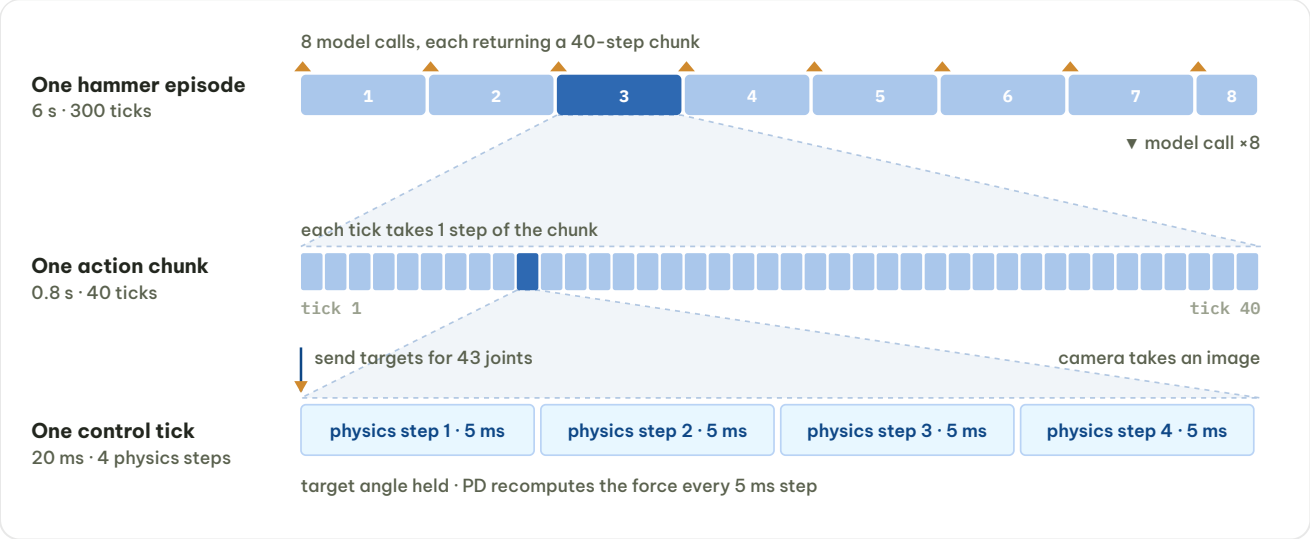
Step	20 / 20
Time t	2000 ms
Velocity v	11.00 m/s
Position x	16.300 m

After 2 s (20 steps): computed 16.30 m, exact 16.00 m, error 0.300 m (1.9 %)

Each step uses the same three lines as card 6: `a` (constant here) → `v = v + a·Δt` → `x = x + v·Δt`. With the default values and Δt = 5 ms as in the project, the error after 2 s is only about 1.5 cm; with Δt = 200 ms it grows to over half a metre. A robot arm joint is computed exactly like this, with position x replaced by the angle θ and acceleration a by the angular acceleration α = τ / I, the torque τ being recomputed at every step.

8 Control ticks and physics steps

The simulation runs on **three nested clocks**. The fastest is the Isaac Sim physics step; every 4 physics steps make one control tick of the pipeline; every 40 ticks the GR00T model is called once for a new action chunk.



Three nested clocks. Each model call gives a 0.8 s chunk; each chunk runs over 40 control ticks; each tick is 4 physics steps of 5 ms.

Physics step

5 ms

200 Hz · PhysX in Isaac Sim

Control tick

20 ms

50 Hz · = 4 physics steps

One action chunk

0.8 s

40 ticks · 1 model call

One hammer episode

6 s

300 ticks · 1,200 physics steps · 8 model calls

CLOCK	PERIOD	RUN BY	WHAT IT DOES
Physics step	5 ms · 200 Hz	PhysX (Isaac Sim)	Computes each joint's force with PD from the target angle (card 4), solves contacts and friction between fingers, objects and table; updates every body's position
Control tick	20 ms · 50 Hz	The project's pipeline	Reads the observation, takes one step of the action chunk, limits joint speed, lets AGILE compute the 12 leg joints, sends target angles for 43 joints; the camera takes one image
Action chunk	0.8 s · 40 ticks	GR00T server	Looks at the image, the instruction and the joint state, returns 40 steps of target angles for arms, hands and waist

Constraint in the code: the physics step × steps per tick must equal one tick exactly (0.005 s × 4 = 0.02 s). A mismatched configuration fails as soon as it is loaded (`ControlCfg` in `g1gr00t/config.py`). Scenario 2 (section III.02) uses another rate to match the real G1 data recorded at 20 Hz: still 200 Hz physics but 10 steps per tick, i.e. 20 Hz control; its 16-step chunk also lasts 0.8 s.

9 Inside one control tick

Each tick the pipeline runs exactly one loop: read the state, decide the target angle for all 43 joints, then let physics run 4 steps with that target.

01 · OBSERVE
Image and state
Head camera, 43 joint angles, velocities, IMU

→

02 · ACTION CHUNK
Take the next step
After 40 steps, call the GR00T server

→

03 · SAFETY
Speed limit
Arms 0.2 rad/tick (card 5)

→

04 · LEGS
AGILE keeps balance
80 observation values → 12 leg joints

→

05 · PHYSICS
4 steps × 5 ms
Target angle held

```
every control tick (20 ms):
  obs   = backend.observe()                    # image, 43 joint angles, velocities, IMU
  step  = scheduler.step(policy.fetch, obs)    # 1 of 40 steps; call server when out
  upper = rate_limiter.apply(step["upper_q"])  # arms ≤ 0.2 rad/tick (card 5)
  lower = agile.step(q, dq, imu)               # 12 leg joints, also 50 Hz
  robot.set_joint_position_target(upper + lower)
  for _ in range(4):                           # 4 steps × 5 ms, target held
      sim.step()                               # PD forces; contacts, friction
  camera.update()                              # image for the next tick
```

HOLDING THE COMMAND

The target angle is held for 4 physics steps

The command is a target angle, not a force. Within the 20 ms of a tick the PD controller recomputes the force 4 times from the changing actual angle, so the joint moves towards the target instead of jumping. A real robot works the same way: the motor driver takes a position command and runs its own, much faster control loop.

TWO RATES

Why physics must run faster than control

Collisions and finger–object contacts only stay stable with a small time step; long steps easily let bodies sink into each other or bounce. The control tick, on the other hand, must be exactly 20 ms because AGILE and the demo data are both at 50 Hz: with another tick, one step of the action chunk would no longer mean 20 ms as during training.

10 Simulation time and real time

One model call takes about 1.2–1.5 s (median during evaluation, section I.02), longer than the 0.8 s of one action chunk. Simulation and the real robot handle this gap very differently.

SIMULATION

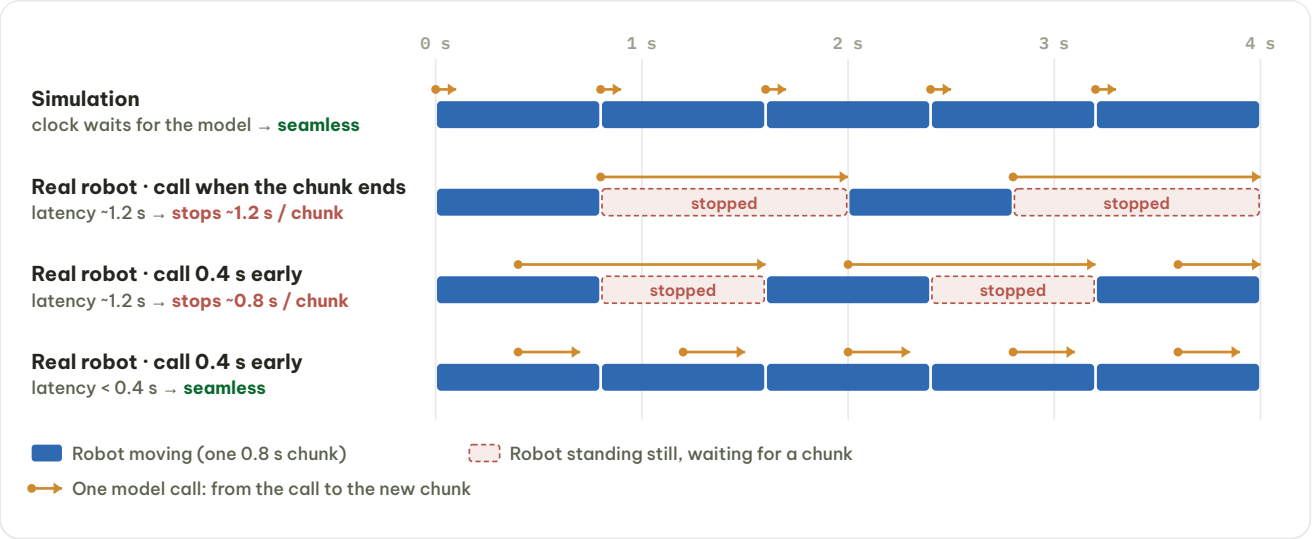
The simulation clock waits for the model

Simulation time only advances when `sim.step()` is called. While waiting for the server no physics step runs, so latency does not affect the result. The cost is speed: one 6 s hammer episode takes ~36 s of real time on average (mix400 run, 8 model calls per episode).

REAL ROBOT

Time does not stop

If the next call waits until the chunk runs out, the robot stands still while waiting. The code already has `PrefetchingChunkScheduler`: it calls the server when 20 ticks (0.4 s) of the chunk remain, so the new chunk arrives as the old one ends. The trade-off is that the new chunk is computed from an image taken 0.4 s earlier.



The first 4 seconds on a time line. Simulation never has to stop because its clock waits for the model. On the real robot, motion is seamless only when the call is made 0.4 s early *and* latency is below 0.4 s; the model itself takes ~0.1 s, the rest is the network path (section I.02).

II.04 Data research

Few real demos, plenty of training data.

Fine-tuning GR00T needs a few hundred demos per task, but teleop demos are expensive and the model is highly sensitive to visuals. This section explains three techniques for getting more data without more teleop (the project has applied the latter two; the first is still only a proposal): **domain randomization**, **real2sim** and **demo generation**. Each technique has an interactive illustration showing how it works.

TECHNIQUES
Domain randomization · real2sim · demo generation

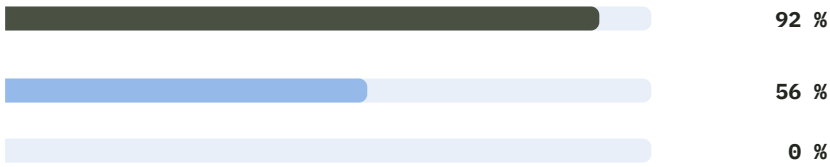
APPLIED TO
Apple & Plate · Multi Fruit · Hammer & Bin

KEYWORDS
Visual gap · digital twin · MimicGen

1 Problem 1: Visual Gap

If every demo is recorded in the same scene, the model memorises that image. The apple task shows this clearly.

Same apple → plate checkpoint, only the scene changes — success rate · measured, section III.01



Same checkpoint, same instruction; the 92 % figure was measured with the NVIDIA client, the other two with the project's pipeline. The model saw only one scene during training, so it latched onto that image instead of the apple. Domain randomization (card 2) is the remedy.

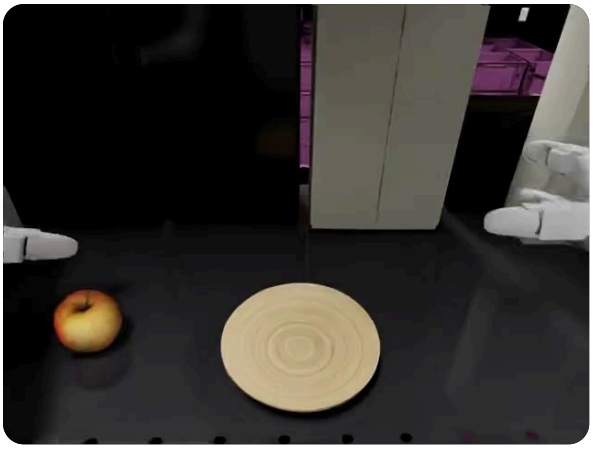
2 Solution 1: Domain Randomization

In one sentence: randomly vary everything **unrelated** to the task (lighting, colours, background, camera angle) so the only thing the model can rely on is what stays constant: **the object and the hand**. Faced with a new scene, even a real image, the model treats it as just another variant (Tobin et al., 2017).

Try it: two ways to diversify images

METHOD 1 · 2D IMAGE AUGMENTATION

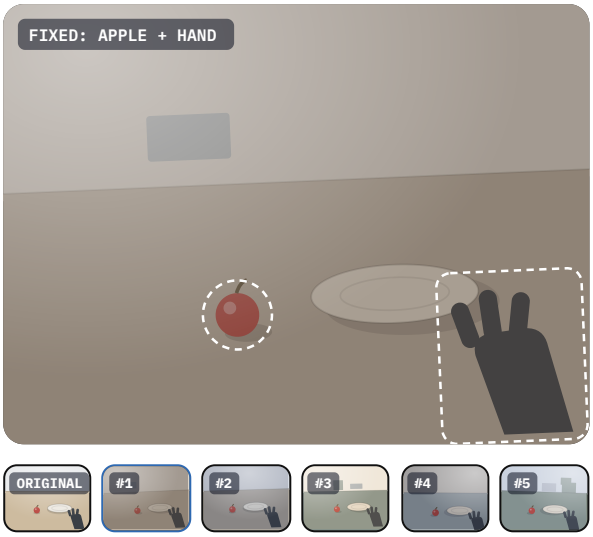
REAL IMAGE · HEAD CAMERA



Color jitter: cheap, applied to images during training, but it **cannot create shadows, reflections or new viewpoints**. The project used this method when fine-tuning Multi Fruit.

METHOD 2 · DOMAIN RANDOMIZATION

ILLUSTRATION · NOT MEASURED DATA



Change the scene itself, then **re-render**. The apple and the hand (dashed outline) stay the same: that is what the model must learn.

Factors that can be randomized

Three groups of factors, from easy to hard:

APPEARANCE

What the model sees

Lighting, materials, textures, background, decorative objects outside the manipulation area.

GEOMETRY

Layout and viewpoint

Object position and orientation; camera position, tilt and focal length.

DYNAMICS

Physics

Mass, friction, joint stiffness, latency (Peng et al., 2018). Needed when moving to the real robot.

> **Trade-off:** the wider the randomization, the more robust the model, but the more data and training steps it needs. Prioritise the factors that actually caused the drop (here, lighting and rendering), then widen.

Application in the project

APPLIED TO	DONE	STILL MISSING
Apple & Plate	Measured visual gap 92 → 56 → 0 %	Randomize the scene, then replay the 208 demos (proposed in section III.01)
Multi Fruit	Fruit, position, orientation, plate and distractor fruit changed every episode; color jitter during fine-tuning	Lighting and materials not yet varied

3 Problem 2: Sim Data Scarcity

The Multi Fruit scenario has no suitable simulation dataset for G1 + Dex3. The closest is **PhysicalAI-Robotics-GR00T-Teleop-G1** (1,095 teleop trajectories), but it was recorded on a real robot and cannot be used directly in simulation.

GAP 1

No object positions

The data has only joint angles and video; there is no record of where the fruit and plate are, so the scene cannot be built.

GAP 2

Real image ≠ rendered image

Real RealSense video does not match the simulated images the model will see at run time.

GAP 3

Actions mean different things

Actions on the real robot and in simulation share the same names but carry different meanings.

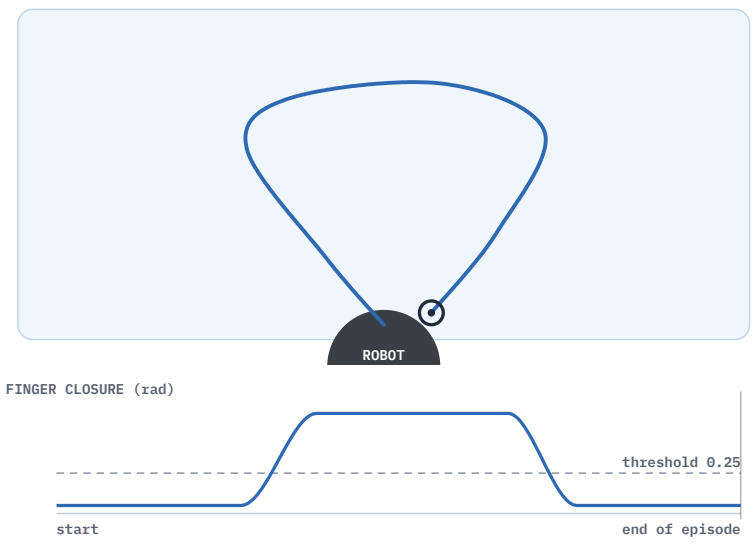
4 Solution 2: Real2sim

In one sentence: rebuild a **digital copy** (digital twin) of the real scene in simulation, then replay the real motion in it to obtain simulation data. The dataset has no object positions, so the project **infers the scene from the hand motion**: where the hand closes is where the fruit is, where the hand opens is where the plate is.

Try it: reconstruct the scene from one trajectory

TOP-DOWN VIEW OF THE TABLE

ILLUSTRATION · NOT MEASURED DATA



STEP 1 / 5

Forward kinematics (FK)

From the joint angles in each frame, multiply the transforms along the arm (URDF file) in sequence to get the palm position. Joined together, these form the hand trajectory over the table.

In the project: Custom FK written with numpy only; the torso pose is inferred from the assumption that both feet stand flat on the floor (section II.03).

5 Problem 3: Teleop Bottleneck

The Hammer & Bin scenario is completely new: there is no existing dataset or checkpoint, and every demo has to be collected by Meta Quest headset teleop in Isaac Lab.

SLOW

~10 demos per session

The first 100 real demos took 10 teleop sessions.

UNSTABLE

Many attempts must be redone

The robot hand freezes when the controller signal is lost, and lags from the 2nd attempt.

INCONSISTENT

Failed demos mixed into the data

Some demos were marked successful while the hammer was still resting on the bin wall or the hand had not let go.

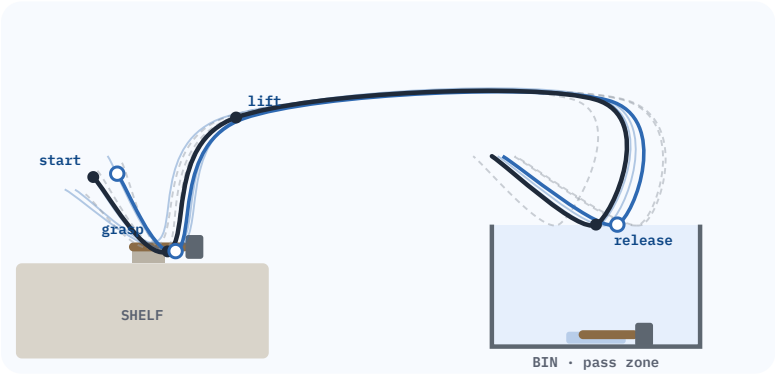
6 Solution 3: Demo Generation

In one sentence: take a teleop demo as a **template**, slightly shift the key points (start, grasp, release) and the speed to get a new demo, then **re-run it in simulation**: only demos that still succeed are kept. This is the MimicGen approach (Mandlekar et al., 2023), which NVIDIA used at large scale to generate 780,000 trajectories for GR00T N1.

Try it: generate hammer → bin demos

SIDE VIEW · SHELF → BIN

ILLUSTRATION · OFFSETS EXAGGERATED FOR VISIBILITY



5 kept

3 rejected

LATEST GENERATED DEMO · #8

Time	×0.96
Start offset	1.2 cm
Grasp offset	4 mm
Release offset	+1.0 cm

✓ Re-run in simulation: hammer in the middle of the bin → KEEP

In the actual project: **100 / 133** attempts kept, 60 template demos, ~17 minutes for 100 demos.

Template demo (teleop) — Generated demo, passed

---- Generated demo, rejected (hammer not in the middle of the bin)

7 Combination and conclusion

The three techniques do not replace one another; they chain into a single pipeline. The combined approach for the stage of moving the model to the real robot:

01
Real teleop
Few demos, in the working environment →

02
Real2sim
Build the scene, replay in simulation →

03
Demo generation
Multiply the count, filter in simulation →

04
Domain randomization
Diversify appearance and physics →

05
Fine-tune & sim2real
Evaluate in simulation, then on the real robot

CONCLUSION

The project has validated two of the three techniques: **real2sim** turned real-robot data into 790 simulation episodes (66 %), and **demo generation** doubled the teleop data in ~17 minutes (66 % with mix200). **Domain randomization** has so far only reached measuring the visual gap and proposing a method; it is the next step to take before moving to the real robot.

III.01 Scenario Apple & Plate

G1 picks an apple from the shelf and places it on a plate.

A sample scenario from NVIDIA Isaac Lab Arena, re-run with a **control pipeline built by the project**. The project verified the pipeline layer by layer (open-loop against the dataset, demo replay) and then evaluated NVIDIA's checkpoint in three scenes. Results: **92 %** in the original environment, **56 %** when only the lighting differs, **0 %** when the whole scene differs — the model is highly sensitive to visuals, and the pipeline works correctly.

DATASET

208 episodes

Source	**NVIDIA · XR teleop**
Frames	**35,066**
Rate	**50 Hz**

FINETUNE

None no retraining

Uses	**NVIDIA checkpoint**
Tuned for	**Apple → plate**
Instruction	**From the dataset**

MODEL

GR00T N1.7 3B

Checkpoint	**GN1x-Tuned-Arena**
Embodiment	**G1 + Dex3**
Action chunk	**40 steps · 50 Hz**

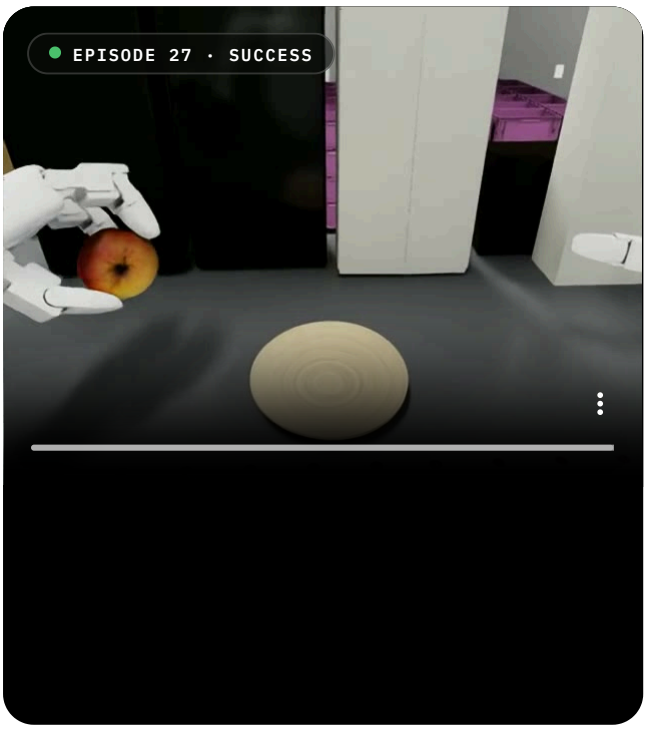
RESULT

92 % success

92 % ORIGIN… | 56 % LIGHTI… | 0 % SCENE

Same model; only the runtime environment changes

1 Video Showcase



The robot picks up the apple and places it on the plate, seen from the head camera

INSTRUCTION SENT TO THE MODEL

"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."

2.9 s
Time to success

1.0 cm
Apple centre to plate centre

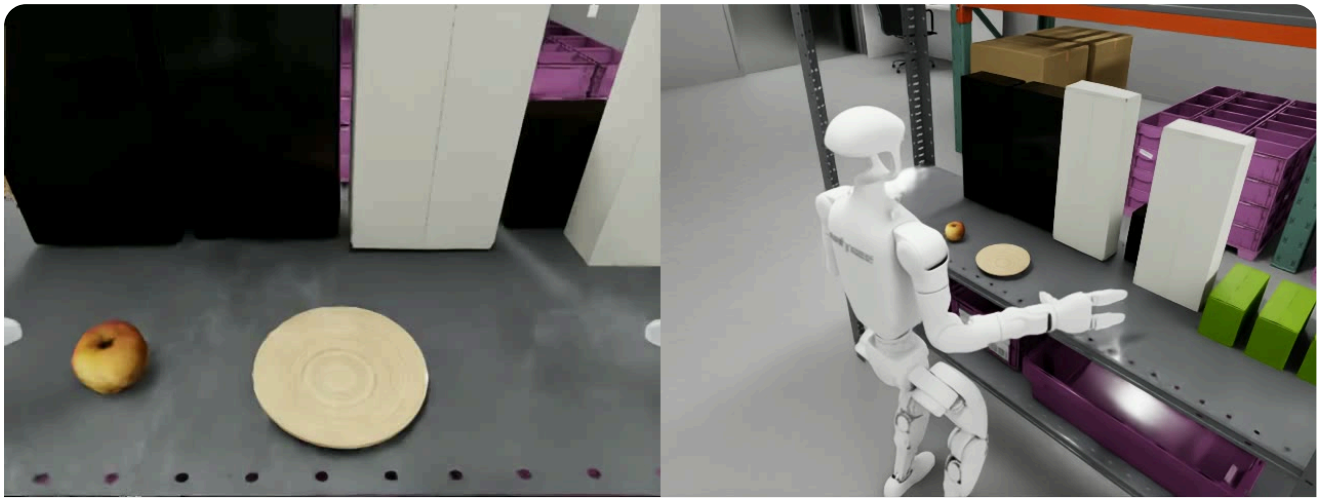
4
Model calls, 40 steps each

1.1 s
Average latency per call

Taken from the 50-episode run in the reconstructed Galileo scene (28/50 successes), NVIDIA checkpoint.

2 Scenario setting

The G1 robot stands in front of a shelf in Arena's Galileo lab; the apple and the plate sit next to each other on the shelf surface. The image below combines the **first frame** of the episode in the Video Showcase (the image the model receives at step 0) with an oblique overview.



Frame 0 · left: robot head camera, the image the model receives · right: oblique view from an external camera (Galileo scene reconstructed in Isaac Sim, same camera angle as the Hammer & Bin scenario), for illustration only.

SCENE & OBJECTS	DETAILS
Background	Galileo lab (`galileo_locomanip.usd`); 3 equipment boxes blocking the hand's reach are hidden (jetson_orin_06, jetson_orin_03, hesai_box_06)
Shelf surface	Hidden support slab 0.8 × 1.5 × 0.04 m; top surface at 0.765 m
Apple	Placed at (0.58, 0.27) m
Plate	Placed at (0.58, 0.06) m — ~21 cm from the apple, to the robot's right
Friction	Dex3 finger friction increased to hold the apple firmly

ROBOT, CAMERA & CONTROL	DETAILS
Robot	Unitree G1 + Dex3 hands, standing at (0.25, 0.08) m, facing the +x axis; legs balanced by the AGILE controller, pelvis height ~0.73–0.75 m (command 0.72 m)
Camera	Mounted on `head_link` , 640×480, 15 mm focal length, tilted down ~35°
Rate	Physics 200 Hz, control 50 Hz (4 physics steps / command)
Episode	At most 300 steps = 6 s; each model call returns a chunk of 40 steps
Instruction	"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
Success	Apple centre < 8 cm from the plate centre, 0–4 cm above the plate and velocity < 0.1 m/s

3 Scenario goals

NVIDIA's sample scenario serves as a **reference baseline**: the model and data already exist, so any discrepancy comes from the project's pipeline or environment.

GOAL 1

Understand the data path

Fully understand the flow from camera and joint angles to the model, and from the model to the G1's motors.

GOAL 2

Rewrite the pipeline

Build a standalone package that does not depend on Arena at runtime — a mandatory step before moving to the real robot, since Arena runs only in simulation.

GOAL 3

Verify and measure

Prove the pipeline correct layer by layer, then measure the success rate of NVIDIA's checkpoint and find the causes of failure.

4 Scenario data

DATASET ARENA-G1-STATIC-PICKNPLACE	VALUE
Source	XR headset teleop in Isaac Lab, 50 Hz — the data used to fine-tune NVIDIA's checkpoint
Episodes	208 files (the card states 200; metadata indexes up to 251)
Total frames	35,066 (11.7 minutes); each episode 2.3–5.3 s
Instruction	"Pick up the apple from the shelf and place it onto the plate on the same shelf next to it."
Observations	Only the left hand grasps the apple; waist and right hand almost motionless; robot stands still, torso height ~0.72 m

5 Problems encountered

When Arena's pipeline was recreated with the project's own package, every control stage worked correctly, but the success rate depended strongly on the **camera images** the model received.

> **Visual gap:** same checkpoint, same instruction; changing only the simulated scene takes the success rate from **92 %** (NVIDIA client, original Arena) down to **56 %** and then **0 %** (project pipeline).

PROBLEM 1

The model fails in scenes that differ from the dataset

The project-built scene (box table, grid floor, dome light) gives **0/50** successes, with only 11/50 lifting the apple. The model reaches out like in the demos but never commands a grasp.

PROBLEM 2

Rebuilding the exact scene is still not enough

The reconstructed Galileo scene matches the dataset's layout and camera angle but reaches only **56 %**, versus 92 % in the original Arena. The remaining difference is mainly lighting and rendering.

PROBLEM 3

The model learned from a single scene

The checkpoint was fine-tuned on data recorded in exactly one scene under one lighting condition, so it is tied to those images rather than to the objects to be manipulated.

PROBLEM 4

Hard to transfer to the real environment

The real robot will face lighting, backgrounds and materials very different from simulation; with the current visual sensitivity, the checkpoint cannot be used directly.

6 Proposed method

We propose **domain randomization**: when generating training data, randomly vary the visual factors unrelated to the task, so the model is forced to rely on the objects and hand pose rather than the background and lighting.

Domain randomization is also a common technique for **sim2real**; here it is applied first to the gap between simulated scenes, as a prerequisite step before moving to the real robot.

> **Status:** not yet carried out. This card only describes the method and how to apply it to the existing pipeline.

Randomized factors

Appearance — what the model sees but which does not affect how the task is done:

LIGHTING

Intensity, colour, direction

Vary the intensity and colour temperature of dome/rect lights, light direction, and toggle baked lights — exactly the factor that dropped 92 % to 56 %.

MATERIALS

Colour and texture

Change the colour, glossiness and texture of the shelf/table surface, the plate and the room background; keep object shapes unchanged.

SCENE

Background and surrounding objects

Change the backdrop, add/remove boxes and decorative objects outside the manipulation area.

Geometry — viewpoint and object positions, so the model does not memorise a fixed layout:

CAMERA

Pose and parameters

Small perturbations of the head camera's position, tilt and focal length; add noise, exposure and color jitter during training.

OBJECTS

Position and orientation

Shuffle the position and orientation of the apple and plate within the hand's reach.

How to apply

STEP 1
Randomized scene
Isaac Lab / Replicator reset events change the factors above every episode →

STEP 2
Demo replay
Replay the actions of the 208 demos in each scene, recording new images →

STEP 3
Filter
Keep episodes that still meet the success condition →

STEP 4
Fine-tune again
From the NVIDIA checkpoint on the diversified data →

STEP 5
Cross-evaluation
Galileo, the project-built scene and unseen scenes

Expected outcome: narrow the 92 % → 56 % → 0 % gap between scenes. The metric is the success rate in the project-built scene (currently 0 %) and the reconstructed Galileo scene (currently 56 %), with ≥ 50 episodes per configuration.

7 Implementation process

Isaac Lab Arena's execution flow was reproduced with the project's own package `standalone_g1_gr00t`: each Arena stage was separated out, rewritten and verified individually in a notebook, then assembled into a closed loop with no remaining dependency on Arena.

STEP 1 · ARENA
Run the original Arena
NVIDIA checkpoint in the original environment — 92 % reference point →

STEP 2 · NB01–02
Model communication
ZMQ server, observation and action translation →

STEP 3 · NB03–04
Verify each block
Open-loop against the dataset, AGILE legs →

STEP 4 · NB05–06
Isaac Sim backend
Robot, camera, apple/plate, demo replay →

STEP 5 · NB07–08

Closed loop & packaging

Galileo scene, runner, safety layer

STEP	WORK	NOTEBOOK / TOOL	RESULT
1	Run the Arena pipeline with NVIDIA's client and checkpoint to obtain a baseline	Isaac Lab Arena	92 % in the original environment
2	ZMQ + msgpack protocol, modality contract; table of 4 joint orderings, `build_observation`, `decode_action`, `ChunkScheduler`	nb01_server_wire, nb02_joints_obs_action	Correctly sends/receives 40-step chunks
3	Real images and states from the dataset → server → compared with recorded actions; reconstruct the 80-dimensional observation vector of the ONNX leg policy	nb03_open_loop_dataset, nb04_agile_onnx	Error 10–70× smaller than the amplitude; AGILE bit-exact
4	Build the backend by hand: SimulationContext → robot → head camera → apple/plate → finger friction → observe/apply; assemble the full data path	nb05_backend_anatomy, nb06_full_pipeline	Stands stably for 10 s; demo replay succeeds
5	Reconstruct Arena's Galileo scene, run 50 GR00T episodes; package runner, policy, `sanitize_chunk`, `RateLimiter`	nb07_full_pipeline_loop, nb08_runner_policy_safety	56 % (lighting mismatch), 0 % (project-built scene)

8 Results achieved

Figures measured at each step of the process, from block-by-block verification to closed-loop runs. The same NVIDIA checkpoint and the same instruction for every run.

Original Arena

92 %

NVIDIA client · original environment

Reconstructed Galileo

56 %

28/50 · Wilson 42–69 %

Project-built scene

0 %

0/50 · lifted 22 %

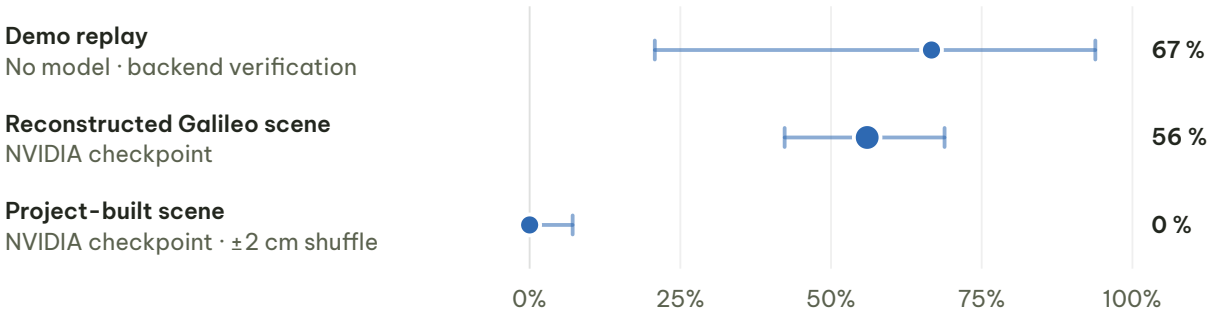
Open-loop error

≤ 0.032 rad

MAE 0.008–0.032 · 10–70× smaller than the amplitude

Success rate with Wilson 95 % confidence interval

Same checkpoint, only the scene changes. Demo replay uses no model; it only verifies the backend.



STEP	MEASURED FIGURES
Model communication	40-step chunk per call; latency ~1.2 s/call (median over 50 episodes; the simulation waits for the model, so results are unaffected)
Open-loop against the dataset	MAE left arm 0.032 · right arm 0.019 · left hand 0.008 rad (3 episodes, 11 chunks)
Instruction	Dataset instruction 0.025 rad; Arena default instruction 0.047 rad; nonsense instruction 0.039 rad
AGILE legs	Bit-exact with the reference implementation; stands stably for 500 ticks, torso 0.73–0.75 m, maximum tilt ~10°
Demo replay	2/3 successes; apple 3 cm from the plate centre; joint tracking error 0.04 rad
Closed loop · Galileo	**28/50 successes; lifted 72 %; median apple–plate distance 3.8 cm; median time to success 2.7 s**
Closed loop · project-built	0/50 successes; lifted 22 %; median apple–plate distance 20.6 cm

9 Results analysis

The runs are compared phase by phase to determine where failures come from.

PHASE (50 EPISODES)	PROJECT-BUILT	GALILEO
Apple lifted	11 (22 %)	36 (72 %)
Placed on the plate / lifted	0 / 11	28 / 36 (78 %)
Success	**0 %**	**56 %**
Falls	3	2

FINDING 1

Vision determines the outcome

Same checkpoint and instruction: 92 % (NVIDIA client, original Arena) → 56 % → 0 % (project pipeline) solely from changing the scene. The two confidence intervals 0–7 % and 42–69 % do not overlap.

FINDING 2

Failures lie in the perception and approach phase

In the project-built scene the model almost never lifts the apple (22 %). In Galileo, once lifted, 78 % are placed on the plate — the manipulation skill is still there.

FINDING 3

The project's own pipeline works correctly

28 successes with final distances of 1–6 cm, together with the open-loop and replay checks, rule out data translation errors.

FINDING 4

Implications for deployment

A model fine-tuned on a single scene does not generalise; data must be collected in the working environment, or visual diversity must be increased during training.

Details of the 22 failures (Galileo scene)

12 Apple not lifted –

9 times the apple barely moved; 2 times the apple was pushed and slid 4–9 cm; 1 time the apple was knocked off the shelf.

Likely cause: grasp slipped or wrong approach. The initial positions are identical ⇒ the difference comes from randomness in model sampling.

8 Lifted but placed off target –

6 times released 9–20 cm from the plate (closest 9.1 cm, just above the 8 cm threshold); 2 times the apple was dropped off the shelf.

2 Fell right at the start of the episode –

Fell at tick 7 and tick 30, before touching the apple. The first arm chunk overlaps the period in which AGILE is still stabilising after reset. We propose letting the robot "warm up" for a few dozen ticks before handing control to the model.

Factors ruled out

LATENCY

The simulation waits for the server; latency is the same for successful and failed episodes.

SAFETY LAYER

Only clips arm joints at chunk boundaries; it never clipped the hands or waist.

PIPELINE BUGS

28 successes with final distances of 1–6 cm ⇒ the data chain works correctly.

10 Scenario conclusion

CONCLUSION

A model fine-tuned on a single scene is **highly sensitive to visuals**: 92 % in the original environment used for recording, 56 % when only the lighting differs, 0 % when the scene is completely different. Real-world deployment requires collecting data directly in the working environment or increasing visual diversity during training.

- The pipeline has been **mastered and independently reproduced**; the code is ready to swap in a real-robot backend.
- The exact instruction used during data recording must be used; each configuration needs ≥ 50 episodes to draw conclusions.
- Not yet done: running the project pipeline inside the original Docker Arena for a direct comparison with the 92 % baseline; Galileo scene with ±2 cm shuffling; warm-up phase after reset.

III.02 Scenario Multi Fruit

G1 picks the fruit named in the instruction and places it on the plate.

The table holds 4 kinds of fruit (apple, pear, star fruit, grapes); the instruction decides which fruit to pick. From 1,095 teleop trajectories on the real G1 robot, the project **reconstructs the simulated scene from joint data alone** (real2sim), generates 790 training episodes and fine-tunes GR00T. Results: the reused apple checkpoint reaches **0 %**; the fine-tuned model reaches **66 %** on randomized scenes with distractor fruit and **never picks the wrong fruit**.

DATASET

790 episodes

Source	**Real2sim · real G1**
Frames	**86,822**
Test set	**89 episodes · 20 Hz**

FINETUNE

20,000 steps

Time	**2 h 34 min**
Batch · GPU	**32 · 1× H100**
Loss	**1.50 → 0.019**

MODEL

GR00T N1.7 3B

Checkpoint	`g1_fruits_n17`
Instructions	**4 kinds of fruit**
Action chunk	**16 steps · 20 Hz**

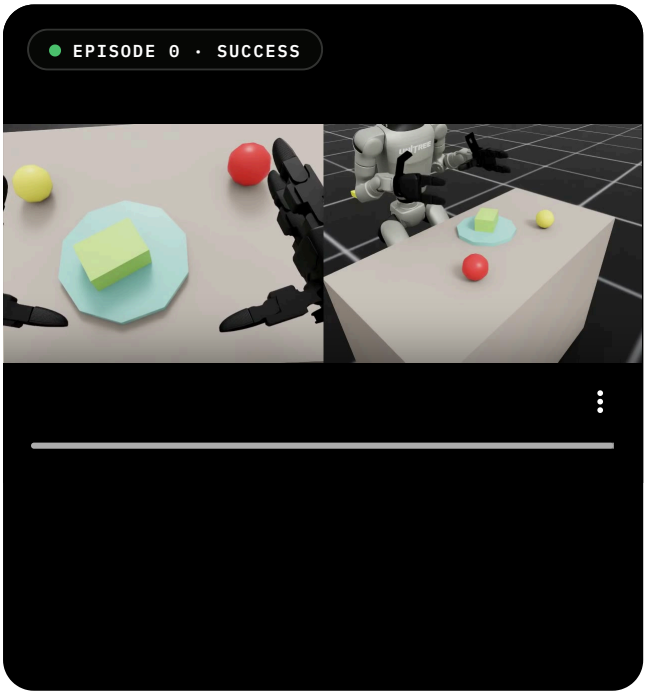
RESULT

66 % success

0 % NOT TUNED | 66 % FINE-TUNE

Never picked the wrong fruit

1 Video Showcase



The robot picks the bunch of grapes from among the other fruit and places it on the plate

INSTRUCTION SENT TO THE MODEL

"Pick up the green grapes and place it on the plate"

1.4 cm
Fruit centre to plate centre

16.5 cm
Maximum lift height

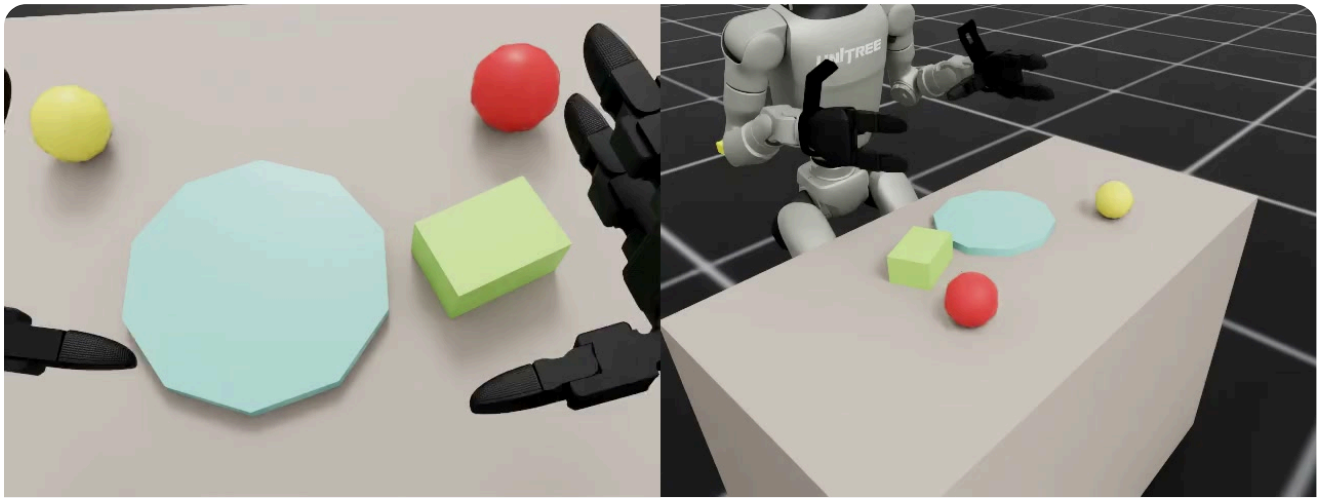
2
Distractor fruit, not disturbed

10 s
Episode length (200 steps × 50 ms)

Taken from the run of 50 randomized real2sim episodes (33/50 successful). The grapes are the green block.

2 Scenario setting

The G1 robot stands fixed in front of a table; on the table are a plate and three fruit, one of which is the target while the other two are distractors. The image below is the **first frame** of the episode shown in the Video Showcase (episode 0: green grapes).



Frame 0 · left: robot head camera, the image the model receives · right: external camera, used only for recording video. The target is the grapes (green block); the distractors are the apple and the pear.

SCENE & OBJECTS	DETAILS
Table	Simple table on a grid floor; dome light 700 + sun 600
Plate	Kinematic plate, radius 0.115 m; in episode 0 placed at (0.40; 0.08) m
Fruit	4 kinds as simple blocks (apple, pear, star fruit, grapes = green block); friction 1.5
Target	Episode 0: grapes at (0.41; −0.12) m
Distractors	2 fruit placed outside the hand's sweep area (episode 0: apple, pear)
Randomization	Each episode changes the target fruit, fruit position and orientation, plate position and distractor fruit; separate seed

ROBOT, CAMERA & CONTROL	DETAILS
Robot	Unitree G1 29 DoF + Dex3 hands, pelvis fixed at (0; 0; ~0.74) m
Camera	Head-mounted RealSense D435, 640×480 (fx 606) + 1 external camera for recording
Rate	Physics 200 Hz, 10 substeps → control 20 Hz

ROBOT, CAMERA & CONTROL	DETAILS
Episode	Up to 200 steps = 10 s; receives a 16-step chunk, executes 8, then queries again
Instruction	Depends on the target fruit, e.g. "Pick up the green grapes and place it on the plate"
Success	Fruit lifted > 3 cm, lying on the plate (within radius + 1 cm, lowest point above the table surface), distractor fruit moved ≤ 1 cm

3 Scenario goals

Bring teleop data from the real G1 robot into simulation (**real2sim**) to train and evaluate a model that picks the fruit named in the instruction.

GOAL 1

Closed-loop evaluation

A simulation environment for a task that previously had only real-robot data.

GOAL 2

Data multiplication

Change distractors, positions and lighting without additional teleop.

GOAL 3

Repeatable measurement

Compare checkpoints and training configurations on the same yardstick.

4 Scenario data

Dataset **PhysicalAI-Robotics-GR00T-Teleop-G1** (NVIDIA GEAR, CC-BY-4.0): teleop on the real G1 robot, RealSense camera 640×480, 20 Hz. The robot picks the fruit named in the instruction and places it on the plate. The data contains **only joint angles and video**, no object positions.

FRUIT	EPISODES	FRAMES
Apple	311	29,864
Grapes	233	26,970
Pear	264	32,319
Star fruit	287	34,640
Total (dataset card states 1,000)	**1,095**	**123,793**

5 Problems encountered

The goal is a model that picks the fruit named in the instruction. Before creating its own data, the project tried fine-tuning on existing datasets.

> **No suitable simulation dataset.** The only set with the right robot (G1) and the right pick-and-place task was recorded on the **real robot**, so it cannot be used directly in Isaac Sim.

PROBLEM 1

Existing checkpoint cannot be reused

The apple checkpoint from scenario 1 with the 4-fruit instructions reaches **0 %**: the model learned only one task in one scene.

PROBLEM 2

No simulation dataset

Public datasets were reviewed for trial fine-tuning: no simulated set exists for G1 + Dex3 pick-and-place of multiple fruit by instruction.

PROBLEM 3

The only suitable set is real data

PhysicalAI-Robotics-GR00T-Teleop-G1: same G1 robot, same pick-and-place of fruit onto a plate, but recorded on the real robot.

PROBLEM 4

Real data lacks information needed for simulation

Only joint angles and video, **no object positions**; real images do not match rendered images; actions on the real robot and in simulation have different meanings.

6 Proposed method

Solved with **real2sim**: use the joint trajectories of the real data to rebuild the corresponding simulated scene, replay it in Isaac Lab and record it as a simulation dataset — keeping the real motion of the teleoperator while images and physics stay consistent with the evaluation environment.

STEP 1
Forward kinematics (FK)
URDF G1 29-DoF + Dex3 → torso pose, hand trajectory, grasp/release timing →

STEP 2
Infer object positions
Fruit = palm at grasp; plate = position at release →

STEP 3
Safe distractors
Placed outside the hand's sweep, the fruit-carrying path and the plate →

STEP 4
Replay & filter
Isaac Lab; keep successful episodes that do not touch distractors →

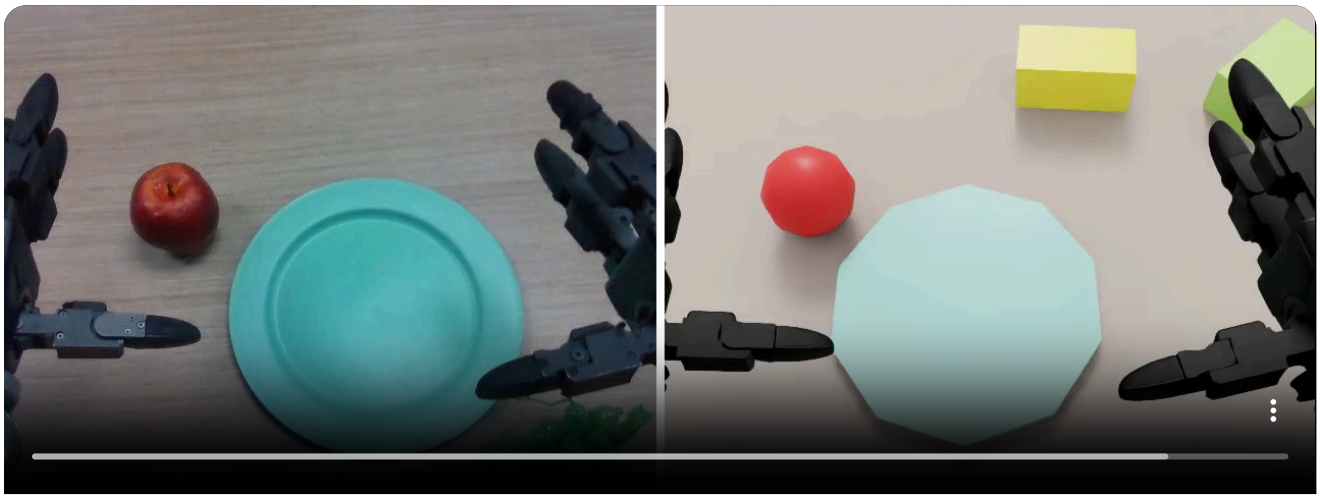
STEP 5

LeRobot dataset

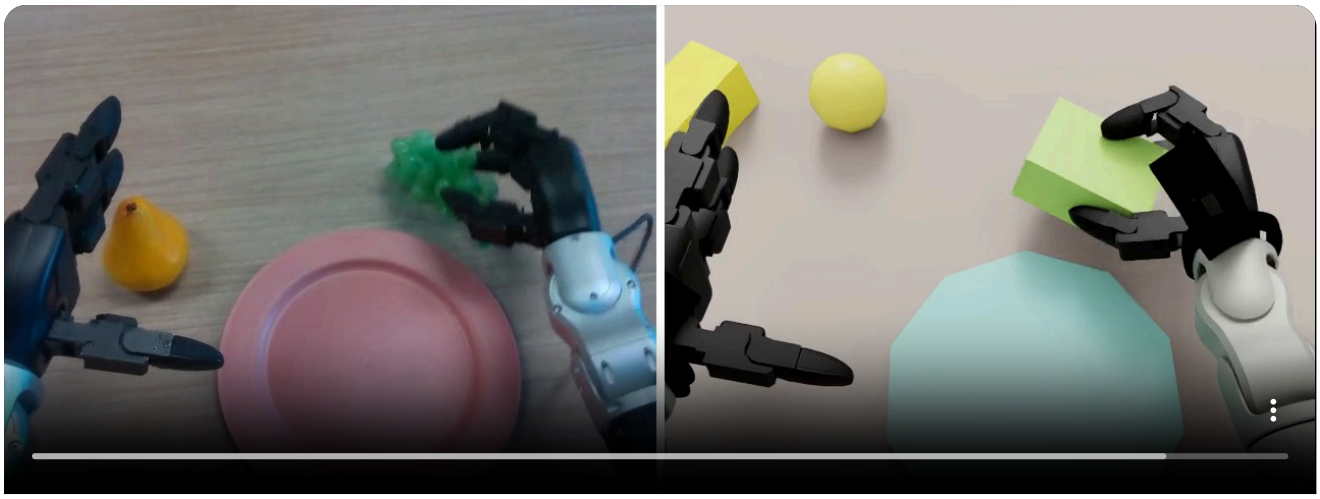
Multi-task, 4 fruit; train/test split by source episode

- **Vision-free reconstruction:** assume both feet are flat on the floor to infer the torso tilt; detect grasp/release from finger closure; custom FK written with numpy only.
- **Simulation:** G1 + Dex3 with fixed pelvis, D435 camera with RealSense parameters, physics 200 Hz / control 20 Hz, stiff PD for the arms, fruit friction 1.5, fixed plate.
- **Scoring:** fruit lifted > 3 cm; finally lying on the plate; no distractor moved > 1 cm.
- **Data-leakage prevention:** test scenes never appear in train; 100 evaluation scenes are kept, including scenes whose replay failed, so as not to bias toward easy scenes.

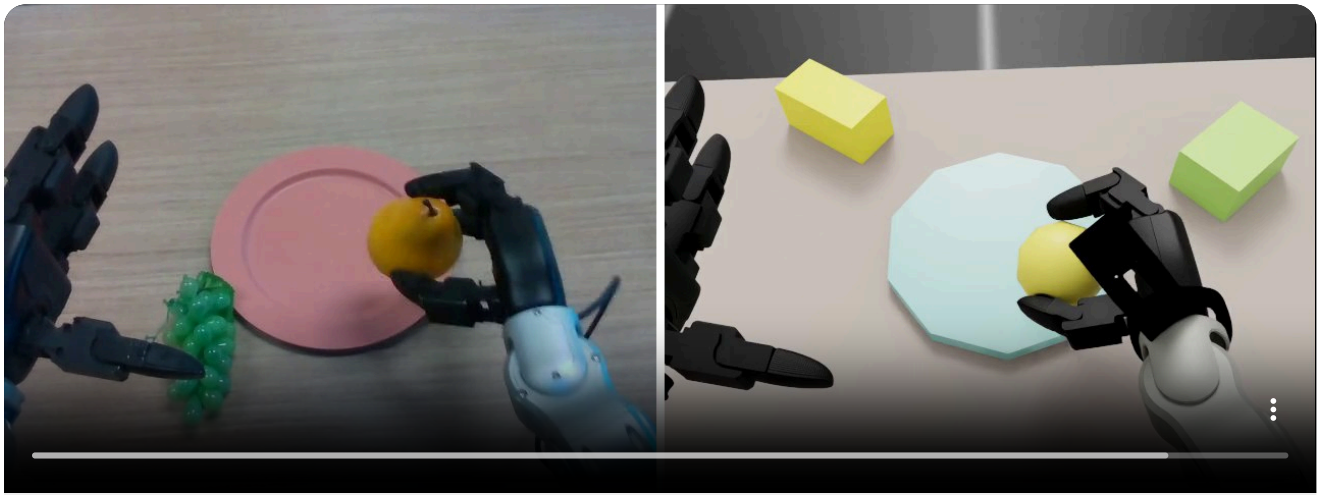
Three real ↔ simulation video pairs. Robot head camera, 20 Hz: left is the original teleop video, right is the real2sim episode replayed from the same joint trajectory. Both episodes have the same length, so they run frame by frame in sync; the poster of each video is a representative frame of one phase (start, grasp, place).



Pair 1 · Apple · real episode 0 ↔ sim 0 · 111 frames (5.6 s). From the very first frame, the apple and plate positions in simulation nearly coincide with the real image even though no vision is used: the apple is inferred from the palm at grasp, the plate from the position at release. The bunch of grapes in the corner of the real image is not in the joint data and so is not reconstructed; the simulation places 2 new distractors (star fruit, grapes) far from the hand's sweep. Left: real · right: simulation.



Pair 2 · Grapes · real episode 1 ↔ sim 0 · 102 frames (5.1 s). The arm and Dex3 hand poses while reaching, closing the fingers and lifting all match the real video because the joint angles are the same. The soft bunch of grapes is replaced by a rigid green block; the real pear on the left is not reconstructed, and the simulation places a pear and a star fruit as distractors elsewhere. Left: real · right: simulation.



Pair 3 · Pear · real episode 0 ↔ sim 0 · 131 frames (6.6 s). The fruit is carried to the plate area at the same moment as in the real video. The real plate is pink, the simulated plate is always a turquoise polygon; the wooden table top becomes a plain plane; the simulated view shows both the table edge and the background behind it. Left: real · right: simulation.

SAME

Motion and task layout

Arm trajectory, finger closure and grasp/release timing are kept from teleop; target fruit and plate positions match the real image; the left hand rests in the same pose; the instruction is kept verbatim.

DIFFERENT

Visuals and distractors

Fruit are simple geometric blocks; plate, table top and lighting come from the simulation; distractors in the real image are dropped and replaced by 2 random, safely placed distractors; the camera angle is slightly off. Therefore the model learns only on rendered images, not on real images.

SPLIT	EPISODES	FRAMES	NOTES
Train	**790**	**86,822**	**4 kinds of fruit, 20 Hz**
Test	89	—	Scenes not in train
Evaluation scenes	100	—	30 apple, 24 pear, 26 star fruit, 20 grapes

7 Workflow

The source dataset is teleop on the real G1 robot, so it cannot run directly in simulation: it must be **converted via real2sim** to obtain the corresponding scenes and simulation data, then GR00T is **fine-tuned** on that data, and only then is the closed loop run in Isaac Sim.

STEP 1
Real data
1,095 G1 teleop trajectories, joint angles + video only

→

STEP 2
Real2sim conversion
FK infers fruit and plate positions; replay & filter in Isaac Lab

→

STEP 3
Simulation dataset
Multi-task LeRobot · train 790 / test 89

→

STEP 4
Fine-tune
GR00T N1.7 · 20,000 steps · 1× H100

→

STEP 5
Run & evaluate
Randomized scenes, 50 episodes

STEP	WORK	NOTEBOOK / TOOL	RESULT
1	Download the 4-fruit teleop dataset (apple, pear, star fruit, grapes); try the apple checkpoint from scenario 1	PhysicalAI-Robotics-GR00T-Teleop-G1	Reused apple checkpoint: 0 %
2	URDF forward kinematics → hand trajectory, grasp/release timing → fruit and plate positions; place distractors outside the hand's sweep; replay actions in Isaac Lab, keep successful episodes	`g1_real2sim` (urdf_fk, scene, distractors, isaac_scene)	A simulated scene for each real trajectory

STEP	WORK	NOTEBOOK / TOOL	RESULT
3	Merge the 4 fruit into one multi-task dataset; split train/test by source episode; keep 100 evaluation scenes	make_groot_dataset, lerobot_io	790 / 86,822 frames train · 89 test
4	Fine-tune GR00T N1.7-3B, `NEW_EMBODIMENT`, 31-dim state/action, horizon 16 steps @ 20 Hz, batch 32	launch_finetune.py on the H100 server	`g1_fruits_n17` · 2 h 34 min · loss 1.50 → 0.019
5	Build scenes like the dataset, choose an instruction, the model picks on its own; run N episodes with separate seeds	nb09_multi_object_task_pipeline, nb10_eval_multi_object	66 %, never picked the wrong fruit

8 Fine-tune

Following NVIDIA's official guide (fine-tuning on sim data): GR00T N1.7-3B, `NEW_EMBODIMENT`, camera `rs_view`, 5 state/action groups (31 dims), horizon 16 steps @ 20 Hz (0.8 s); batch 32 on H100; LLM frozen, vision + projector + diffusion tuned; color jitter (brightness 0.3 · contrast 0.4 · saturation 0.5 · hue 0.08).

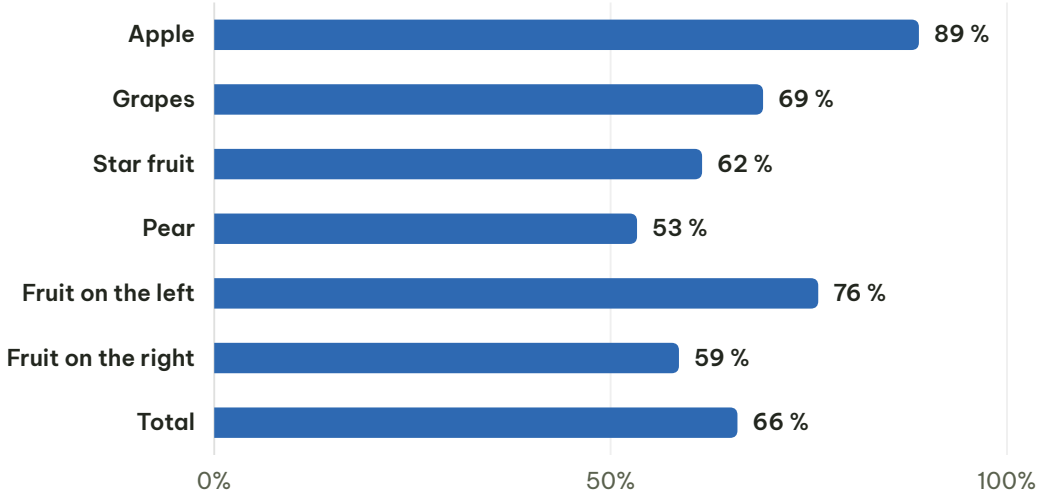
9 Results

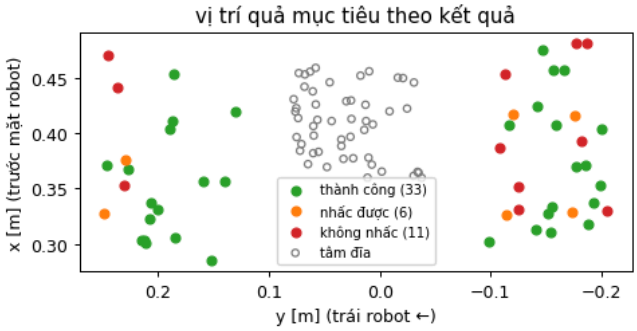
Each episode randomizes the target fruit, fruit position/orientation, plate position and distractor fruit; a separate seed per episode. Instructions verbatim from the dataset (e.g. "Pick up the green grapes and place it on the plate"). Receives 16 steps, executes 8, then queries again; up to 10 s. 50 episodes, 0 technical errors.

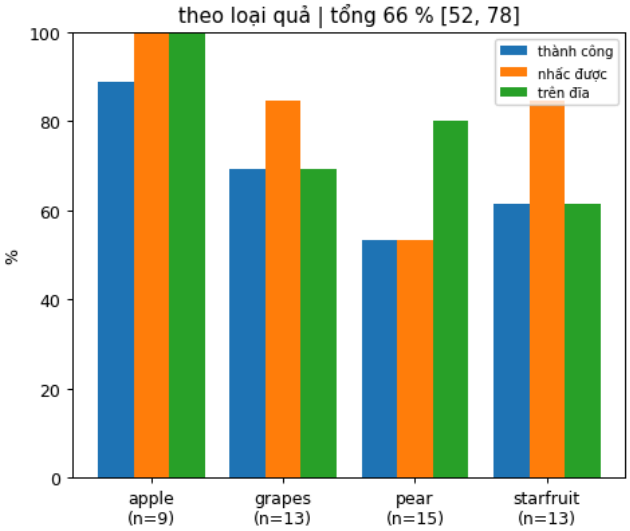
MODEL	SUCCESS	WILSON 95 %	LIFTED	ON PLATE	WRONG PICK
Real2sim fine-tune	**33/50 (66 %)**	**52–78 %**	**78 %**	**76 %**	**0 %**
Arena apple checkpoint (baseline)	0/50 (0 %)	0–7 %	0 %	0 %	0 %

Success rate by group · real2sim model

Hover over a bar to see the number of trials. Small n (9–29) ⇒ wide confidence intervals.







Target fruit positions by outcome (green = success, orange = lifted but not successful, red = not lifted, grey ring = plate centre) and rate per fruit.

OBSERVATION

The model understands the instruction

Never picked the wrong fruit even though other fruit are always on the table.

OBSERVATION

Failures occur mainly in the lift phase

11/50 not lifted; the pear has the lowest lift rate (53 %).

NOTE

The baseline is a model for a different task

Not the original N1.7 without fine-tuning; 0 % only shows that a model for a different task cannot be reused.

Technical lesson: the first trial run used a configuration that did not render the camera at every control step (an Isaac Lab 3.0 specific) — the model saw a frozen image. This was detected and fixed; the results of that run are not used.

10 Result analysis

The 50 episodes of the fine-tuned model are tracked through each phase to see where the losses occur.

PHASE (50 EPISODES)	EPISODES	RATE
Target fruit lifted	39	78 %
Lifted and fruit placed on the plate	34	68 %
Success (distractors not moved)	**33**	**66 %**

FAILURE TYPE (17 CASES)	COUNT	SYMPTOM
Not lifted	11	Grasp slips or the hand does not close enough; the pear has the lowest lift rate (53 %)
Lifted but not on the plate	5	Released off the plate; the star fruit has the lowest on-plate rate (62 %) despite an 85 % lift rate
On the plate but distractor moved	1	Hand touched a distractor fruit > 1 cm

FINDING 1

The model understands the instruction

0 wrong picks in 50 episodes that always have other fruit on the table: the language part is learned well from the real2sim data.

FINDING 2

The bottleneck is the grasp phase

11/17 failures are failures to lift the fruit. Further improvement should focus on grasp data (shape, friction of each fruit).

FINDING 3

Differences by fruit and table side

Apple 89 %, grapes 69 %, star fruit 62 %, pear 53 %; left 76 %, right 59 %. Small n, overlapping confidence intervals — not enough to conclude.

FINDING 4

Scope of conclusions

Training and evaluation both in simulation: the results serve for relative comparison and do not predict performance on the real robot.

11 Scenario conclusion

CONCLUSION

From teleop data of the real G1 robot containing only joint angles and video, the project **built a simulation dataset with real2sim** (790 episodes) and fine-tuned GR00T: from **0 %** with the reused apple checkpoint to **66 %** on randomized scenes with distractor fruit, **never picking the wrong fruit**. The model understands the instruction; the remaining losses lie mainly in the grasp phase.

- Built a **real2sim pipeline from joint data alone**: forward kinematics infers fruit and plate positions, distractors are placed safely, and replay and filtering run automatically in Isaac Lab.
- Fine-tuned GR00T N1.7 in 2 h 34 min on one H100; 11/17 failures are failures to lift the fruit, with the pear having the lowest lift rate.
- The results are only for **relative comparison** in simulation: simulation is easier than reality (stiff PD, high friction, fixed plate) and does not predict performance on the real robot.
- Next steps: increase cheap data by changing distractor layout, lighting and materials and replaying; add grasp data by shape and friction of each fruit.

III.03 Scenario Hammer & Bin

G1 picks a hammer from the shelf and drops it into a bin.

A completely new task designed by the project: the left hand grasps the hammer handle, lifts it over the bin wall, drops it in the middle of the bin and pulls the hand away. It is harder than the apple because the hammer is **long, thin and lies close to the shelf surface**, and easily ends up resting on the bin rim. The project built the environment, collected **100 teleop demos** with a Meta Quest headset, generated additional demos and fine-tuned GR00T (~1 hour on 1 H100) — the full cycle took about 6 days. Results: untuned **0 %** → 200 episodes **66 %** → 400 episodes **63 %** → relaxed at-rest threshold **73 %**.

DATASET

200 · 400 episodes

Source	**Teleop + generated**
Frames	**28,022 · 54,541**
Rate	**50 Hz**

FINETUNE

10k · 20k steps

Time	**61 min · 2 h 15**
Batch · GPU	**32 · 1× H100**
Final loss	**0.020 · 0.019**

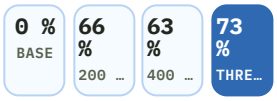
MODEL

GR00T N1.7 3B

mix200	`hammer_mix200_n17`
mix400	`hammer_mix400_n17`
Action chunk	**40 steps · 50 Hz**

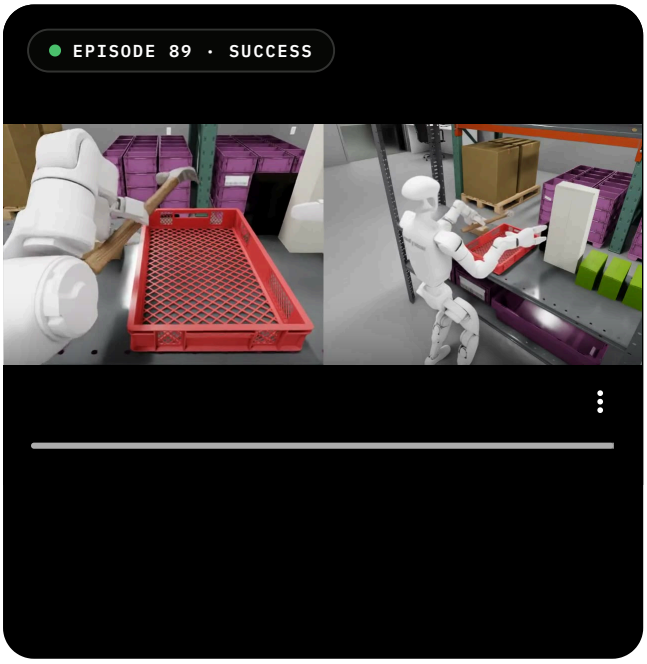
RESULT

73 % success



At-rest threshold relaxed 0.01 → 0.02

1 Video Showcase



The robot grasps the hammer handle, lifts it over the bin wall, drops it in the middle of the bin and lets go

> INSTRUCTION SENT TO THE MODEL
> `"put the hammer in the bin"`

3.0 s
Time to success

12.8 cm
Maximum hammer lift

0.7 cm
Hammer centre to bin centre

0 N
Hand force on hammer: released

ⓘ Taken from the 100-episode run with the best configuration (73/100).

2 Scenario setting

The G1 robot stands in front of a shelf in the Galileo lab; the hammer lies on the left with its head propped on a small block, and a red plastic bin sits on the right. The image below is the **first frame** of the episode shown in the Video Showcase.



Frame 0 · left: robot head camera, the image the model receives · right: external camera, used only for recording video.

SCENE & OBJECTS	DETAILS
Background	Galileo lab; the boxes shield_controller_box_02, jetson_orin_05 and jetson_orin_07 are hidden so the hand does not collide (clearance ≥ 2 cm)
Hammer	`hammer_2_handal_robolab` 33 × 13 × 3 cm at (0.63, 0.25) m, yaw 0; the hammer head rests on a 5 cm block (tilted ~10°) so the handle is easy to grasp
Bin	`red_container` scaled 0.65 → 40 × 27 × 5.9 cm at (0.68, 0.00) m
Distance	Hammer is 5.1 cm from the bin edge

ROBOT, CAMERAS & CONTROL	DETAILS
Robot	Unitree G1 + Dex3 at (0.25, 0.08) m; the left hand grasps the handle, lifts it over the 5.9 cm bin wall, moves ~25 cm to the right and releases
Cameras	Robot head camera (model input) + 1 external camera for recording
Rate	50 Hz control; each model call returns a chunk of 40 steps
Episode	At most 300 steps = 6 s
Instruction	"put the hammer in the bin"
Success	Hammer centre ≤ 10 cm from the bin centre and below the wall; hammer at rest (< 0.01 m/s, < 0.1 rad/s — later relaxed to 0.02 m/s); hand released (22 hand links ≤ 0.01 N); held for 10 consecutive steps

3 Scenario goals

This scenario tests the ability to **teach the robot a new task from scratch** with tools and data built by the project.

GOAL 1

A completely new task

Define a task with no existing data or model: pick the hammer from the shelf and drop it into the bin.

GOAL 2

The full cycle

Go through every stage: building the environment, collecting teleop demos, data augmentation, fine-tuning, evaluation.

GOAL 3

Measure cost and benefit

Know how much data and training time is needed, and how much each improvement adds to the success rate.

4 Scenario data

As a new scenario, there was no existing dataset: the project collected its own demos with **Meta Quest headset teleop** in Isaac Lab at 50 Hz, then added generated demos (card 6). Every demo was replayed against the strict success condition before being used for training.

DATASET HAMMER_TO_BIN	VALUE
Source	Meta Quest headset teleop in Isaac Lab, 50 Hz; the first 100 real demos were collected over 10 sessions
Real demos	~13,700 steps; hammer lifted 10.6 cm on average; final hammer centre 3.7 cm from the bin centre (max 8.4 cm)
mix200 set	200 episodes (100 real + 100 augmented), 28,022 frames (~9.3 min), LeRobot v2.1, 131 MB
mix400 set	400 episodes (mix200 + 200 new teleop demos), 54,541 frames (~18 min)
Instruction	"put the hammer in the bin"
Check	All demos successful; for mix200: minimum hammer lift 6.4 cm, mean final distance 3.8 cm

5 Problems encountered

The hammer → bin task is completely new: no data, no model, no ready-made scene. The biggest difficulty is getting enough good data for fine-tuning.

> **Nothing to start from.** All data had to be collected by teleop, which is the most labour-intensive stage: 100 real demos took 10 sessions.

PROBLEM 1

No existing data

There is no dataset or checkpoint for picking a hammer and placing it in a bin on the G1; the base model without fine-tuning cannot do the task.

PROBLEM 2

Teleop is slow and laborious

The operator wears a Meta Quest headset and performs each demo by hand; one session yields only about 10 demos, so many sessions are needed.

PROBLEM 3

Teleop is unstable

The robot hand freezes when the controller signal is lost, and it lags from the second attempt onwards; many attempts had to be discarded and redone.

PROBLEM 4

Uneven demo quality

Some demos were scored as successful while the hammer still rested on the bin wall or the hand had not let go; bad demos mixed into the data teach the model the wrong behaviour.

6 Proposed approach

Solved by **generating additional data**: from real teleop demos, new demos are created automatically in the style of MimicGen, replayed in simulation, and only those meeting the strict success condition are kept. This multiplies the data without extra teleop hours, and every demo used for training has been verified.

GENERATION

Transforming template demos

Real demos serve as templates (60 different templates); time is scaled ×0.85–1.2; start (σ 1 cm), grasp (σ 4 mm) and release (±1–1.2 cm) points are shifted and joined with min-jerk trajectories; 0.2–0.6 mm hand jitter is added.

FILTERING

100/133 attempts kept

Each generated demo is replayed in simulation against the strict success condition; 100 of 133 attempts passed and went into the mix200 set.

METRIC	AUGMENTED	REAL
Episode length (steps)	145	137
Hammer lift	10.5 cm	10.6 cm
Distance to bin centre, mean	4.0 cm	3.7 cm
Distance to bin centre, max	8.0 cm	8.4 cm



One augmented demo: first, middle and last frames. Poses at the key points differ by less than 1 cm from the real demo; 60 different template demos are used.

7 Workflow

A completely new scenario: no existing data or model. The project built the environment, **created its own data by teleop**, **ran augmentation** to multiply the data, **fine-tuned** GR00T, and only then ran the closed-loop evaluation.

STEP 1 · 07/10
Environment design
Objects, layout, support block, success condition

→

STEP 2 · 08–10/10
Teleop demo collection
Meta Quest · 100 real demos

→

STEP 3 · 08–10/10
Data augmentation
MimicGen-style generation · 100/133 kept

→

STEP 4 · 12/10
Fine-tune
mix200 10k steps · mix400 20k steps

→

STEP 5 · 12–19/10
Run & evaluate
Arena and local Isaac Sim · 50–100 episodes

STEP	WORK	NOTEBOOK / TOOL	RESULT
1	Choose hammer and bin; prop the hammer on a 5 cm block; hide background boxes in the hand's way; write the strict success condition	Isaac Lab Arena, hammer_to_bin/scripts	Final layout, clearance ≥ 2 cm
2	Collect demos with the Meta Quest headset using presets and teleop diagnostics; replay each demo against the strict condition	teleop_kit, record_demos.sh	100 real demos (10 sessions), later 200 more
3	Transform template demos (timing, grasp/release points, hand jitter), replay and filter	gen_scripted_demos.py	100 augmented demos → mix200 set
4	Fine-tune GR00T N1.7-3B; LLM frozen, vision + projector + diffusion tuned	H100 server	mix200: 61 min · mix400: 2 h 15 min
5	Run N closed-loop episodes, classify failures, record videos	Arena eval, nb11_hammer_to_bin, nb12_eval_hammer_to_bin	0 % base → 66 % → 63 % → 73 %

8 Fine-tune

Training time

61 min

1× H100 · 0.36 s/step · ~53 GB VRAM

Steps · batch

10k · 32

~11 epochs · checkpoint every 2k steps

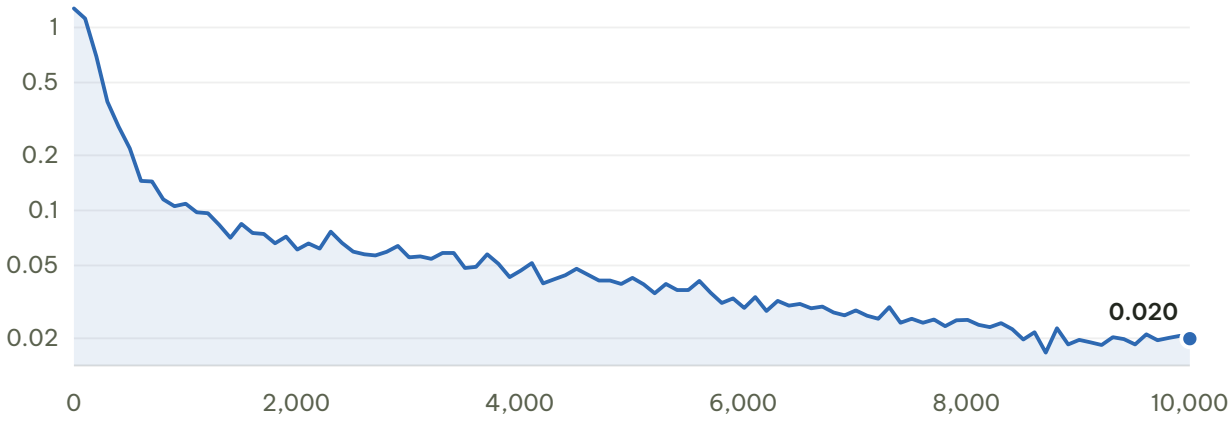
Final loss

0.020

from 1.27 at step 10

Training loss · log scale

GR00T N1.7-3B on mix200. LLM frozen; vision + projector + diffusion tuned. Hover to see each step.



9 Results achieved

Figures measured across the teleop → augmentation → fine-tune → evaluation cycle, in local Isaac Sim (`inside` condition: hammer centre inside the bin, at rest, released).

Best · local

73 %

mix400 · threshold 0.02 · 73/100

Local · mix200

66 %

66/100 · Wilson 56–75 %

Data

400 demos

54,541 frames · 300 teleop + 100 generated

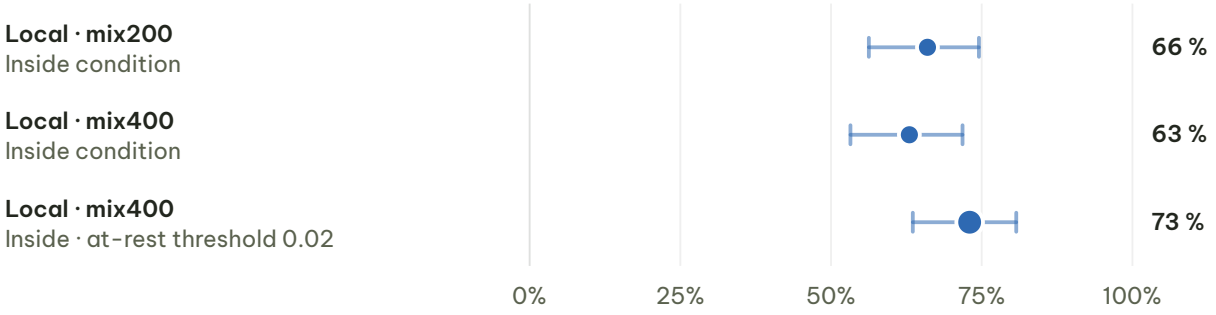
Fine-tune loss

0.019

mix400 · 20k steps · 2 h 15

Success rate with Wilson 95 % confidence interval

Local Isaac Sim · inside condition · 100 episodes per run.

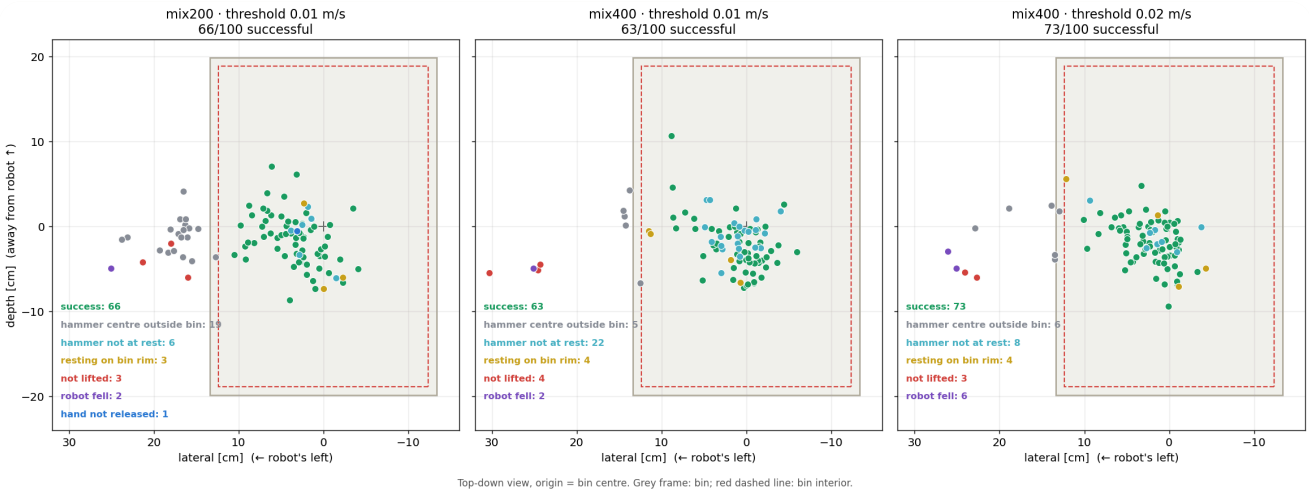


STEP	MEASURED FIGURES
Teleop	100 real demos (10 sessions, ~13,700 steps); later 200 more
Augmentation	133 generation attempts → 100 demos meeting the strict condition
Dataset	mix200: 200 episodes, 28,022 frames · mix400: 400 episodes, 54,541 frames
Fine-tune	mix200: 10k steps, 61 min, loss 0.020 · mix400: 20k steps, 2 h 15 min, loss 0.019
Local · mix200	66/100; lifted ≥ 3 cm 95/100; median final distance 6.1 cm
Local · mix400	63/100; lifted ≥ 3 cm 94/100; median final distance 4.4 cm
Local · mix400, threshold 0.02	**73/100; lifted ≥ 3 cm 91/100; mean time to success 2.5 s**

10 Result analysis

Outcome breakdown of the three 100-episode runs in local Isaac Sim, to see what more data and a relaxed threshold change.

OUTCOME (100 EPISODES)	MIX200	MIX400	MIX400 · MANUAL COUNT	MIX400 · THRESHOLD 0.02
Success	**66**	**63**	**87**	**73**
Hammer centre outside the bin	19	5	5	6
Hammer not at rest at timeout	6	22	0	8
Hammer resting on the bin rim	3	4	2	4
Not lifted (< 1 cm)	3	4	4	3
Robot fell	2	2	2	6
Hand not released	1	0	0	0



Hammer centre position at the end of each episode, top-down view, origin at the bin centre · left: mix200 · middle: mix400 · right: mix400 with at-rest threshold 0.02 m/s. With mix200, the hammer was often dropped off to the left-hand side, just outside the bin wall (grey cluster); with mix400 the points gather more around the bin centre. With mix400 at threshold 0.01, most failures are hammers already in the middle of the bin but not at rest at timeout (22 light-blue points); in the run with threshold 0.02 this group shrinks to 8. Red points (not lifted) and purple points (robot fell) are cases where the hammer never reached the bin.

FINDING 1

Grasping and lifting work

The hammer was lifted ≥ 3 cm in 91–95/100 episodes in all three runs; only 3–4 failed to lift it. Losses are mainly in the release phase.

FINDING 2

More data places the hammer more precisely

mix400 reduces hammers landing outside the bin from 19 to 5, and the median final distance from 6.1 to 4.4 cm.

FINDING 3

The at-rest threshold is the remaining bottleneck

The middle plot shows that with mix400 (threshold 0.01 m/s), all 22 "not at rest" failures lie inside the bin with the hand released. These points are even closer to the bin centre than the successes (median 2.9 cm vs 4.3 cm). The hammer only keeps moving slowly until the 6 s timeout (final velocity 0.005–0.023 m/s, rotation 0.04–0.32 rad/s), so it never meets the at-rest condition. These are failures of the scoring rule rather than of placement. In the run with threshold 0.02 m/s (right plot), this group drops to 8 and the rate rises to 73 %.

FINDING 4

Why a hammer lying still is scored as "not at rest"

The at-rest condition (kept unchanged from Arena) only looks at the hammer's **instantaneous velocity** at each simulation step: it must be below 0.01 m/s and 0.1 rad/s at the same time. All 22 hammers lie inside the bin with the hand released, and the videos we reviewed show no drifting. If the hammer really moved at the measured speed (0.5–2.3 cm/s) for the ~3.5 s from reaching the bin centre to timeout, it would drift several centimetres; in fact the final distance to the bin centre differs from the closest approach by only 1.2 cm (median). So the hammer is mostly **vibrating in place**: the physics simulation does not bring the velocity exactly to 0 when the hammer lies on the bin's mesh floor (12/22 hammers lie flat on the floor, versus 8/63 successes), and the bin itself is a dynamic body standing on the shelf. This small vibration exceeds 0.01 m/s, so an instantaneous threshold is too strict for the simulation noise; a more reasonable rule is to check whether the hammer moves over a time window (e.g. 0.5 s).

FINDING 5

More falls are not caused by the model or the threshold

The 73 % run had 6 robot falls, versus 2 in each of the other two runs. All 6 happened right at the start of the episode, within 0.02–0.78 s (at most 39/300 steps), before the hand reached the hammer: the hammer was still on the shelf, 25.6 cm from the bin centre. At that point the model had barely started controlling the robot, and the at-rest threshold is only used to score the end of the episode, so it cannot cause a fall. This matches the reset issue recorded in nb12: when the previous episode ends with the arm and torso far from the default pose, the reset does not fully clear the old state (hips still bent, joints still moving), and the robot steps or falls on the first tick. Even with a double reset, replaying 20 demos still gave 2/20 bad starts. The difference of 6 vs 2 is not statistically significant (Fisher $p = 0.28$); across the three runs it is 10/300 episodes (≈ 3 %). Excluding these falls, the 73 % run reaches 73/94 ≈ 78 %. The reset step needs fixing (e.g. checking pose and joint velocities before starting an episode) to remove this noise from the evaluation.

> **Note:** the run with threshold 0.02 is a new 100-episode run, not a re-scoring of the earlier run, so it is still subject to random variation.

11 Scenario conclusion

CONCLUSION

The project **taught the G1 a completely new task on its own**: from 0 % untuned to **66 %** with only 200 episodes (100 teleop + 100 generated) and ~1 hour of fine-tuning on one H100, with the full cycle taking about 6 days. Grasping and lifting work; the remaining losses are in dropping the hammer into the bin and in how runs are scored.

- Completed the full cycle with in-house tools: environment building, teleop demo collection, **data generation**, fine-tuning, evaluation.
- Generating demos multiplies the data without extra teleop hours (133 generation attempts → 100 demos meeting the strict condition).
- Doubling the data (mix400) places the hammer more precisely (outside the bin 19 → 5 times) but the overall rate did not rise (63 %); with an at-rest threshold of 0.02 it reaches **73 %**.
- Not yet addressed: change the "at rest" scoring to check hammer displacement over a time window instead of instantaneous velocity; fix the reset step that makes the robot fall at the start of an episode (≈ 3 % of episodes).

IV.01 Deliverables

What the client receives, where it is, how to use it.

All deliverables of this phase are in **one folder with 6 parts**, numbered in the recommended reading order: report → source code → data → model → results → simulation assets. The raw data and checkpoints are **provided separately**.

FOLDER	MAIN PACKAGE	PROVIDED SEPARATELY
shimizu-robot/	**6 parts**	**Raw data + checkpoints**

1 Deliverable package structure

Each tile is a subfolder. Click a tile to see the list of files inside (card 3).

01 report/

Report

This interactive report (opens in a browser, no installation needed) and a markdown version for editing.

View contents →

02 codes/

Source code

Standalone control pipeline, 12 research notebooks, real2sim tools, Arena patches and the teleop script kit.

View contents →

03 datasets/

Training data

Hammer → bin datasets (200 and 400 episodes) and the real2sim 4-fruit dataset (790 train + 89 test), in LeRobot format.

View contents →

04 models/

Trained models

3 fine-tuned checkpoints (hammer → bin mix200, mix400; real2sim 4 fruits), ready to run on the GR00T server.

View contents →

05 evals/

Evaluation results

Per-episode logs, summary tables, charts and success / failure videos for the 3 scenarios.

View contents →

06 assets/

Simulation assets

3D files used to build the scenes: an offline copy of NVIDIA files (Galileo background, hammer, bin, G1 robot) plus the apple and plate.

View contents →

2 Intended users by role

Not everyone needs the whole package. The table below suggests which parts to read first, by role.

PART	PROJECT MANAGER	SOFTWARE ENGINEER	AI / ROBOTICS ENGINEER	DATA COLLECTOR (TELEOP)
01 Report	✓	·	·	·
02 Source code		✓	✓	·
03 Data			✓	·
04 Model		·	✓	
05 Results	✓		✓	
06 Assets		·	✓	

■ Primary use ▢ Reference when needed

3 Detailed content inventory

Click a folder to see the main files inside.

01 **report/ — Report** –

`Open: index.html` `No installation needed`

- `index.html` — **interactive report**: table of contents, videos, comparison tables, PDF export
- `assets/`, `images/` — interface, demo videos, images
- `src/`, `build.py`, `README.md` — content of each section and how to edit the report
- `markdown/` — text version of the report (10 files) for quoting and editing

02 codes/ — Source code

Python · 84 unit tests · Installed environment not included

- `standalone_g1_gr00t/` — pipeline controlling G1 with GR00T, independent of Arena
  - `g1gr00t/` — package: server connection, joint mapping, observations, actions, safety layer, AGILE, Isaac Lab backend, runner
  - `notebooks/` — 12 research notebooks (nb01 → nb12)
  - `configs/` — configurations for the apple and hammer scenarios and the balance controller
  - `scripts/`, `tools/` — closed-loop runs, open-loop evaluation, asset download, automated notebook execution
  - `tests/` — unit tests and in-simulation tests
  - `README.md`, `Makefile` — installation and run commands
- `g1_real2sim/` — scene reconstruction from real G1 data: FK from URDF, distractor objects, Isaac Lab scene, LeRobot read/write, GR00T configuration
- `arena_bua_thung/` — Isaac Lab Arena extension for the hammer scenario
  - `patches/` — 4 environment patches: hammer-bin layout, support block, strict success condition
  - `teleop/` — scripts to launch teleop, record, replay, merge demos, diagnose
  - `scripts/` — augmented demo generation, dataset checks, scene verification

03 datasets/ — Training data

LeRobot v2.1 format · H.264 video 640×480

FOLDER	CONTENTS
`hammer_to_bin/static_hammer_mix200/`	200 episodes (100 real teleop + 100 generated), 28,022 frames @ 50 Hz
`hammer_to_bin/static_hammer_mix400/`	400 episodes (mix200 + 200 new teleop demos), 54,541 frames @ 50 Hz
`real2sim_multi_fruit/g1-fruits-sim-train/`	Simulated training set from real G1 data: 790 episodes, 86,822 frames @ 20 Hz (apple 246, pear 211, star fruit 174, grape 159)
`real2sim_multi_fruit/g1-fruits-sim-test/`	Test set: 89 episodes, 9,906 frames, scenes not seen in training; `test_scenes.json` lists the evaluation scenes

Each dataset contains `meta/` (description, instructions, statistics), `data/` (joint angles, actions per episode) and `videos/` (robot head camera).

04 models/ — Trained models

Each folder is a checkpoint ready to use with the GR00T server: weights (3 `safetensors` files), config, processor, normalisation statistics, plus the training log and script; the optimizer state is not included.

CHECKPOINT	TRAINED ON
Hammer → bin, 200 episodes `hammer_to_bin_mix200/` · step 10,000	mix200 · 10,000 steps · 1× H100 · 61 min
Hammer → bin, 400 episodes `hammer_to_bin_mix400/` · step 20,000	mix400 · 20,000 steps · 1× H100 · 2 h 15 min
Real2sim 4 fruits `real2sim_multi_fruit/` · step 20,000	790 simulated episodes · 20,000 steps · batch 32
Apple → plate (NVIDIA) not included in the package	Not retrained — download from Hugging Face into `models/isaaclab_arena/static_apple_tutorial/gn1x_tuned_static_apple/` (the default path in the config)

05 evals/ — Evaluation results

- `apple_plate/` — apple → plate, NVIDIA checkpoint, 50 episodes per run with 50 head-camera videos
  - `self_built_scene_50ep/` — self-built scene: 0/50
  - `galileo_scene_50ep/` — reconstructed Galileo scene: 28/50
- `multi_fruit/` — 4 fruits, 50 episodes per run; videos recorded for the first 10 episodes
  - `finetune_real2sim_50ep/` — real2sim fine-tuned model: 33/50
  - `baseline_apple_ckpt_50ep/` — baseline reusing the apple checkpoint: 0/50
- `hammer_to_bin/` — hammer → bin, 100 episodes per run with 100 videos
  - `arena_mix200_100ep/` — mix200 in Arena (external + head camera): 56/100
  - `local_mix200_100ep/` — mix200, local Isaac Sim: 66/100
  - `local_mix400_100ep/` — mix400, local Isaac Sim: 63/100
  - `local_mix400_at_rest_0.02_100ep/` — mix400, at-rest threshold relaxed to 0.02 m/s: 73/100

`README.md` at the top level summarises the results and origin of each run. Common files in each run: `episodes.jsonl` (one line per attempt), `summary.json` / `summary.md` (success rate, confidence interval, failure classification), videos `…_success.mp4` / `…_fail.mp4`.

06 assets/ — Simulation assets

`Location: ~/workspace/shimizu-robot/assets` `The source code reads from this path`

- `nucleus_cache/` — offline copy of the NVIDIA files Isaac Lab downloads when building a scene: the Galileo background, the `hammer_2` hammer, the red bin, the G1 robot and the lower-body controller model (WBC). With it, runs need no network; without it, Isaac Lab downloads them again.
- `objects/objaverse/apple_01.usd`, `objects/hot3d/clay_plates.usd` — the apple and plate for the apple → plate scenario in the standalone pipeline.

The 4-fruit real2sim scenario does not use this folder (the robot URDF is in `codes/g1_real2sim/assets/`).

4 Separately delivered source data

The raw HDF5 data of the hammer task, needed only to **re-convert or generate more training data**; not needed to run the models, read the report or view the results.

COMPONENT	NEEDED FOR
Raw hammer task teleop data (HDF5)	Re-converting, generating more demos
mix200 training set in HDF5	Converting to LeRobot with a different configuration
mix400 training set in HDF5	same as above

Not included: installed Python environments (recreate them with the commands in the source code) and NVIDIA's original data and checkpoints (download from Hugging Face).

5 Licensing and terms of use

COMPONENT	TERMS
Base model GR00T N1.7-3B and the checkpoints fine-tuned from it	Under the NVIDIA Open Model Agreement
NVIDIA apple → plate checkpoint (Arena)	Model card states "ready for non-commercial use only"
NVIDIA datasets (apple → plate, real G1 data)	CC-BY-4.0: usable, attribution required

COMPONENT	TERMS
Hammer task teleop data, generated dataset, real2sim dataset	Created by the project
VLA model demo videos (section II.01)	Third-party
3D assets in `assets/` (NVIDIA Isaac / Arena files, Objaverse apple, HOT3D plate)	Owned by NVIDIA and third parties; check the terms before redistributing

IV.02 Environment setup

Prepare two machines to re-run the scenarios.

Sections IV.02–IV.04 explain how to re-run the scenarios yourself to verify the figures in this report. Two machines are needed: a **workstation** that runs the simulation and notebooks (client), and the client's **GPU server** that runs GR00T (server). This section sets up both sides; starting the server and running the evaluation are covered in IV.03 and IV.04.

CLIENT

Isaac Sim 6.0.1 · Isaac Lab 3.0.0-beta2 · standalone_g1_gr00t

SERVER

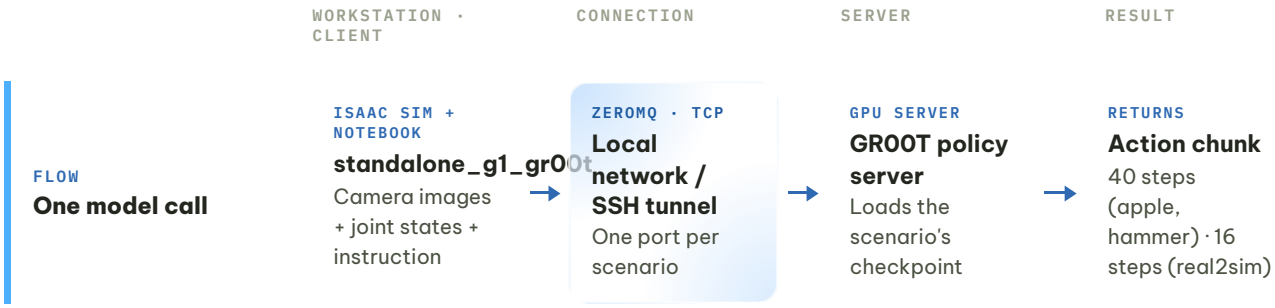
Client's GPU server · Isaac-GR00T 4b1dca9

CONNECTION

ZeroMQ over the local network or an SSH tunnel

1 Connection architecture

The notebook on the workstation sends camera images, joint states and the instruction to the GR00T server; the server returns an action chunk. The two machines can connect directly over the local network, or through an SSH tunnel if the server only listens on `127.0.0.1`.



2 Hardware and software requirements

WORKSTATION · CLIENT

Simulation and notebooks

- Ubuntu 24.04, NVIDIA RTX GPU (the project used an RTX 5090), driver 580
- Isaac Sim **6.0.1** installed locally at `~/isaac-sim-6` (Python 3.12)
- Isaac Lab **release/3.0.0-beta2** at `~/IsaacLab`, installed into the same Python as Isaac Sim
- Isaac Lab Arena **release/0.2.1** at `~/IsaacLab-Arena` (source of the joint YAML files and the modality config)
- `uv`, `make`, `git`; Internet access for the first installation

SERVER

Client's GPU server

- Linux, CUDA-capable NVIDIA GPU (the project used 1× H100 80 GB)
- Isaac-GR00T commit **4b1dca9**, `uv` environment (Python 3.10), `ffmpeg`
- Hugging Face access to the base model `nvidia/GR00T-N1.7-3B` (NVIDIA license accepted)
- Checkpoints from `models/` and the modality config for each scenario (card 5)
- A port reachable from the workstation, or SSH access

Keep the exact version pair: all results in this report were measured on Isaac Sim 6.0.1 + Isaac Lab 3.0.0-beta2 and Isaac-GR00T 4b1dca9. Isaac Sim 6.1 requires driver ≥ 595 and has not been tested with this code.

3 Place the deliverable package on the workstation

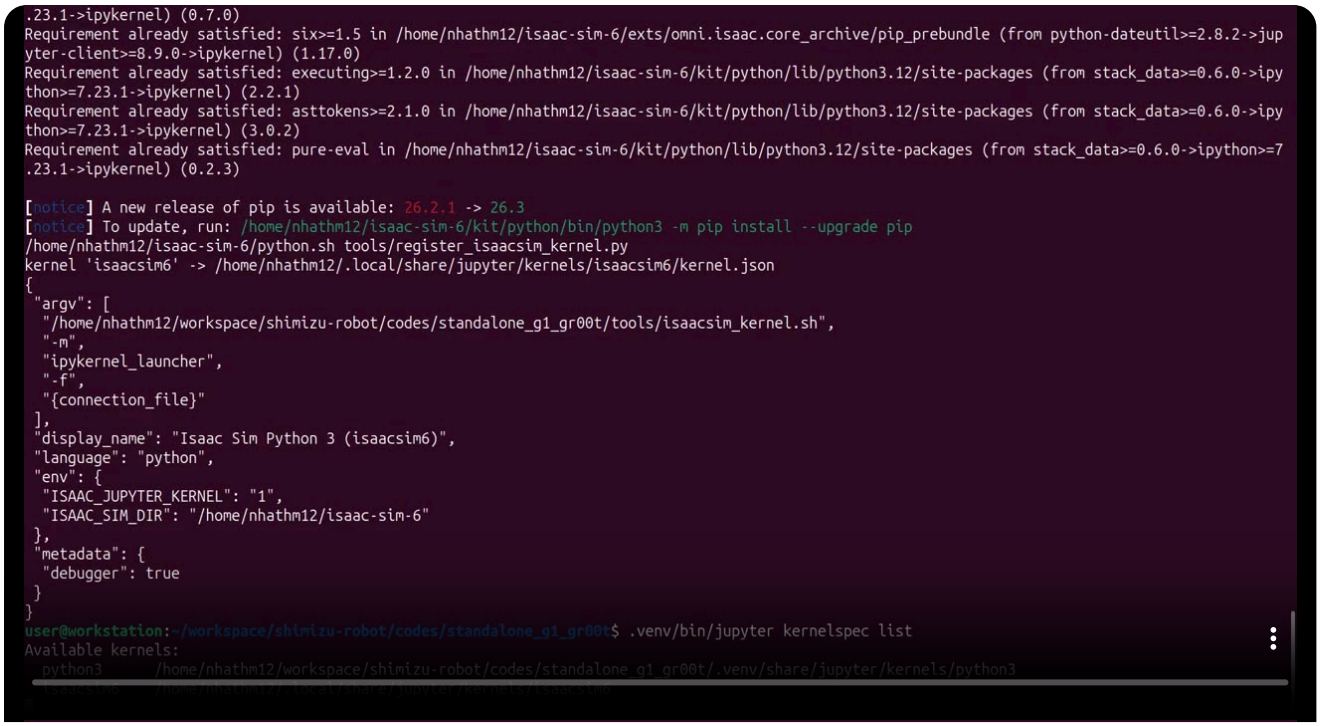
The code reads `assets/`, `datasets/` and `models/` from `~/workspace/shimizu-robot/`, so extract the package at this location. Isaac Sim, Isaac Lab and Arena go in the home folder as in card 2.

```
~/workspace/shimizu-robot/      # deliverable package (IV.01)
├── codes/standalone_g1_gr00t/  # client: g1gr00t package, notebooks, Makefile
├── assets/                     # nucleus_cache + apple, plate
├── datasets/                   # LeRobot datasets
└── models/                     # checkpoints (copy to the server, card 6)
~/isaac-sim-6/   ~/IsaacLab/   ~/IsaacLab-Arena/
```

Installing elsewhere: the Makefile lets you override `ISAAC_PY` (Isaac Sim's Python), `ARENA_DIR` and `ISAAC_GR00T_DIR`, e.g. `make sim-test ISAAC_PY=/opt/isaac-sim/python.sh`. Dataset and model paths are in `configs/*.yaml`.

4 Install the client on the workstation

The `standalone_g1_gr00t` package uses two environments: **.venv** (Python 3.10, for tests and notebooks without simulation) and **Isaac Sim's Python** (for notebooks with simulation).



Sample video: running the client check commands on a workstation with Isaac Sim installed (the `make sim-test-scene` part is sped up). The host names shown are `user@workstation` and `user@gpu-server`.

```
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t

# .venv environment
make venv            # uv venv .venv --python 3.10, install package + dev + notebook
make sync            # copy 4 joint/WBC YAML files from ~/IsaacLab-Arena, record the source hash
make assets          # download the AGILE leg policy (ONNX) into assets/wbc/
make test            # unit tests that need no simulation

# Isaac Sim environment
export PYTHONNOUSERSITE=1   # keep packages in ~/.local out of Isaac Sim's Python
make sim-install     # install the package into ~/isaac-sim-6/python.sh
make sim-test        # unit tests with Isaac Sim's Python
make sim-test-scene  # scene tests: robot stands stably, table / apple / plate
make sim-kernel      # register the "isaacsim6" Jupyter kernel for simulation notebooks
```

DATA ON THE WORKSTATION	PATH	USED FOR
AGILE leg policy	`codes/standalone_g1_gr00t/assets/wbc/` (downloaded with `make assets`)	All simulation notebooks
Galileo scene, hammer, bin, G1 robot assets	`assets/nucleus_cache/` (included in the package)	Apple scenario (Galileo scene) and hammer; runs offline

DATA ON THE WORKSTATION	PATH	USED FOR
Apple, plate	`assets/objects/`	Apple scenario (self-built scene)
Hammer → bin dataset mix200	`datasets/hammer_to_bin/static_hammer_mix200/`	nb12: demo replay, image colour matching
NVIDIA apple → plate dataset	`datasets/isaaclab_arena/static_apple_tutorial/` (downloaded from Hugging Face)	Demo replay, open-loop for the apple scenario

Done when: `make test`, `make sim-test` and `make sim-test-scene` all pass with no failing tests, and Jupyter shows the **Isaac Sim Python 3 (isaacsim6)** kernel.

5 Install the server on the GPU server

Install Isaac-GR00T at the exact commit used for training. The table below shows which checkpoint and modality config each scenario needs; card 6 explains how to put them on the server.



Sample video: cloning Isaac-GR00T at the right commit, creating the `uv` environment and checking the GPU on the server (download parts are sped up). The host names shown are `user@workstation` and `user@gpu-server`.

```
# prerequisites: git, uv, ffmpeg (sudo apt-get install -y ffmpeg)
git clone https://github.com/NVIDIA/Isaac-GR00T.git ~/Isaac-GR00T
cd ~/Isaac-GR00T
git checkout 4b1dca9d88d2a0b9ea5a65aa61c82ff89f5c4f0e
git submodule update --init --recursive
```

```
uv sync --python 3.10      # create the environment, install GR00T and GPU libraries (a few minutes)
uv run python -c "import gr00t, torch; print('GR00T ok | CUDA', torch.cuda.is_available())"
# if the base model is not yet in the Hugging Face cache, log in with an account
# that has accepted the nvidia/GR00T-N1.7-3B license:
hf auth login
```

SCENARIO · CLIENT PORT	CHECKPOINT IN `MODELS/`	MODALITY CONFIG
Apple → plate 5556 · `g1_static_apple.yaml`	NVIDIA checkpoint downloaded from Hugging Face	Arena version

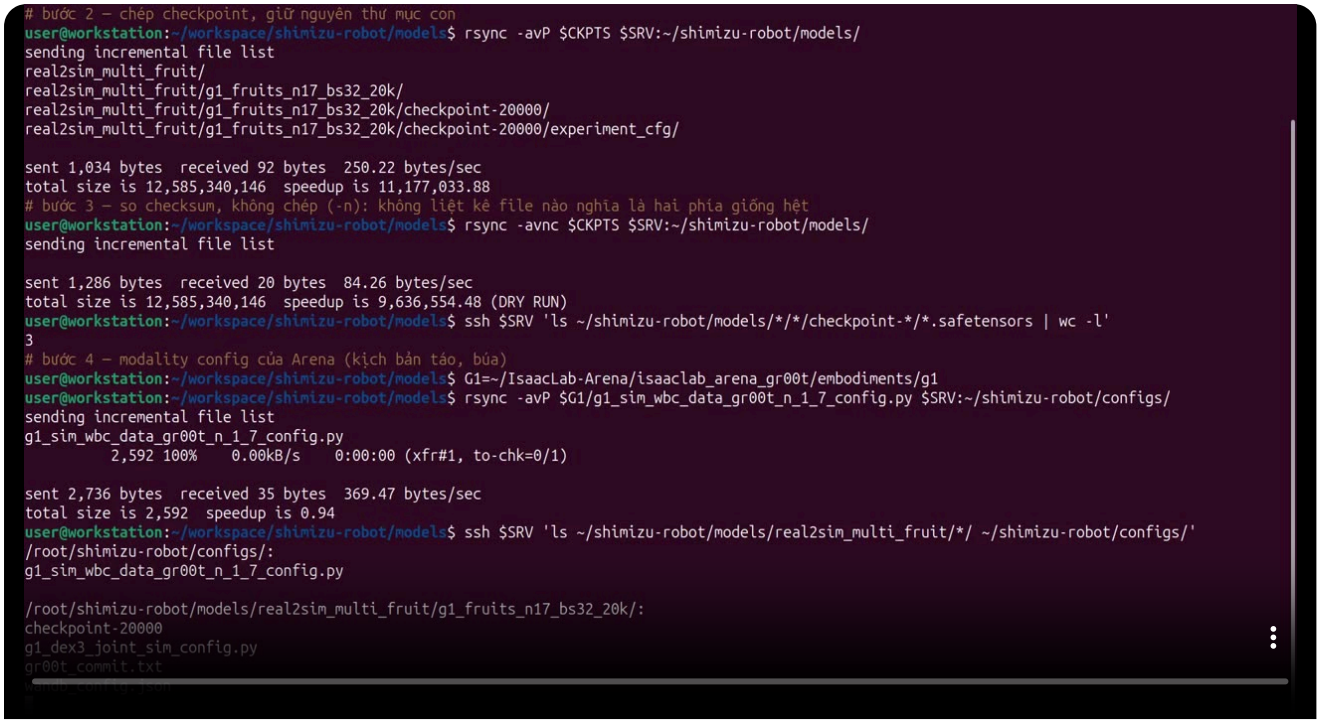
Hammer → bin
5557 · `NB12_PORT`
	`hammer_to_bin_mix200/`
	`hammer_mix200_n17/checkpoint-10000`
	`hammer_to_bin_mix400/`
	`hammer_mix400_n17/checkpoint-20000`
		Arena version

Real2sim 4 fruits 5556 · `NB10_PORT`	`real2sim_multi_fruit/` `g1_fruits_n17_bs32_20k/checkpoint-20000`	Real2sim version

The modality config must match the one used in training: with the wrong file the server still runs, but the model receives the joints in the wrong order. The Arena version is `isaaclab_arena_gr00t/embodiments/g1/g1_sim_wbc_data_gr00t_n_1_7_config.py` in Arena **release/0.2.1** (copy that file alone to the server, or clone Arena). The real2sim version is `g1_dex3_joint_sim_config.py`, located in `real2sim_multi_fruit/g1_fruits_n17_bs32_20k/` (next to `checkpoint-20000`) and in `codes/g1_real2sim/groot/`. Compare `md5sum` on both sides before running.

6 Copy the delivered checkpoints to the server

The three fine-tuned checkpoints are in the `models/` folder of the deliverable package, and each folder works directly with the GR00T server: weights (3 `safetensors` files), configuration, processor and normalization statistics. Copy only the checkpoints for the scenarios you want to verify.



Sample video with the real2sim checkpoint. The server in the video already has an identical copy, so rsync only compares and does not re-copy the 12 GB; the first copy takes longer depending on network speed. The host names shown are `user@workstation` and `user@gpu-server`.

STEP 1
Create folders on the server
E.g. `~/shimizu-robot/models`

→

STEP 2
Copy the checkpoints
`rsync` from the workstation, resumable if interrupted

→

STEP 3
Check integrity
Compare checksums on both sides, all 3 weight files present

→

STEP 4
Copy the modality config
Arena version for the apple and hammer scenarios

```
# run on the workstation
SRV=user@gpu-server
CKPTS="hammer_to_bin_mix200 hammer_to_bin_mix400 real2sim_multi_fruit"
cd ~/workspace/shimizu-robot/models

# step 1 — create folders on the server
ssh $SRV 'mkdir -p ~/shimizu-robot/models ~/shimizu-robot/configs'

# step 2 — copy the needed checkpoints, keeping the subfolders
rsync -avP $CKPTS $SRV:~/shimizu-robot/models/

# step 3 — compare checksums without copying (-n):
# no files listed means both sides are identical
rsync -avnc $CKPTS $SRV:~/shimizu-robot/models/
# count weight files: expect 9 (3 checkpoints × 3 files)
ssh $SRV 'ls ~/shimizu-robot/models/*/*/checkpoint-*/*.safetensors | wc -l'
```

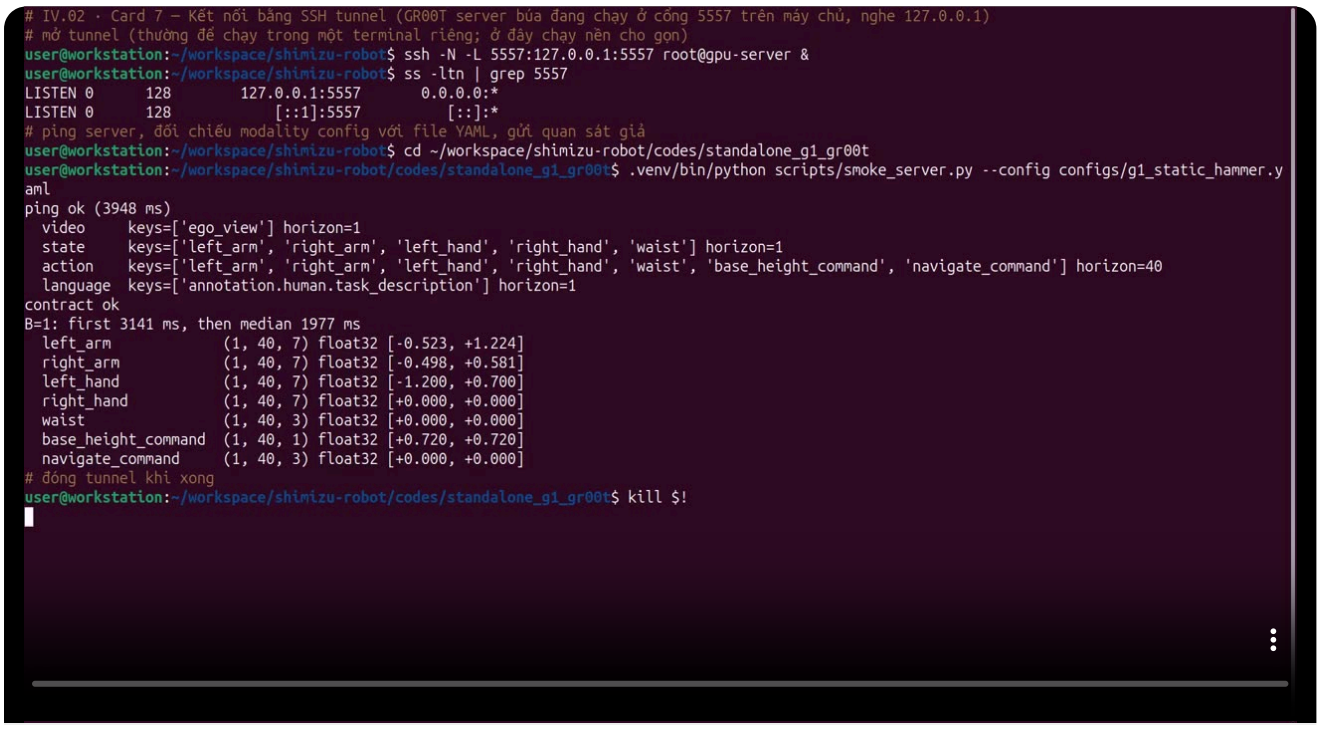
```
# step 4 — Arena modality config (apple, hammer scenarios)
G1=~/IsaacLab-Arena/isaaclab_arena_gr00t/embodiments/g1
rsync -avP $G1/g1_sim_wbc_data_gr00t_n_1_7_config.py \
      $SRV:~/shimizu-robot/configs/
```

FOLDER IN `MODELS/`	`--MODEL-PATH` ON THE SERVER
`hammer_to_bin_mix200/`	`~/shimizu-robot/models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000`
`hammer_to_bin_mix400/`	`~/shimizu-robot/models/hammer_to_bin_mix400/hammer_mix400_n17/checkpoint-20000`
`real2sim_multi_fruit/`	`~/shimizu-robot/models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/checkpoint-20000`

No network transfer possible? Copy the `models/` folder on an external drive and place it on the server with the same structure; still do step 3 using `sha256sum` on the `safetensors` files on both sides. The NVIDIA apple → plate checkpoint is not in the package: download it directly from Hugging Face to the server.

7 Set up the connection

Choose one of two options. The server port must match the port the client expects (card 5); the client port can be changed with the `NB10_PORT` or `NB12_PORT` environment variables or the `server.port` field in the YAML file.



Sample video: opening an SSH tunnel to the hammer server (port 5557), checking the port and pinging the server with `smoke_server.py`. The host names shown are `user@workstation` and `user@gpu-server`.

OPTION 1

Local network

- The server listens on `--host 0.0.0.0`
- Open the scenario's port in the firewall for the workstation only
- The client sets the host to the server IP, e.g. `NB12_HOST=10.0.0.5`

OPTION 2

SSH tunnel

- The server listens on `--host 127.0.0.1`, no port exposed
- The workstation opens the tunnel and keeps it running throughout the evaluation
- The client keeps the default host `127.0.0.1`

```
# option 2, on the workstation (e.g. hammer scenario, port 5557)
ssh -N -L 5557:127.0.0.1:5557 user@gpu-server
```

IV.03 Run the GR00T server

One checkpoint and one port per scenario.

The GR00T server runs on the client's GPU server, loads the checkpoint of the scenario to be verified and listens on a port. The workstation connects to that port (over the local network or an SSH tunnel, section IV.02 card 7) before running the notebooks in section IV.04.

COMMAND
gr00t/eval/run_gr00t_server.py

PORT
5556 (apple, real2sim) · 5557 (hammer)

CHECK
Port listening · ping from the workstation

1 Server configuration per scenario

The paths below follow the layout from section IV.02 card 6 (root `~/shimizu-robot/` on the server). The port is the default value the client expects.

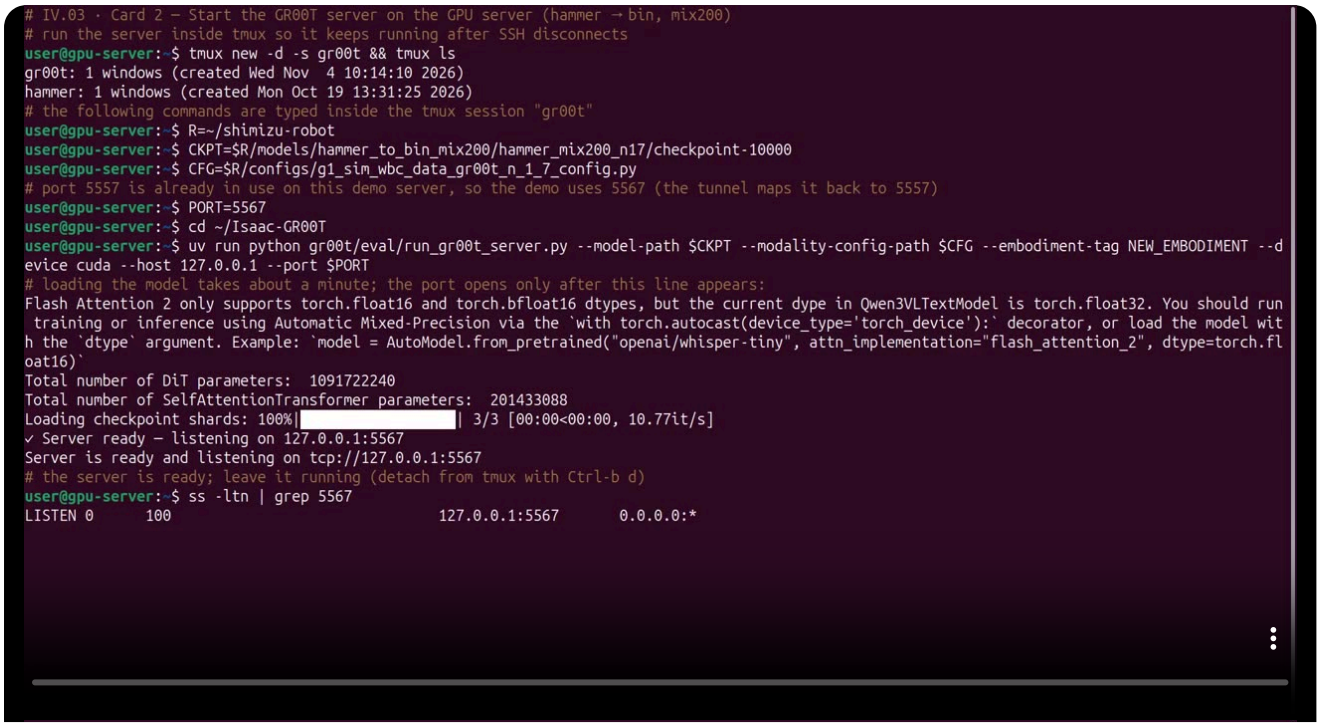
SCENARIO · PORT	CHECKPOINT (`--MODEL-PATH`)	MODALITY CONFIG
Apple → plate 5556	`models/gn1x_tuned_static_apple` (downloaded from Hugging Face)	`configs/g1_sim_wbc_data_gr00t_n_1_7_config.py`
Hammer → bin, mix200 5557	`models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000`	same as above
Hammer → bin, mix400 5557	`models/hammer_to_bin_mix400/hammer_mix400_n17/checkpoint-20000`	same as above
Real2sim 4 fruits 5556	`models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/checkpoint-20000`	`models/real2sim_multi_fruit/g1_fruits_n17_bs32_20k/g1_dex3_joint_sim_config.py`

> Apple and real2sim share port 5556, so only one of them can run at a time. Hammer uses port 5557 and can run in parallel if the GPU has enough memory.

```
# download the NVIDIA apple → plate checkpoint to the server (once)
hf download nvidia/GN1x-Tuned-Arena-G1-Static-PickNPlace \
   --local-dir ~/shimizu-robot/models/gn1x_tuned_static_apple
```

2 Start the server

Run inside `tmux` so the server keeps running when SSH disconnects. The example below is the hammer mix200 scenario; for other scenarios, change only the first three variables according to card 1.



Sample video: running the hammer mix200 server in tmux on the server and waiting for the `✓ Server ready` line (the model loading part is sped up). The server in the video was already using port 5557 for something else, so the sample server listens on port 5567; the tunnel in card 3 maps it to port 5557 on the workstation.

```
tmux new -s gr00t                  # detach: Ctrl-b d · reattach: tmux a -t gr00t
R=~/shimizu-robot
CKPT=$R/models/hammer_to_bin_mix200/hammer_mix200_n17/checkpoint-10000
CFG=$R/configs/g1_sim_wbc_data_gr00t_n_1_7_config.py
PORT=5557

cd ~/Isaac-GR00T
uv run python gr00t/eval/run_gr00t_server.py \
  --model-path $CKPT --modality-config-path $CFG \
  --embodiment-tag NEW_EMBODIMENT --device cuda \
  --host 127.0.0.1 --port $PORT    # 0.0.0.0 if the workstation connects directly over the local network
```

LOADING MODEL
About 1 minute
Port not open yet; a client call now will time out

→

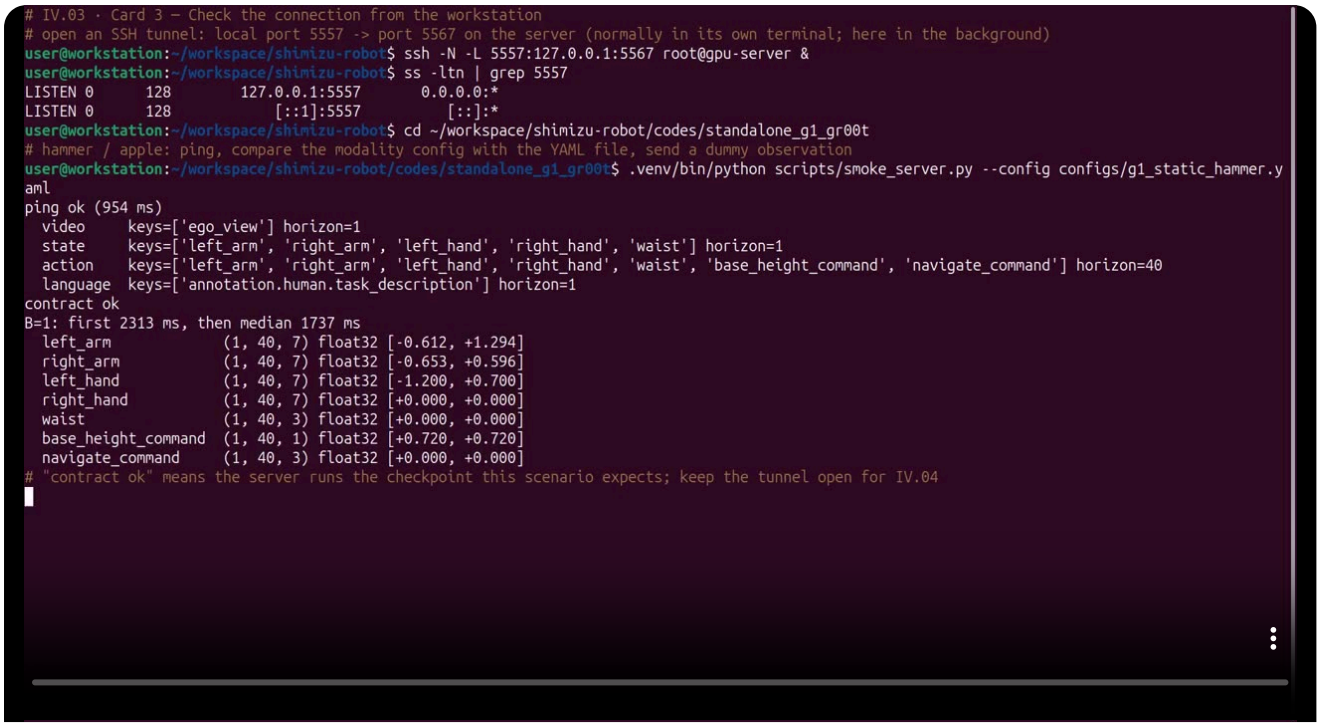
READY
Prints the ready line
`✓ Server ready — listening on …`

→

SWITCH SCENARIO
Ctrl-C and run again
With the new checkpoint, config and port

3 Check the connection from the workstation

If you use an SSH tunnel, open it in a separate terminal and keep it running throughout the evaluation. Then ping the server with the Python of the `.venv` environment.



Sample video: opening the SSH tunnel, checking port 5557 on the workstation and running `smoke_server.py` with the hammer configuration (result `contract ok`).

```
# (if using an SSH tunnel) separate terminal, keep it running
ssh -N -L 5557:127.0.0.1:5557 user@gpu-server

# ping the server and query the loaded modality config
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t
# apple, hammer: ping, compare the modality config with the YAML file, send a dummy observation
.venv/bin/python scripts/smoke_server.py --config configs/g1_static_hammer.yaml
#   (apple: configs/g1_static_apple.yaml)
# real2sim: ping and print the modality config loaded on the server
.venv/bin/python -c "from g1gr00t.client import Gr00tZmqClient, modality_summary as m
c = Gr00tZmqClient('127.0.0.1', 5556); print(c.ping(), m(c.get_modality_config()))"
```

SYMPTOM	CAUSE	FIX
Timeout on the first call	Tunnel not open, wrong port, or the server is still loading the model	`ss -ltn | grep 555` on both sides; wait for the ready line
Connection lost midway	Tunnel or network dropped	Reopen the tunnel and re-run the notebook; nb10 and nb12 resume from the unfinished episode
Notebook reports a wrong video key or joint count	The server has another scenario's checkpoint loaded	Restart the server with the correct checkpoint and modality config from card 1
Each call takes > 3 s	GPU busy or slow network	Does not affect results (the simulation pauses while waiting for the model), only takes longer

IV.04 Run the verification

Re-run the notebooks and compare with the reported figures.

Every scenario is evaluated with notebooks in `codes/standalone_g1_gr00t`, running directly on the local Isaac Sim, **without Docker Arena**. Each notebook has a `make` command to run it without a GUI (headless) and write the results to `data/`.

FOLDER
codes/standalone_g1_gr00t

RUN
make sim-nbXX (headless) · Jupyter (interactive)

OUTPUT
data/: per-episode logs, summary, videos

1 General procedure

01 · SERVER
Load the right checkpoint
Section IV.03, cards 1–2

→

02 · CONNECT
Open the tunnel if needed
Keep it running throughout the evaluation

→

03 · CHECK
Ping the server
Section IV.03, card 3

→

04 · RUN
The scenario's make command
Card 3 below

→

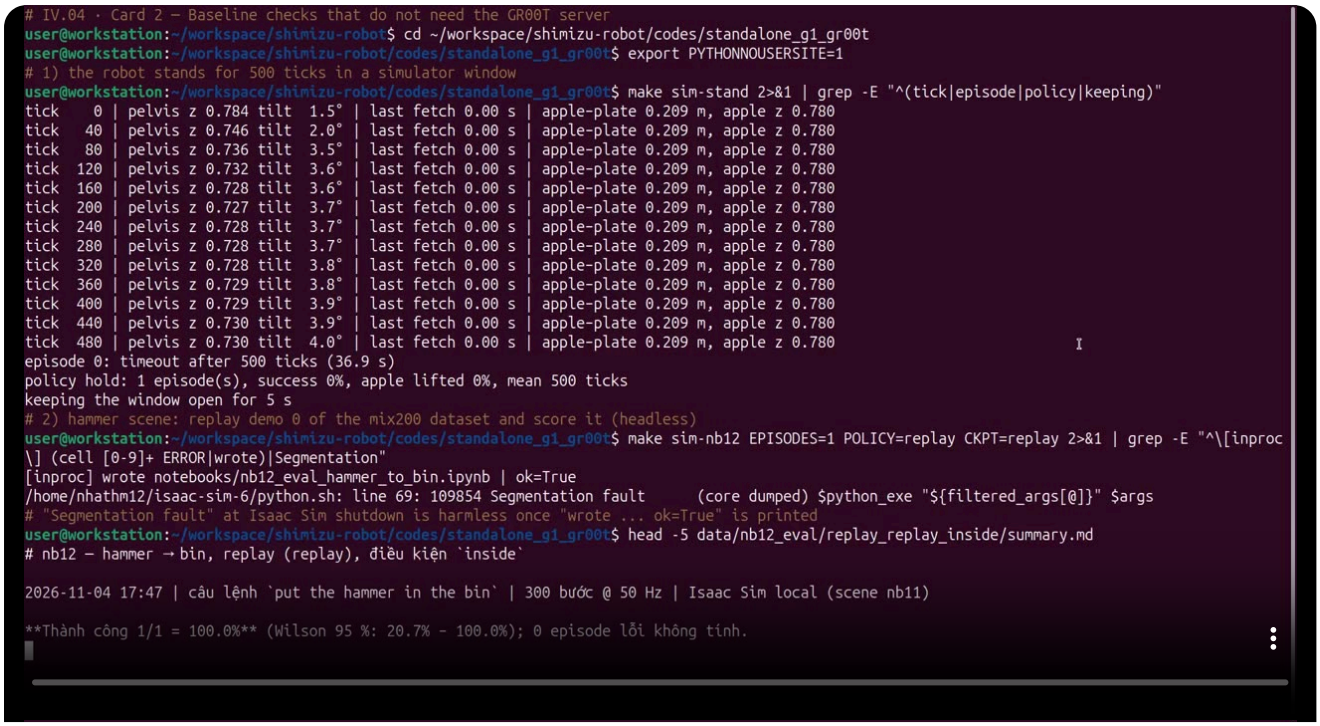
05 · COMPARE
Read the summary
Compare with the confidence interval (card 4)

Results shipped in the package will be overwritten: `make sim-nbXX` writes new output into the `.ipynb` file itself; nb07 also appends to `data/runs_galileo.jsonl` and overwrites the videos `data/videos/gr00t_galileo/epN.mp4`. Back up `notebooks/` and `data/` before running, and rename `runs_galileo.jsonl` so the summary covers only the new run. The original results are also in `evals/`.

Error when Isaac Sim shuts down: at the end of each run you may see `Segmentation fault` and `make: *** Error 1`. If the line `[inproc] wrote notebooks/… | ok=True` appeared before that, the notebook finished and the results were fully written.

2 Baseline checks without a server

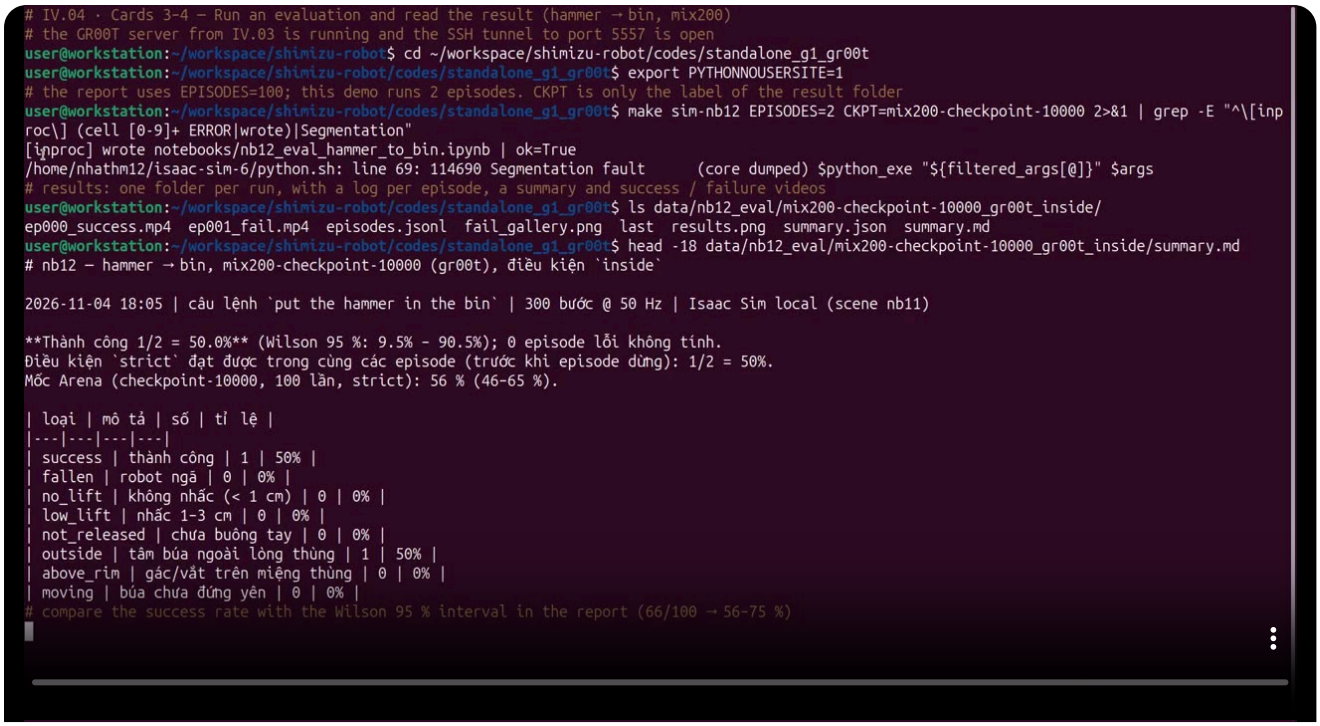
Confirm that the simulation, robot, scene and scoring work correctly before spending time running the model.



Sample video: `make sim-stand` opens the simulation window and the robot stands for 500 ticks; then demo 0 of the hammer dataset is replayed (headless) and the summary reads 1/1. The simulation parts are sped up.

COMMAND	WHAT IT DOES	PASSES WHEN
`make sim-stand`	Robot stands for 500 ticks, with a window	The robot stands stably without falling
`make sim-replay`	Replays demo episode 0 of the apple dataset, with a window	The robot picks up the apple and places it on the plate
`make sim-nb12 EPISODES=1 POLICY=replay CKPT=replay`	Hammer → bin scene, replays demo 0 of the mix200 dataset	The summary reports 1/1 success

3 Verification commands per scenario



Sample video: running nb12 (hammer → bin, mix200) against the server from section IV.03, then viewing the results folder and `summary.md` as in card 4. The video runs only 2 episodes to keep it short (result 1/2); the figures in the report use 100 episodes. The simulation part is sped up.

```
cd ~/workspace/shimizu-robot/codes/standalone_g1_gr00t

# Apple → plate, Galileo scene (apple server, port 5556)
make sim-nb07 EPISODES=50

# Real2sim 4 fruits (real2sim server, port 5556)
make sim-nb10 EPISODES=50

# Hammer → bin (hammer server, port 5557); CKPT is only a label for the results folder name
make sim-nb12 EPISODES=100 CKPT=mix200-checkpoint-10000
make sim-nb12 EPISODES=100 CKPT=mix400-checkpoint-20000
```

SCENARIO	NOTEBOOK · TIME	RESULT IN THE REPORT
Apple → plate, Galileo	`nb07_full_pipeline_loop` ~45 s / episode	**56 %** · 28/50 · Wilson 42–69 %
Real2sim 4 fruits	`nb10_eval_multi_object` ~1–2 min / episode	**66 %** · 33/50 · Wilson 52–78 %
Hammer → bin, mix200	`nb12_eval_hammer_to_bin` ~15–30 s / episode	**66 %** · 66/100 · Wilson 56–75 %
Hammer → bin, mix400	same as above	**73 %** · 73/100 · Wilson 64–81 % 63 % · 63/100 with Arena's at-rest threshold

Hammer at-rest threshold: nb12 currently treats the hammer as at rest when its velocity is < 0.02 m/s and its angular velocity < 0.12 rad/s (variables `LIN_VEL_TH`, `ANG_VEL_TH` in §3). The 73 % result was measured with this threshold. To reproduce 63 % (Arena's original threshold, 0.01 m/s and 0.1 rad/s), change those two variables in the notebook and run again with a different `CKPT` label.

Not covered by the commands above: the 92 % for the apple scenario and the 56/100 for the hammer in Arena were measured with the original Arena client in Docker, not with the standalone notebooks.

4 Read and compare the results

NOTEBOOK	RESULTS FOLDER	WHERE THE SUCCESS RATE IS
nb07	`data/runs_galileo.jsonl`, videos in `data/videos/gr00t_galileo/`	Summary table at the end of the notebook
nb10	`data/nb10_eval/` (change with `NB10_OUT`)	`seed0_….summary.json`
nb12	`data/nb12_eval/<CKPT>_gr00t_inside/`	`summary.json`, `summary.md`

MATCHES THE REPORT

Within the confidence interval

Each episode is randomized; with 50 episodes a difference of about ±13 percentage points is normal, with 100 episodes about ±9 points.

TECHNICAL ERRORS

Not counted in the rate

Failed episodes (server timeout, lost connection) are logged with the reason and excluded from the success rate; the notebook stops if the server does not respond again.

RESUME

No need to start over

nb10 and nb12 skip episodes already present in the results folder; restart the server and re-run the same command.

5 Run interactively in Jupyter

To watch each step or the simulation window, open the notebook in Jupyter instead of running headless.

```
make sim-kernel && make nb     # Jupyter Lab, select the "Isaac Sim Python 3 (isaacsim6)" kernel
```

ISSUE	FIX
`No module named 'isaaclab'`	You are on the `.venv` kernel: switch to the `isaacsim6` kernel
`print` output does not show in the cell	Isaac Lab reassigns `sys.stdout`; see the output in the terminal running Jupyter
Hangs when closing the simulation	Do not call `app.close()`; use Kernel ▸ Restart

IV.05 Video evaluation with VSS

Let a vision-language model watch the video and judge the result.

NVIDIA VSS (Video Search & Summarization) is a toolkit that analyses video with a vision-language model (VLM). This section shows how to install VSS 3.2.1 on the GPU server, then use the **Isaac Sim → VSS bridge** to have VSS watch hammer → bin episode videos and decide success or failure, compared with Arena's verdict. Every step has its commands, the expected result and a video of the real session.

VERSION
VSS 3.2.1 · LVS profile · 1× H100

3 WAYS TO JUDGE
Video summarization · Q&A · Real-time alert

CONCLUSION
Real-time alert, `top_k: 1` : 94.2 % (to be confirmed)

1 What VSS does in the project

Arena scores an episode from the simulation state (where the hammer and the bin are). A real robot has no such state, only video. VSS was tried to answer one question: **looking only at the video, does the VLM reach the same verdict as Arena?**

01 · EPISODE
Video + Arena verdict
Hammer → bin scenario (section III.03)

→

02 · EXPORT
15 fps, 480p
Last frame held for 1.5 s so the VLM sees the end state

→

03 · VSS
VLM watches the video
Cosmos3 nano reasoner on the GPU; Nemotron LLM via build.nvidia.com

→

04 · VERDICT
Success / failure
From events, a YES/NO answer or an alert

→

05 · COMPARE
With Arena's verdict
Card 5

Containers
26
17 always running · 9 only run at start-up

GPU for the VLM
~44 GB
Cosmos3 model 16.3 GB · vLLM capped at 0.5 of GPU memory

Disk
~80 GB
Docker images + model; the largest is 42.9 GB

Install

~12 min

install and first model load

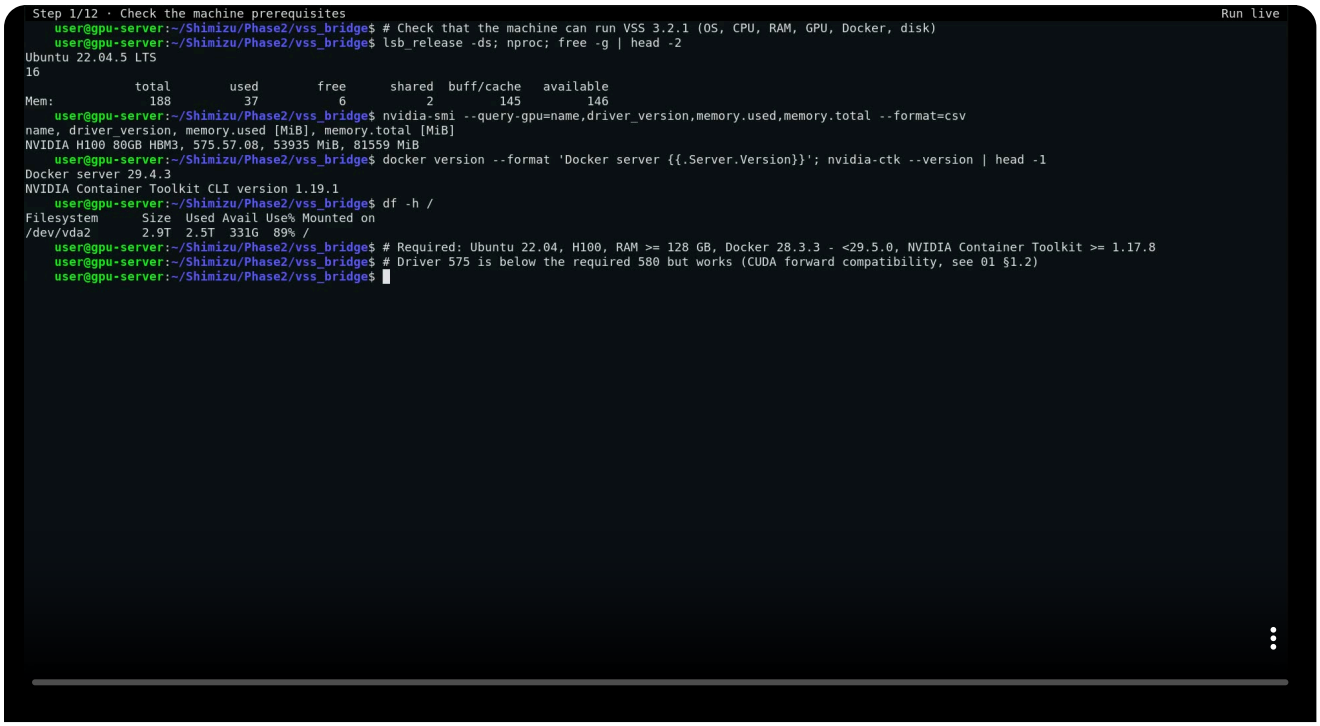
Guide and videos: the folder `vss_guide/` holds the step-by-step guide `07_step_by_step_guide.md` and 12 videos, one per step, each opening with a 5-second title card. The videos use two modes: **Run live** (the commands really run during the recording) and **Check only** (the step was done at install time and re-running it would do harm, so the video only checks its result). In the videos in this report, the server name and IP address are replaced with `user@gpu-server` and `<server-ip>`.

2 Server requirements

Compare the server with the VSS 3.2.1 requirements before installing. The middle column is the machine used for the recordings.

REQUIREMENT	MACHINE AT RECORDING TIME	NOTE
Ubuntu 22.04 or 24.04	Ubuntu 22.04.5 LTS	OK
CPU ≥ 18 cores	16 cores	Slightly short, still runs
RAM ≥ 128 GB	188 GB	OK
H100 GPU	H100 80 GB HBM3	Shared GPU: VSS ~44 GB, the rest for GR00T and other work
Driver 580.65.06	575.57.08	Not upgraded; works through CUDA forward compatibility
Docker 28.3.3 to below 29.5.0	29.4.3	OK
NVIDIA Container Toolkit ≥ 1.17.8	1.19.1	OK
Free disk	~331 GB	The install takes ~80 GB; Elasticsearch needs more than 15 GB free

```
lsb_release -ds; nproc; free -g | head -2
nvidia-smi --query-gpu=name,driver_version,memory.used,memory.total --format=csv
docker version --format 'Docker server {{.Server.Version}}'; nvidia-ctk --version | head -1
df -h /
```



Step 1 · Run live: check the OS, CPU, RAM, GPU, driver, Docker, Container Toolkit and free disk.

3 Installing VSS 3.2.1

Five steps, in this order. Steps 3 and 6 need root and interrupt a shared server, so an administrator does them at an announced time.

02 · KEYS
NGC Personal Key
Log in to nvcr.io →

03 · DOCKER
Downgrade to 29.4.3
Administrator · hold the version →

04 · SOURCE
Clone v3.2.1
Apply 4 patches →

05 · TRIAL
Dry-run
Only in a temporary clone →

06 · INSTALL
Pull images, start
26 containers, 16.3 GB model

```
# step 2: keys in their own file, mode 600; never paste them into chat or commit them
mkdir -p ~/.config/vss && umask 077
cat > ~/.config/vss/keys.env <<'EOF'
export NGC_CLI_API_KEY='nvapi-...'
export NVIDIA_API_KEY='nvapi-...'
EOF
bash -c 'source ~/.config/vss/keys.env; echo "$NGC_CLI_API_KEY" | docker login nvcr.io -u "\$oauthtoken'

# step 3 (administrator): Docker inside the range NVIDIA validated
VER=5:29.4.3-1~ubuntu.22.04~jammy
sudo apt-get install -y --allow-downgrades docker-ce=$VER docker-ce-cli=$VER
sudo apt-mark hold docker-ce docker-ce-cli

# step 4: exactly release 3.2.1, then apply the patches
```

```
git clone --branch v3.2.1 https://github.com/NVIDIA-AI-Blueprints/video-search-and-summarization.git vss
cd vss && git apply /path/to/local-patches.diff

# step 6: install (remote LLM, VLM on the server's GPU)
set -a && source ~/.config/vss/keys.env && set +a
export LLM_ENDPOINT_URL=https://integrate.api.nvidia.com RTVI_VLLM_GPU_MEMORY_UTILIZATION=0.5
bash deploy/docker/scripts/dev-profile.sh up -p lvs -H H100 -i <server-ip> \
  --use-remote-llm --llm nvidia/nemotron-3.5-lightning-30b-a3b
```

#	FILE	PATCH (LOCAL-PATCHES.DIFF)
1	`dev-profile.sh` , `state_down`	Remove only dangling volumes with the `mdx_` prefix, never other users' volumes
2	`dev-profile.sh` , `state_up`	Change kernel settings (disables IPv6 host-wide) only when `VSS_APPLY_KERNEL_SETTINGS=true`
3	`dev-profile.sh` , `state_up`	Keep a pre-set `RTVI_VLLM_GPU_MEMORY_UTILIZATION` (0.5 instead of 0.7) to leave GPU memory for GR00T
4	`vss-agent/configs/config.yml`	The agent calls the LLM through the `openai` client, fixing the 400 `verify_ssl` error

`dev-profile.sh up --dry-run` **is not harmless:** it still overwrites `generated.env` (the file with the keys and the LLM settings) and runs `chmod -R 777` in `data-dir` . Only dry-run in a temporary clone (step 5 video). Also, `dev-profile.sh up` always runs `down -v` first, which **deletes the 17.8 GB model volume**; never re-run it on an installed machine.

4 Post-install configuration and health checks

Two problems only show up at the first request, so fix them right after installing: the Elasticsearch disk watermarks (the default needs ~150 GB free) and the way the agent calls the LLM (patch 4, then `docker restart vss-agent`).

```
# Elasticsearch disk watermarks: safe to run again
curl -s -X PUT 'http://127.0.0.1:9200/_cluster/settings' -H 'Content-Type: application/json' -d '{"persi
  "cluster.routing.allocation.disk.watermark.low":"15gb",
  "cluster.routing.allocation.disk.watermark.high":"10gb",
  "cluster.routing.allocation.disk.watermark.flood_stage":"5gb",
  "cluster.routing.allocation.disk.watermark.flood_stage.frozen":"5gb"}}'

# generated.env holds the keys: grep only the variable names you need, never cat it
E=../vss/deploy/docker/developer-profiles/dev-profile-lvs/generated.env
grep -E '^(LLM_MODE|VLM_MODE|LLM_NAME|VLM_NAME|RTVI_VLLM_GPU_MEMORY_UTILIZATION)=' $E
```

CHECK	PASSING RESULT
Containers of project `mdx`	17 `Up` (`healthy` where there is a healthcheck), 9 `Exited (0)`
LVS service, port 38111	`200` ; model `nim_nvidia_cosmos3-nano-reasoner_bf16-final`
VLM, port 8018, "Say OK"	Answers `OK` , not `EngineDeadError`
Agent, port 8000, "hi"	The agent's greeting, after about 17 s
Elasticsearch, port 9200	`green` ; watermarks 15 / 10 / 5 GB
Kafka, topic `mdx-vlm-incidents`	8 partitions, alert count readable
GPU	`VLLM::EngineCore` process ~43.7 GB
Web UI, port 7777	`200`
Bridge `tests/smoke_test.sh`	`SMOKE TEST PASSED` (mocks, no VSS call)

Common failures: the VLM engine dies while the healthcheck still says `healthy` → `docker restart vss-rtvi-vlm` . Kafka is down → `docker start kafka` . Elasticsearch is not `green` → usually the disk is full.

5 Three ways to judge, and the results on 99 episodes

The bridge has three programs, one per VSS mechanism. Each reads a YAML config and writes a per-episode result (`epNNN.json`) and a summary table `report.md` .

METHOD	HOW IT WORKS	TIME	DECODING
Video summarization (LVS) `vss_bridge.run_pipeline`	Sends the video to `/v1/summarize` (port 38111); the VLM writes a summary and an event timeline, and the bridge reads the verdict from the events with a strict rule	~22 s / episode	Default, with random sampling
Q&A `vss_bridge.qa_agent`	The agent (port 8000) asks the VLM: "when the video ends, is the hammer fully inside the bin? YES/NO"	~82 s / episode	Default, with random sampling
Real-time alert (RT alert) `vss_bridge.rt_alert`	Each episode sits in a fixed 8 s slot; the VLM (port 8018) answers "is there a failure? Yes / No"; Yes = alert = failure	~0.7 s after each 8 s segment of a live stream	Default; the fix: `top_k: 1`

The evaluation used **99 hammer → bin episodes** with Arena verdicts: 13 episodes to tune the question and how the result is read, and **86 held-out episodes** used only for scoring. The baseline is always answering "success": right on 63/86 (73.3 %), because most episodes succeed. A method is only useful if it beats this baseline.

Why the fixed 8 s slots matter: successful episodes stop the moment success is reached, so their clips are shorter. In video summarization and Q&A, clip length alone would have predicted the outcome of all 86 episodes. The real-time alert gives every episode the same length, so it has no such leak; this is part of why its results are the fairest test.

METHOD · 86 HELD-OUT EPISODES	AGREEMENT WITH ARENA	SUCCESSES RECOGNISED	FAILURES RECOGNISED	BEATS THE
Video summarization	52/86 (60.5 %)	56 %	74 %	No
Q&A	51/86 (59.3 %)	51 %	83 %	No
Real-time alert, 5 runs with the same config	74–80 %	83–89 %	35–65 %	No, the ru
Real-time alert, `top_k: 1`	**81/86 (94.2 %)**	**100 %**	**78 % (18/23)**	**Yes (p < 0 measured outcomes known**
Baseline: always answer "success"	63/86 (73.3 %)	100 %	0 %	–

Why the results varied (confirmed): with the same config and the same frames, real-time alert runs disagreed on 28 of 86 episodes; one clip sent eight times got "Yes" five times and "No" three times. The real-time VLM service passes `temperature` to the model only when it is not zero, so `temperature 0` is dropped and the model samples at its default of 1.0. Adding `top_k: 1` forces deterministic decoding: two runs then agreed on all 99 episodes, and agreement with Arena rose to 94.2 %, recognising every success and 18 of 23 failures.

94.2 % is a strong lead, not yet a result. The question was unchanged, but the run was made after the held-out outcomes were known. On the 13 tuning episodes it scored only 8 of 13 and missed both robot falls, and the held-out set contains no falls at all. It must be confirmed on a fresh run. Video summarization and Q&A suffer from the same decoding defect and have not been re-run with `top_k: 1`, so the comparison between the three methods is not yet fair.

- **Q&A** called 31 of 63 successes failures, with answers such as "the hammer is partially inside the bin", although the final frames and Arena's measurements of the two groups are nearly identical. Likely causes: the shallow bin floor and the word "fully" in the question.
- **The VSS agent rewrote the question** before passing it to the VLM in 46 of 99 cases, so the question actually asked was not fixed.

6 Re-running the three methods on 2 episodes (step 9–11 videos)

The guide records the three commands on two episodes: ep003 (Arena: success) and ep025 (Arena: failure, hammer outside the bin). Each episode was run several times on the same (bit-identical) video with the same config: one stored earlier run, the Vietnamese recording and the English recording; Q&A has one more run, in the UI demo video.

```
cd <vss_bridge folder>
python3 -m vss_bridge.run_pipeline --config configs/guide_en_lvs.yaml \
  --inputs ../data/W8/vss_inputs/ep003.mp4 ../data/W8/vss_inputs/ep025.mp4      # video summarization
python3 -m vss_bridge.qa_agent --config configs/guide_en_qa.yaml --episodes ep003,ep025   # Q&A
python3 -m vss_bridge.rt_alert --config configs/guide_en_rt.yaml --episodes ep003,ep025   # alert, top_k
cat out/guide_en/{lvs,qa,rt}/report.md
```

EPISODE · ARENA	VIDEO SUMMARIZATION	Q&A	REAL-TIME ALERT
ep003 success	success, success, **failure** 2/3 correct	NO, YES, NO, YES 2/4 correct	No, No, No **3/3 correct**
ep025 failure (hammer outside the bin)	empty → failure; "hammer fully inside the bin" → **success**; "hammer never lifted" → failure 2/3 correct, no correct description	NO every time correct every time	Yes, Yes, Yes **3/3 correct**

The recordings agree with the full evaluation in card 5: video summarization and Q&A change their answers from run to run, while the real-time alert with `top_k: 1` gives the same answer every time. The times in the videos (summarization 19–21 s, Q&A 52–54 s, alert 0.3–0.45 s per episode) belong to the recordings, not to the full evaluation.



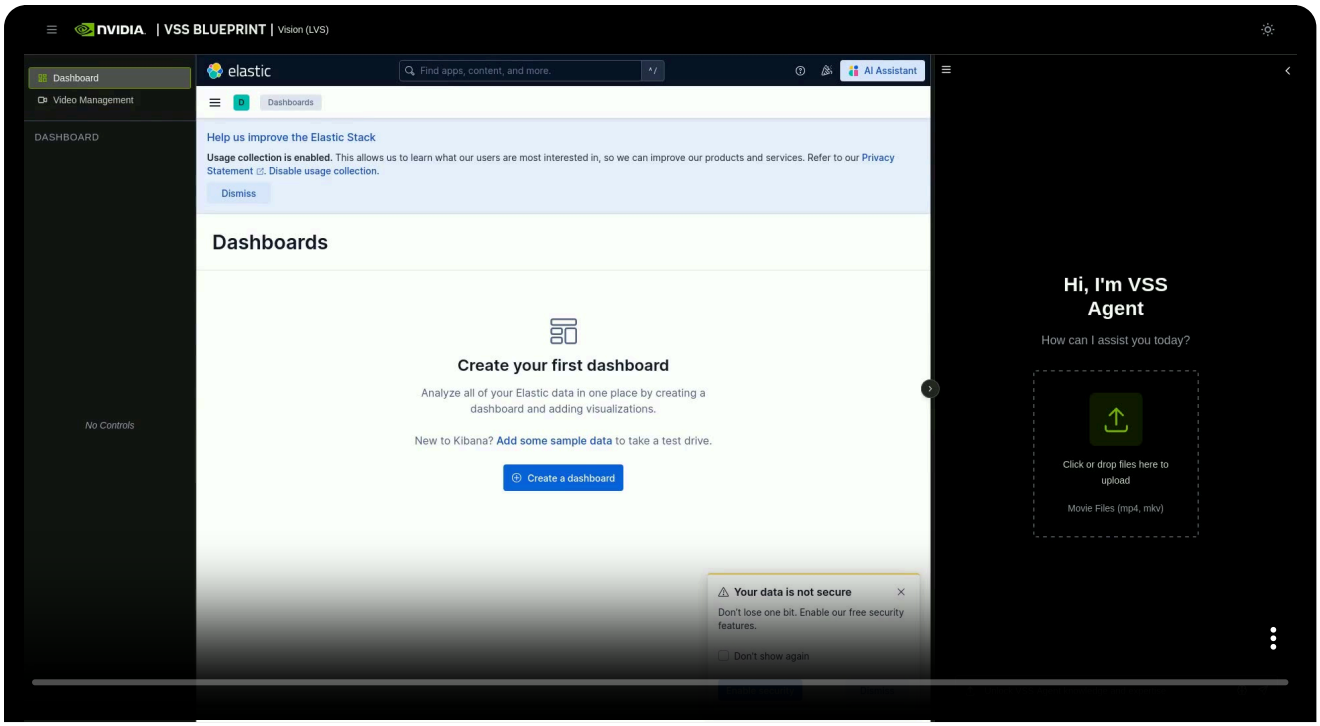
Step 9 · Run live: summarize ep003 and ep025, read `report.md` and the prompt the VLM actually received. The 43 s wait is fast-forwarded ×6.

7 Web UI

The UI on port 7777 is for watching uploaded videos and asking the agent by hand. It only accepts the server's internal address or `localhost`, so open it from a workstation through an SSH tunnel:

```
ssh -N -L 7777:127.0.0.1:7777 user@gpu-server     # then open http://localhost:7777
```

TIME IN THE VIDEO	SCREEN
0:05	**Dashboard**: Kibana embedded in the page, no dashboard created yet
0:11	**Video Management**: 99 uploaded videos; upload a video, add an RTSP stream, search, a "+ Chat" button on each video
0:17–0:21	Search for `ep003` and play it: two half-frames (robot head camera, third-person view)
0:34	Open the **Chat** panel: "Hi, I'm VSS Agent"



Step 12 · Run live: a tour of the web UI. This video sends no question.

8 Findings and actions

#	FINDING	ACTION	WHO
1	`dev-profile.sh --dry-run` still overwrites `generated.env` and changes permissions in `data-dir`	Only dry-run in a temporary clone (step 5)	Anyone running the script
2	`docker-ce` and `docker-ce-cli` are not held now; an `apt upgrade` can move Docker out of the validated range	`sudo apt-mark hold docker-ce docker-ce-cli`	Administrator
3	The VLM service drops `temperature 0` and samples at 1.0: the same video gets different answers	Send `top_k: 1` in all three methods; re-run video summarization and Q&A with `top_k: 1`	Tech lead
4	The 94.2 % was measured after the held-out outcomes were known; the held-out set has no robot falls	Confirm with a run on fresh episodes, including robot falls	Tech lead
5	The agent rewrote the question in 46 of 99 cases; Q&A called 31 of 63 successes failures	Fix the question sent to the VLM; define "inside the bin" clearly instead of the word "fully"	Tech lead
6	The VLM engine had been dead since 19 October while the healthcheck still said `healthy`; a full disk stopped Kafka and made Elasticsearch read-only	Monitor with a real VLM call and disk usage, not only the healthcheck	Administrator

Conclusion: with the current configuration, video summarization and Q&A do not beat always answering "success" (73.3 %). The real-time alert with `top_k: 1` reached 94.2 % on 86 held-out episodes, fast (under 1 s after each 8 s segment) and stable: this is the direction to pursue. Before VSS replaces or supports Arena's scoring, it needs a confirmation run on fresh episodes, and the other two methods need re-running with `top_k: 1`.

9 Day-to-day operation

Run these in `vss/deploy/docker`, with `E=developer-profiles/dev-profile-lvs/generated.env`.

TASK	COMMAND
Read logs	`docker logs --tail 200 vss-lvs` (or `vss-rtvi-vlm`, `vss-agent`)
Stop / start the whole stack, keeping data	`docker compose --env-file $E stop` / `start`
Free the GPU	`docker stop vss-rtvi-vlm`; start again with `docker start vss-rtvi-vlm`
After a Docker daemon restart	`docker start kafka phoenix` (these two do not restart on their own)
Change the LLM model	Edit `LLM_NAME` in `$E`, then `docker compose --env-file $E up -d --no-deps --force-recreate lvs-server vss-agent`
Regular checks	Call the VLM (the VLM row of the card 4 table) and `df -h /`. The healthcheck does not notice a dead VLM; a full disk stops Kafka and makes Elasticsearch read-only (79 GB of old data had to be cleared, with approval)

Do not run `dev-profile.sh up` or `down` on an installed machine: it deletes the 17.8 GB model volume and needs sudo. **Do not** `cat` `generated.env` or `keys.env`, and do not run `docker compose config` or `docker inspect` without a `grep`: they print the keys to the screen.

V.01 Overall evaluation

Three scenarios, one overall picture.

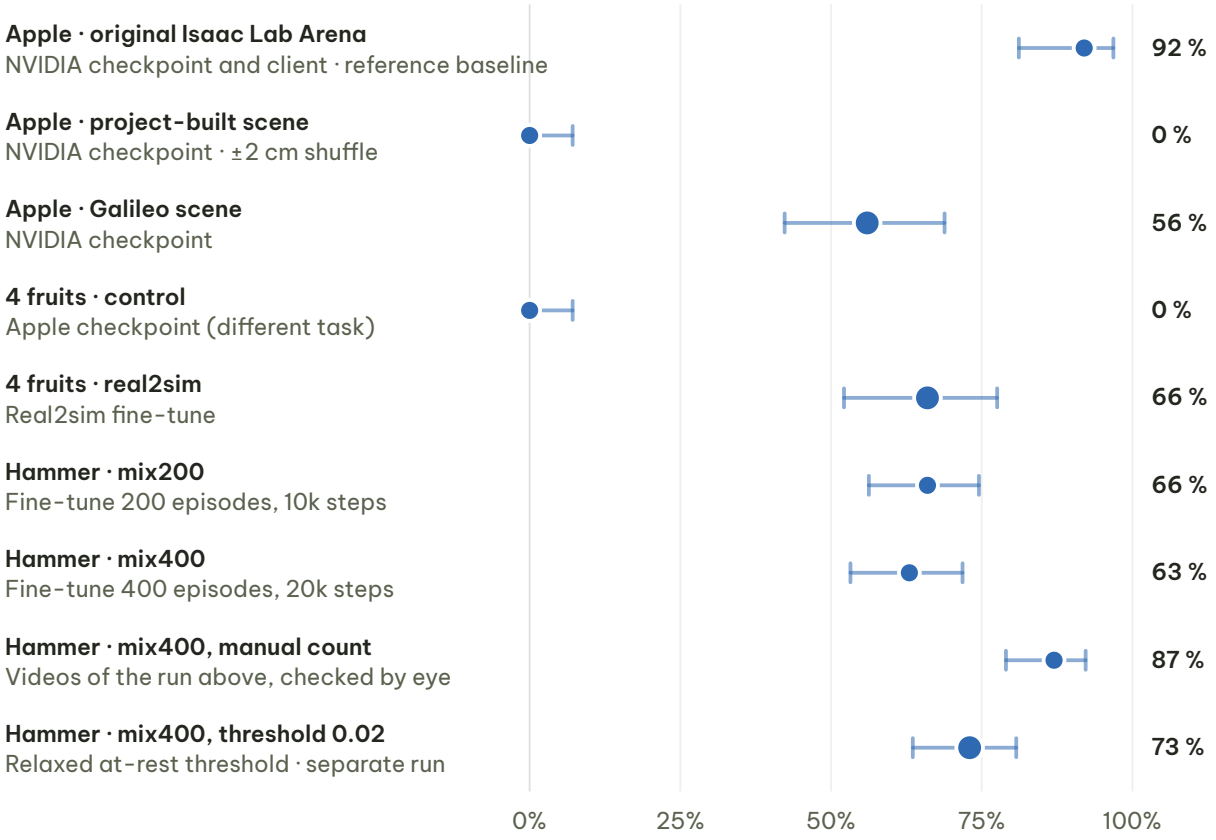
GR00T N1.7 can control G1 to pick and place objects in simulation: **56 %** on the apple task, **66 %** on the 4 fruits, **66–73 %** on the hammer task. Results depend first on the **visual appearance of the scene** and the **fine-tuning data**, and only then on how success is scored.

1 Summary of results

SCENARIO	BEFORE FIXES	BEST RESULT	DECIDING FACTOR	REMAINING BOTTLENECK
III.01 · Apple → plate	0 % (self-built scene)	**92 %** · 46/50 · original Isaac Lab Arena, NVIDIA client **56 %** · 28/50 · reconstructed Galileo	Scene matching the one used for data recording	Perception and approach phase: in an unfamiliar scene only 22 % lift the apple
III.02 · 4 fruits	0 % (apple checkpoint)	**66 %** · 33/50 · real2sim	Fine-tuning on 790 real2sim episodes	Grasp phase: 11/17 failures are failing to lift the fruit
III.03 · Hammer → bin	**0 % (not fine-tuned)**	**66 % · 66/100 · mix200** **73 % · 73/100 · mix400, threshold 0.02** **87 % · 87/100 · mix400, manual count from the videos**	**Self-collected data: teleop + generated**	**Release phase and the "at rest" scoring**

Success rate and 95 % Wilson confidence interval

Dot = measured rate; horizontal bar = confidence interval. Hover or Tab for details.



Note: the NVIDIA client on the original Isaac Lab Arena environment reached 92 % (46/50) on the apple task; that run is the reference baseline (first row of the chart). All other evaluations ran closed-loop on the project's own pipeline, on local Isaac Sim.

Video-based scoring (section IV.05): NVIDIA VSS was tried on 99 hammer → bin episode videos, judging success on its own. On 86 held-out episodes, video summarization (60.5 %) and Q&A (59.3 %) did not beat always answering "success" (73.3 %). After a decoding fix (`top_k: 1`), the real-time alert agreed with Arena on 94.2 %; that run was made after the outcomes were known, so it needs confirming on fresh data.

2 Overall findings

01 · VISUALS

The scene matters more than the model

With the same checkpoint and instruction, the apple task drops from 92 % (NVIDIA client, original Arena) to 56 % (only the lighting differs) and then to 0 % (the whole scene differs). A model fine-tuned on one scene does not generalize to another.

02 · FINE-TUNING

New tasks must be tuned, and tuning is fast

The fruit task scores 0 % when reusing the apple checkpoint, and the hammer task 0 % before tuning; both reach 66 % after fine-tuning. The hammer task needs only 200 episodes and ~1 hour on one H100; the main cost is the data.

03 · LANGUAGE

The model follows the instruction

0 wrong picks in 50 episodes of the fruit task, even though there are always other fruits and distractors on the table. The language part learns well from the real2sim data.

04 · BOTTLENECK

Each scenario falls short in a different phase

The apple falls short in perception and approach, the fruits in grasping, the hammer in releasing. Improvements therefore have to be task-specific; there is no single common fix.

05 · DATA

More data of the same kind has limits

Going from 200 to 400 episodes on the hammer task makes placement more accurate (hammer outside the bin interior 19 → 5 times) but the overall rate does not rise (66 % → 63 %, within the margin of error).

06 · SCORING

How success is scored affects the number

Hammer task: in 22/100 runs the hammer was already in the bin and the hand had let go, yet it was scored "not at rest" because the condition only checks instantaneous velocity. The run with the threshold relaxed to 0.02 m/s reached 73 %.

3 Lessons learned

01

Data & environment matter more than the model

Same checkpoint: 0 % or 56 % just because of the scene.

02

The instruction is part of the "contract"

Manage instructions like controlled configuration parameters.

03

Layered verification saves time

Open-loop → replay → closed-loop.

04

Success conditions need testing

Loose conditions pollute the data; overly strict conditions lower the metric. Agree on thresholds before comparing.

05

Evaluation needs large numbers

50–100 trials per configuration, always with a confidence interval.

06

Generating extra data is cheap

133 generation runs → 100 demos meeting the strict condition, with no extra teleop hours.

07

Fine-tuning is fast and cheap

~1 hour (200 episodes) to 2 h 15 (400 episodes) on 1 H100.

4 Future directions

This phase showed that GR00T N1.7 can drive the G1 to pick and place objects in simulation, and built a data → fine-tune → evaluation process for new tasks. The next steps fall into three groups: **research on models and sensing**, **closing the visual gap between simulation and reality**, then **preparing to run the model on the real robot** (phase 2).

01 · RESEARCH · MODELS

Research other, better models

Evaluate newer GR00T versions and other open-source VLA models (for example π0, OpenVLA) on the same three scenarios with the same scoring, to see where GR00T N1.7 stands. The standalone pipeline talks to the model through a ZMQ server, so a new model only needs its own server; the scene, runner and evaluation notebooks are reused as is.

02 · SIMULATION · IMAGES

Domain randomization of the scene

Randomize lighting, materials, background, camera and object positions when generating data (method in section III.01), so the model does not depend on one scene: the apple task drops from 92 % to 56 % from a lighting change alone. This is the groundwork for sim2real.

03 · SIMULATION · RECONSTRUCTION

3D Gaussian Splatting to reconstruct the real environment

Capture photos or video of the real workplace and reconstruct it with 3D Gaussian Splatting, used as the rendered background in simulation so that lighting and materials match reality; the apple task shows that a lighting change alone drops the rate from 92 % to 56 %. Combined with real2sim, training data would carry both real motion and real imagery. Gaussian Splatting only provides the images; physics still needs separate collision models for the table, shelf and objects.

04 · EVALUATION · VSS

Confirm video-based scoring

Re-run the real-time alert with `top_k: 1` on fresh episodes, including robot falls, to confirm the 94.2 %; re-run video summarization and Q&A with the same fix. If it holds, this is an automatic scorer for the real robot, where there is no simulation state (section IV.05).

05 · REAL ROBOT · DATA

A combined data process

A few teleoperated demos in the actual workplace → real2sim to rebuild the scene → generate more demos and filter them in simulation → domain randomization → fine-tune, evaluate in simulation and then on the real robot (section II.04). Two of the three techniques were validated in this phase.

06 · RESEARCH · SENSING

Depth maps to help the robot judge distance

The model only receives RGB images from the head camera, while many failures come from judging distance: apples released 9–20 cm from the plate, hammers falling outside the bin or resting on its rim. Research using depth maps from the RealSense D435 on the G1 head (or depth estimated from images) as an extra observation for the model, or to check the position before release; this needs data collection and fine-tuning with depth.

Priorities: 04 takes the least effort and makes every later figure more trustworthy; 02 and 03 both target the visual gap and are prerequisites before moving to the real robot; 01 and 06 are research directions to raise the model's capability; 05 is the core of phase 2. Every further evaluation should keep this phase's practice: 50–100 trials per configuration, always with a confidence interval.

Glossary & references.

For the list of deliverables, see section IV.01.

A Glossary

VLA	A model that takes images and language and directly outputs robot control commands
Foundation model	A model pre-trained on large data; fine-tuned for a specific task with little data
Embodiment	A specific type of robot body (joints, cameras, hands)
Action chunk	The sequence of actions the model returns in one inference (40 steps = 0.8 s at 50 Hz)
Flow matching / DiT	A technique that generates continuous data by gradual denoising, used to generate action chunks
Teleop	A person controlling the robot remotely (VR headset) to create demonstration data
Open-loop / closed-loop	Evaluation on pre-recorded data / the model controlling the robot continuously by itself
Real2sim	Transferring data from the real world into simulation
Domain gap	The difference between the training environment and the runtime environment
Visual gap	The model's success rate drops when the scene looks different at runtime than during data recording (apple task 92 → 56 → 0 %)
Domain randomization	Randomly varying lighting, materials, background and camera angle in simulation so the model does not memorize one scene
Demo generation	Transforming existing teleop demos (timing, start point, grasp, release), re-running them in simulation and keeping only the successful ones (the MimicGen approach)
FK / IK	Forward kinematics: compute the hand position from joint angles; inverse kinematics: compute joint angles from a desired position
PD controller	Proportional–derivative: torque = Kp × (target angle − current angle) − Kd × joint velocity; pulls the joint toward the target like a damped spring
Rate limiter	Limits how much a joint command may change from the previous command in each control step (20 ms), up to a threshold (arms 0.2 rad, waist 0.1 rad, hands 1.5 rad)

WBC / AGILE	Whole-body controller; AGILE keeps the G1's legs balanced
Wilson 95 %	A confidence interval for a success rate, suitable for small numbers of trials
MAE	Mean absolute error

B References

1. NVIDIA. *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots.* arXiv:2503.14734, 2025.
2. NVIDIA. Model cards for GR00T N1.7-3B and GN1x-Tuned-Arena-G1-Static-PickNPlace (Hugging Face).
3. Isaac Lab Arena — github.com/isaac-sim/IsaacLab-Arena
4. Isaac-GR00T — github.com/NVIDIA/Isaac-GR00T
5. NVIDIA. GR00T end-to-end workflow: fine-tuning on sim data, sim evaluation (docs.nvidia.com).
6. Datasets nvidia/Arena-G1-Static-PickNPlace-Task and nvidia/PhysicalAI-Robotics-GR00T-Teleop-G1 (CC-BY-4.0).
7. Black, K. et al. *π0: A Vision-Language-Action Flow Model for General Robot Control.* arXiv:2410.24164, 2024.
8. Liu, X., Gong, C. *Flow Straight and Fast: Rectified Flow.* ICLR 2023.